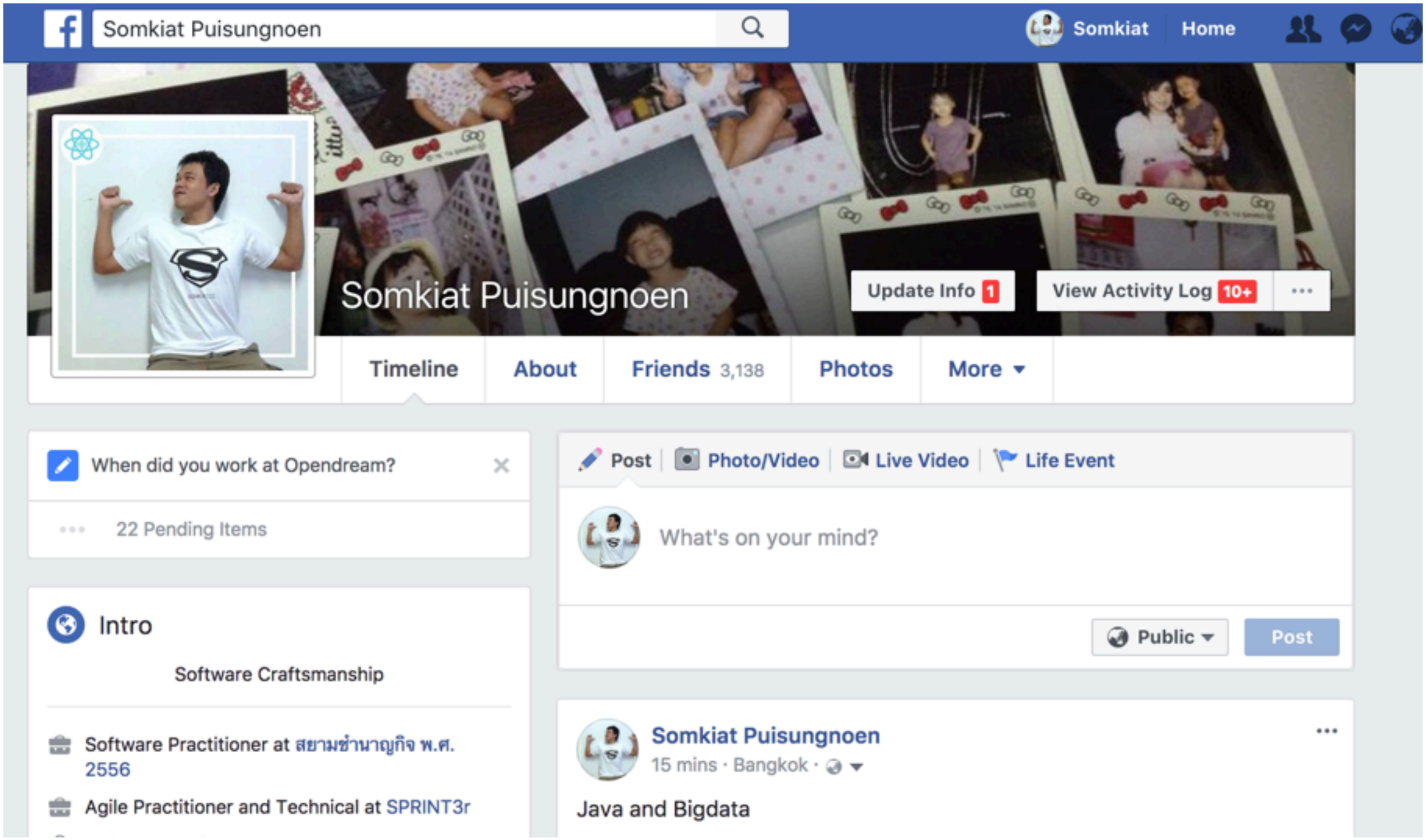
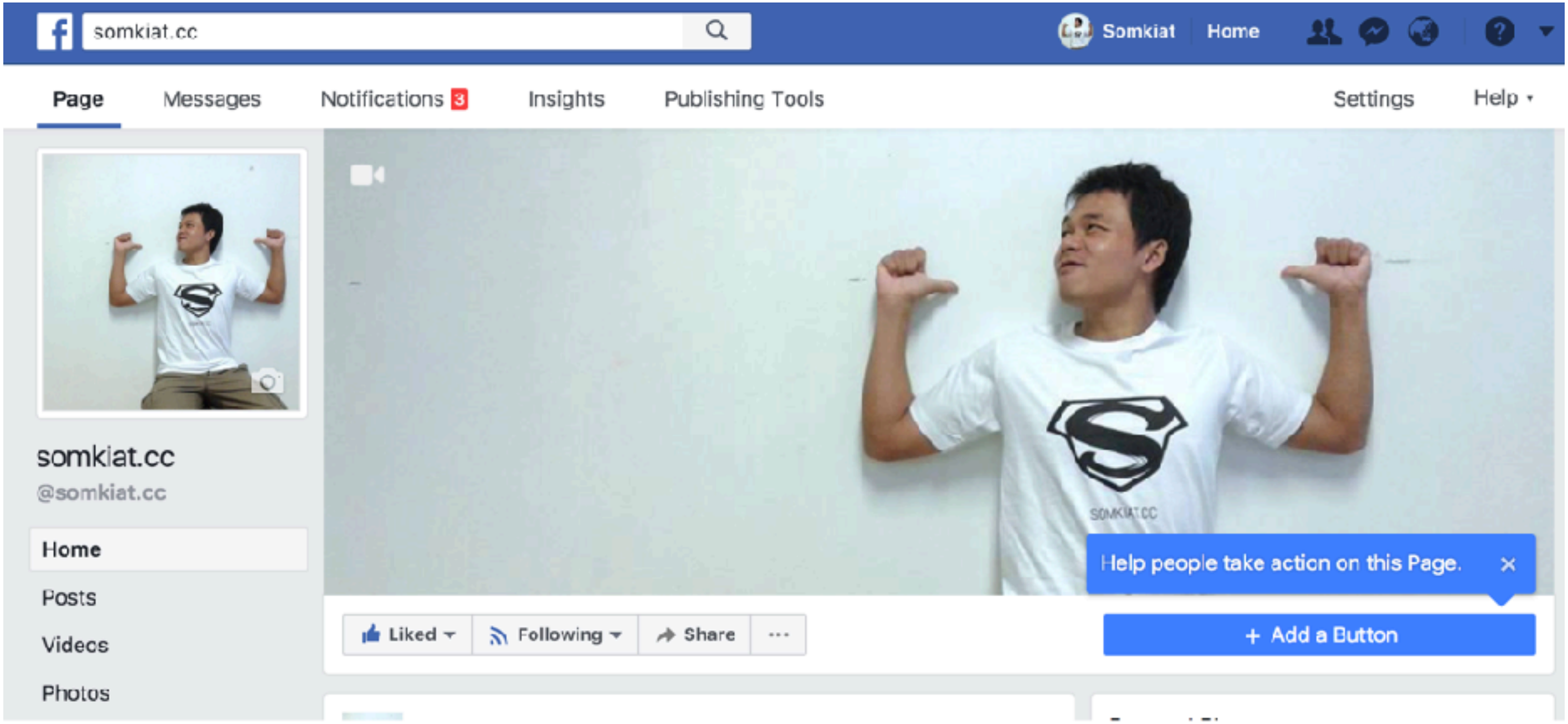
DevOps series

DevOps
Modern Software Development
Docker





DevSec**Ops**

Design | Develop | Test | Deploy

Monitoring and Observability

Continuous Integration and Delivery

Docker and Kubernetes

Section 1

1. Introduction to DevOps
2. DevOps workflow
3. DevOps tools
4. DevOps team topologies
5. Version Control System with Git
6. Manage containers with Docker
7. Docker image, container, Dockerfile, compose

Section 2

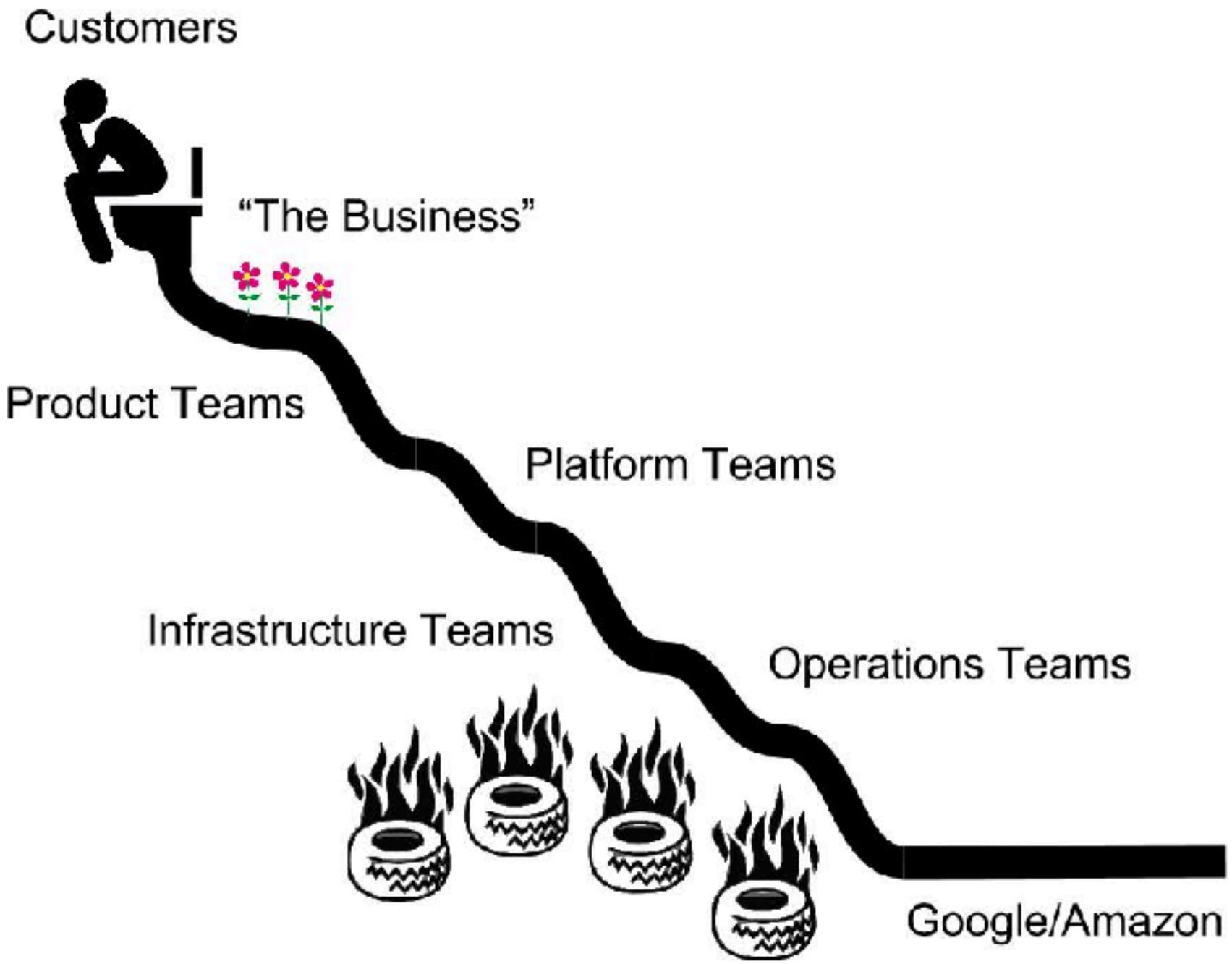
1. Continuous Integration and Delivery (CI/CD)
2. CI/CD principles
3. Design your pipeline
4. Working with Jenkins
5. Create your pipeline
6. Pipeline as a Code
7. Scalable Jenkins with Master/slave

https://github.com/up1/course-devops-workshop

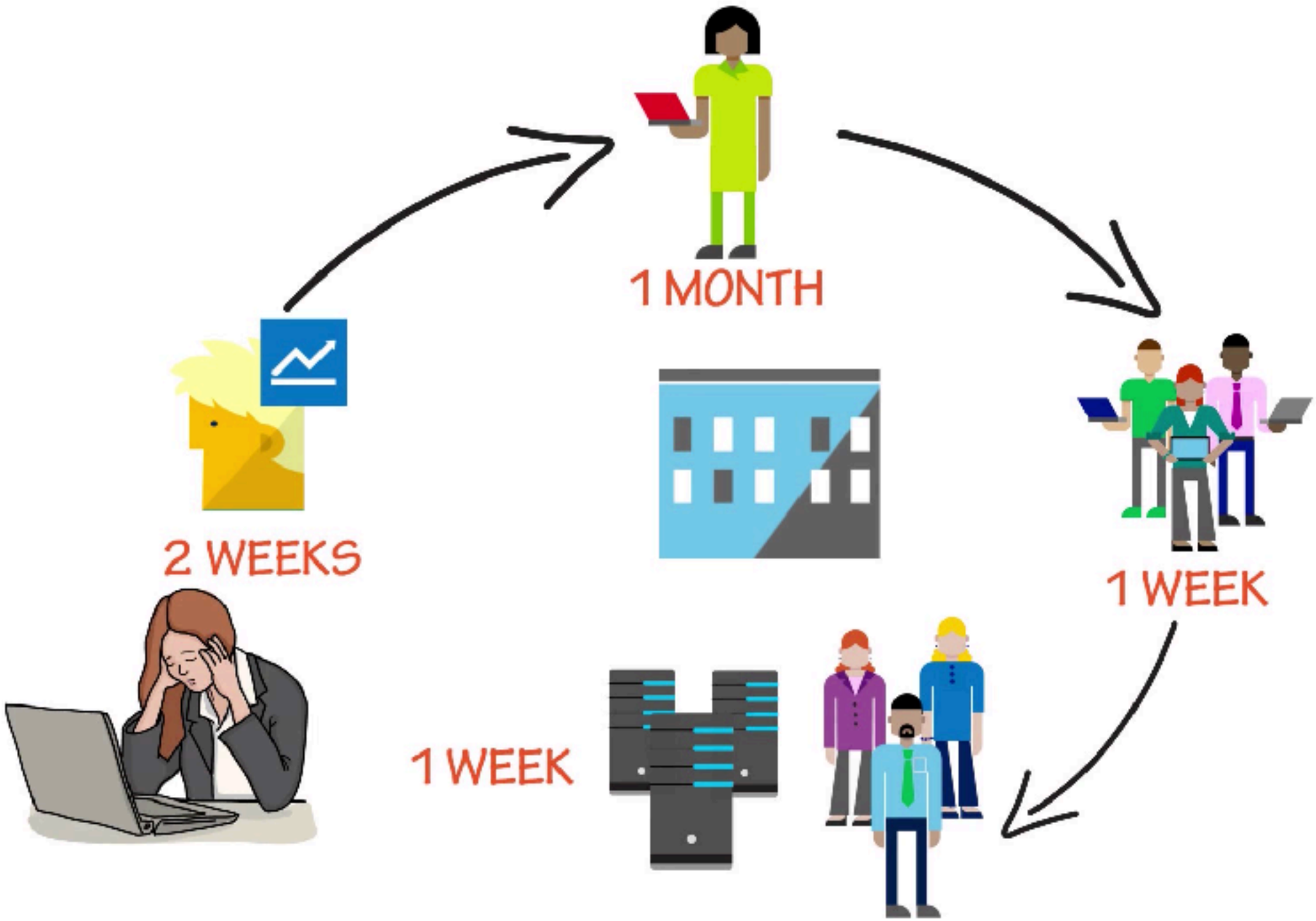
Software Delivery

Quantity vs Quality ?

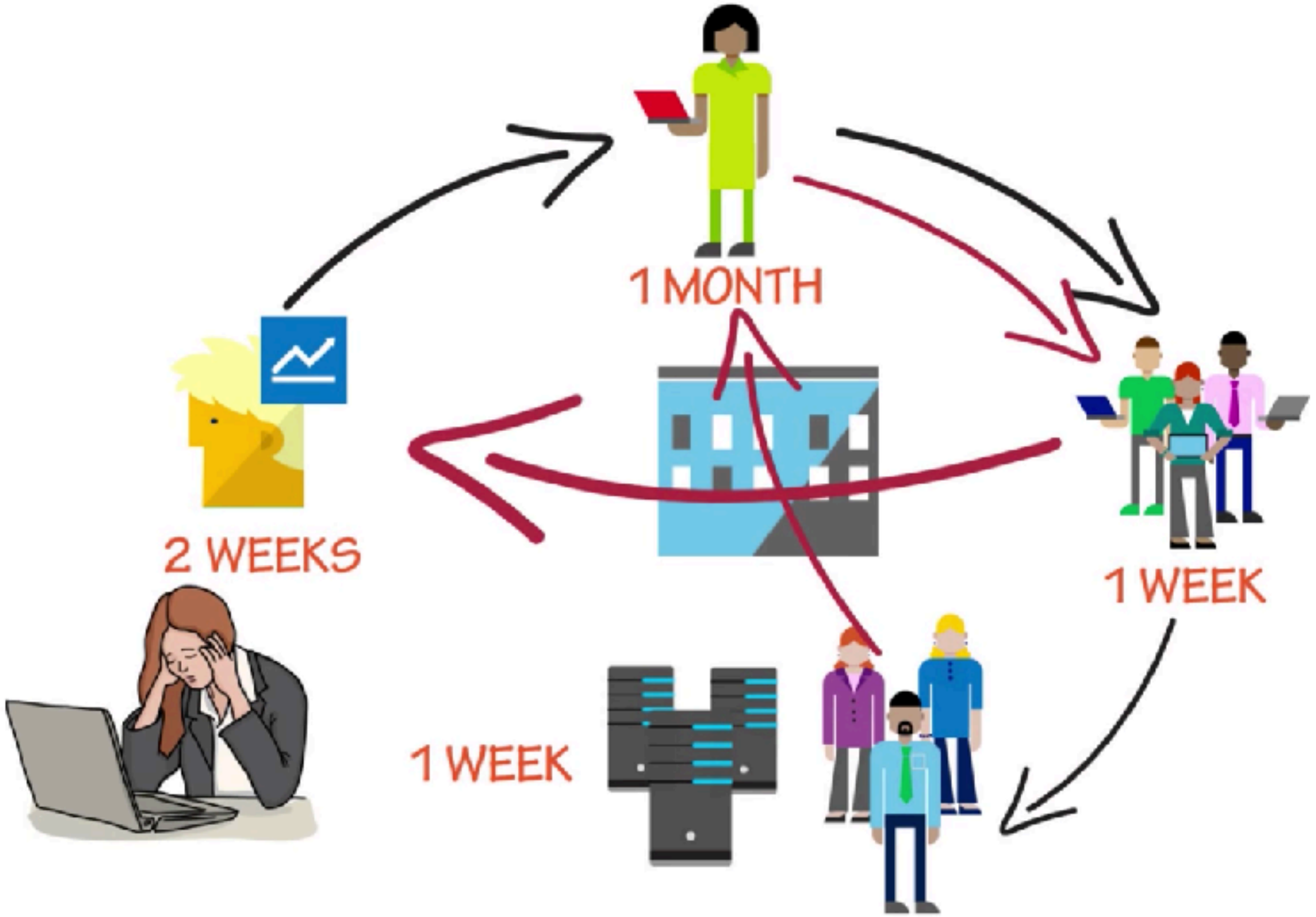
SDLC



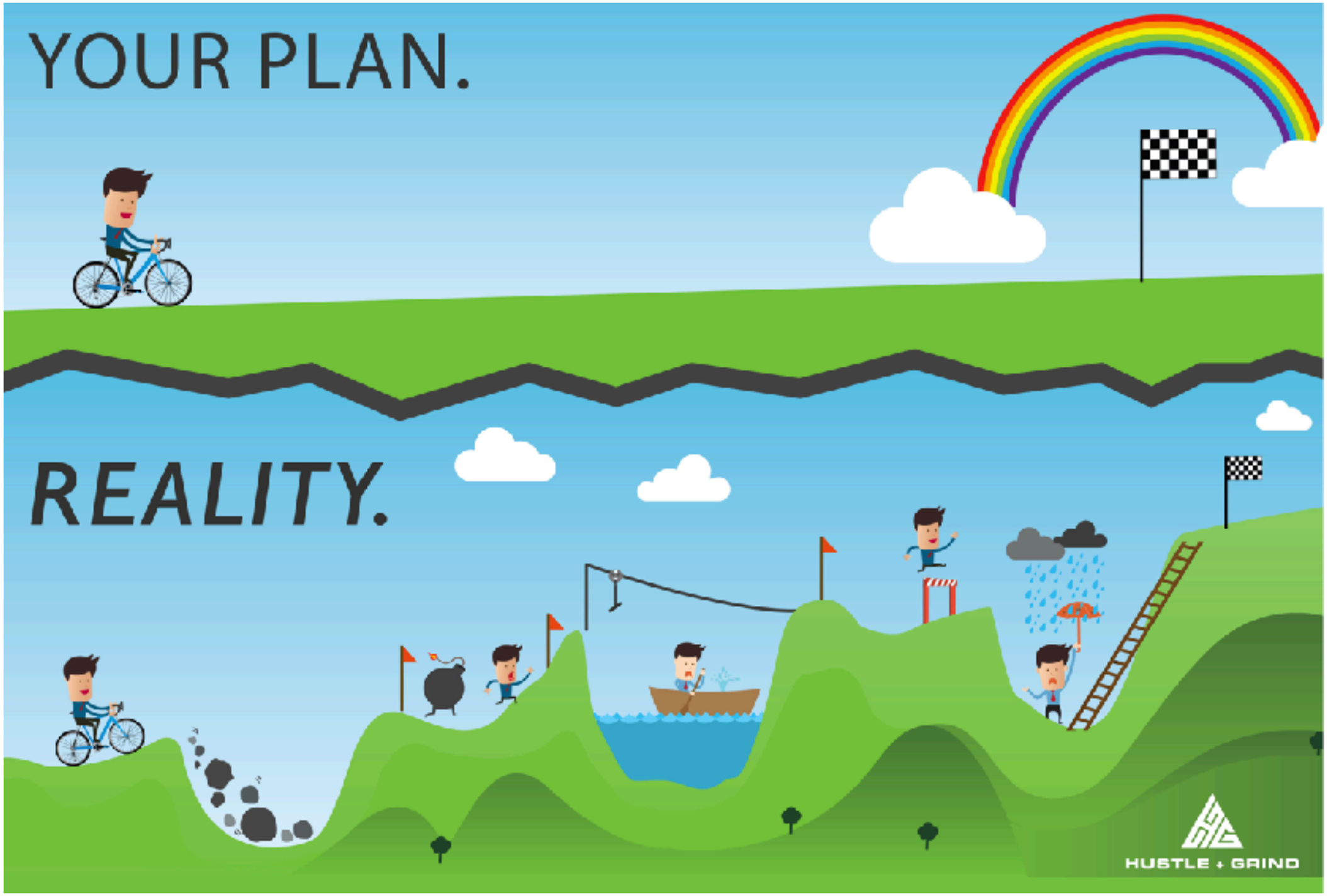
SDLC



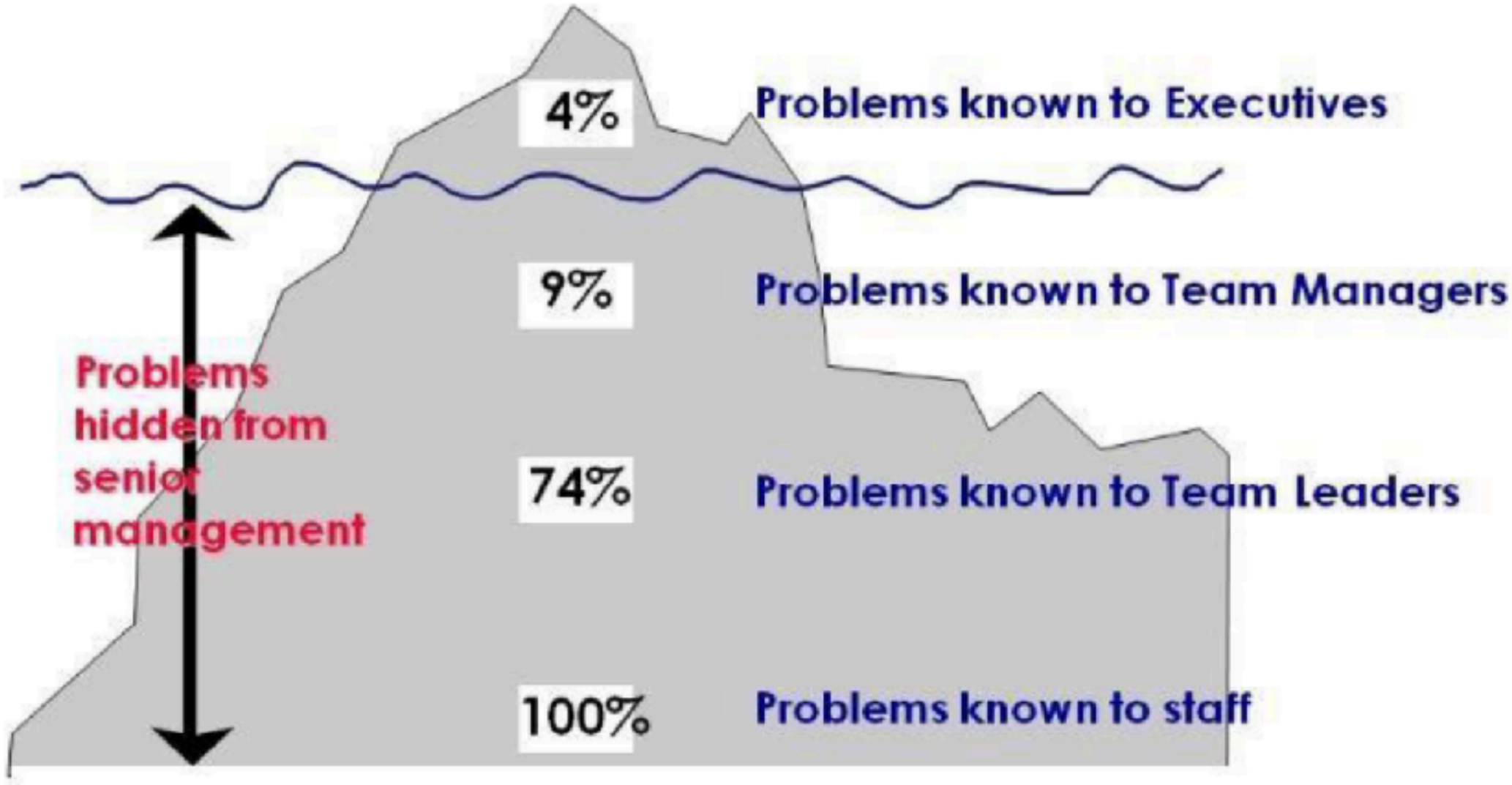
SDLC



Plan vs Reality



The iceberg of ignorance



Consequence of inefficient

Grow Fat

Code base grows. All the things slow down.

Age

Your code base will become a jurassic park introducing new tech becomes hard

Ownership

Who is responsible for which part and more important: who has the pager

Economies of Scale

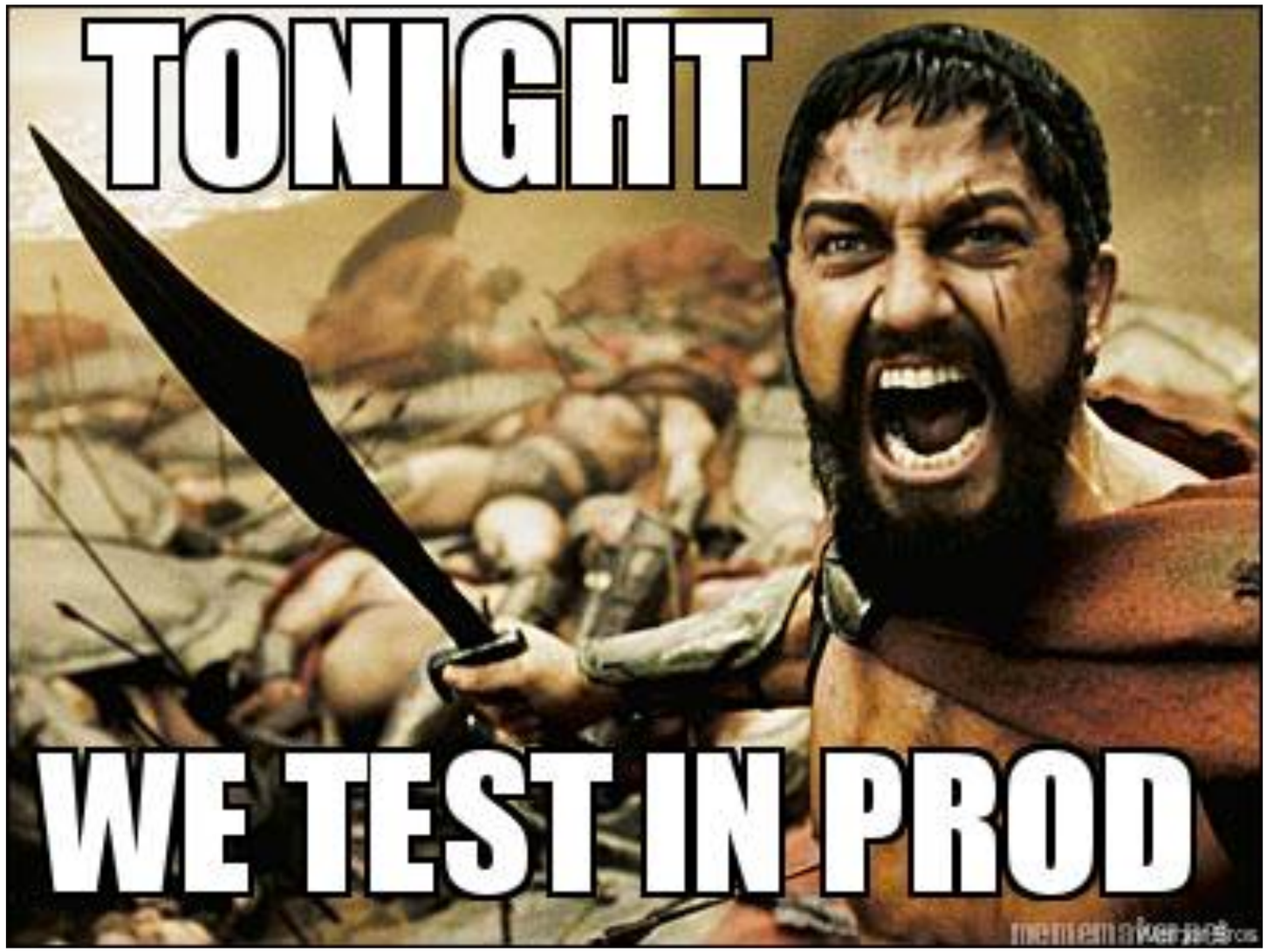
The bigger the team the more they interrupt each other

The Blame Game !!

Consequence of inefficient

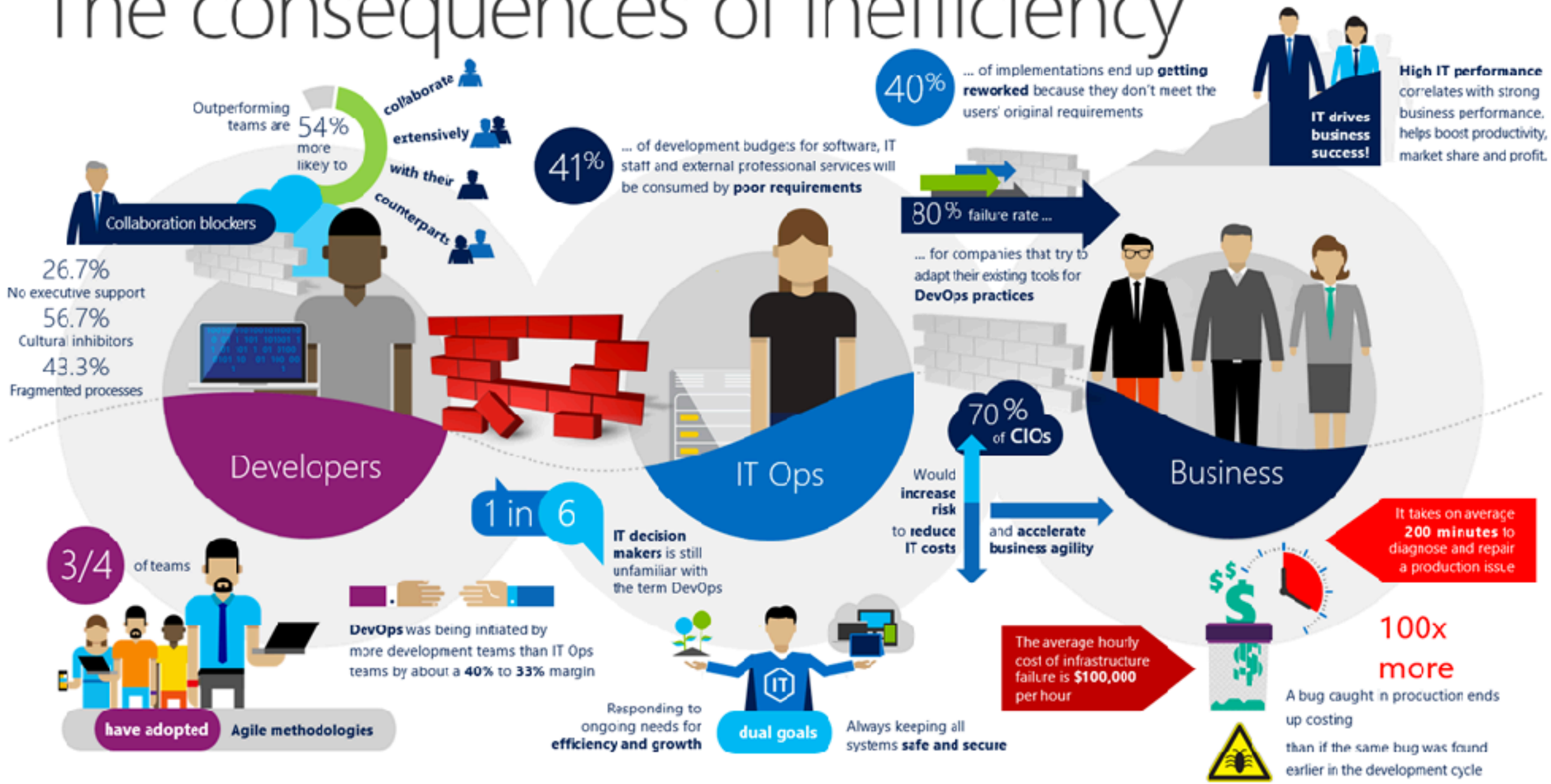
Consequence of inefficient

Consequence of inefficient

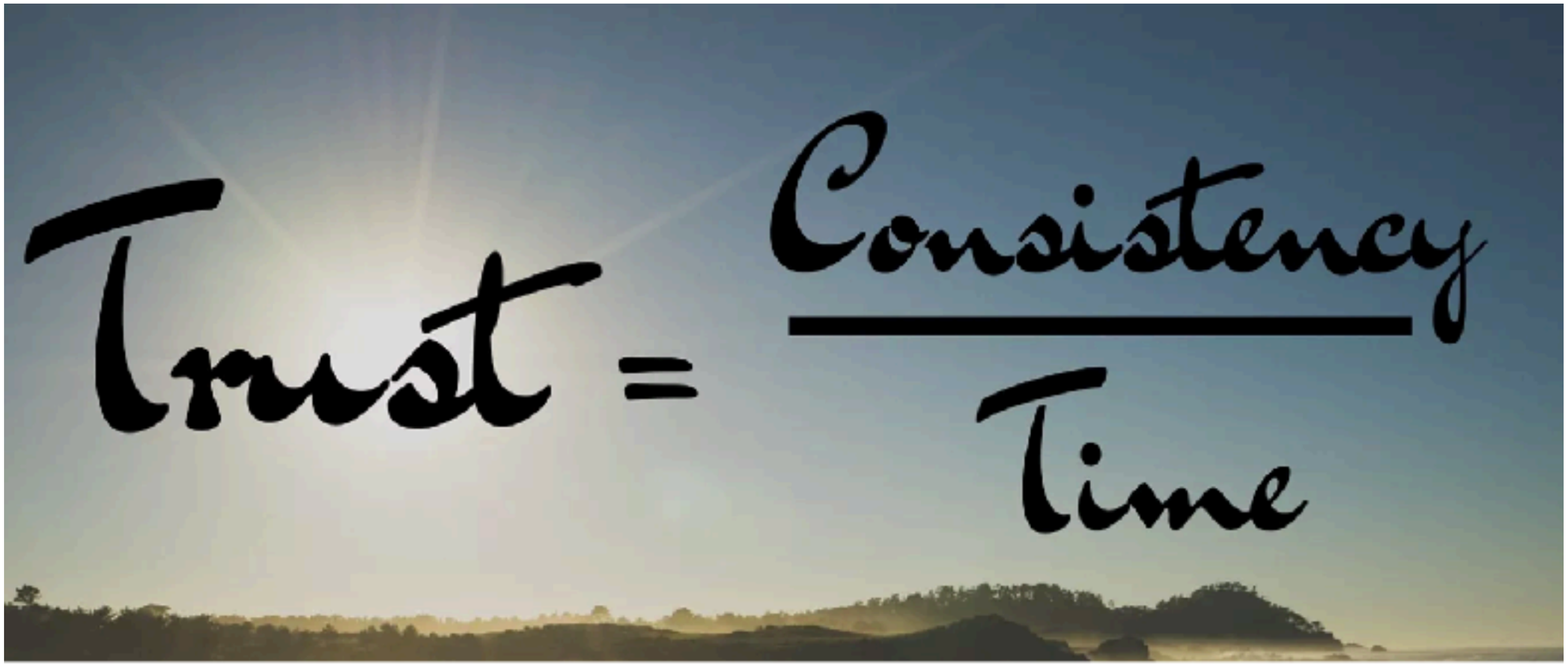


Consequence of inefficient

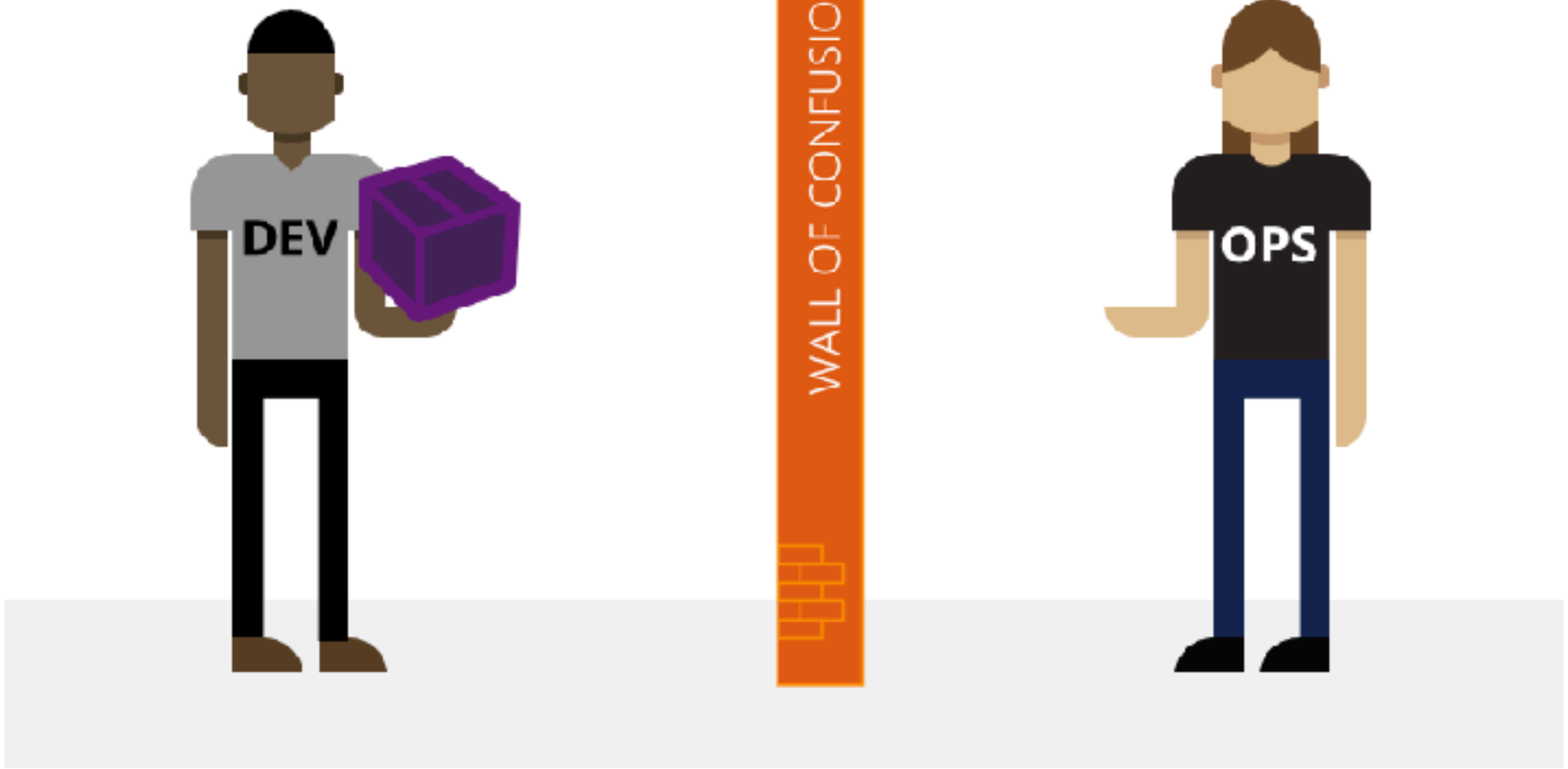
The consequences of inefficiency



https://channel9.msdn.com/Series/DevOps-Fundamentals

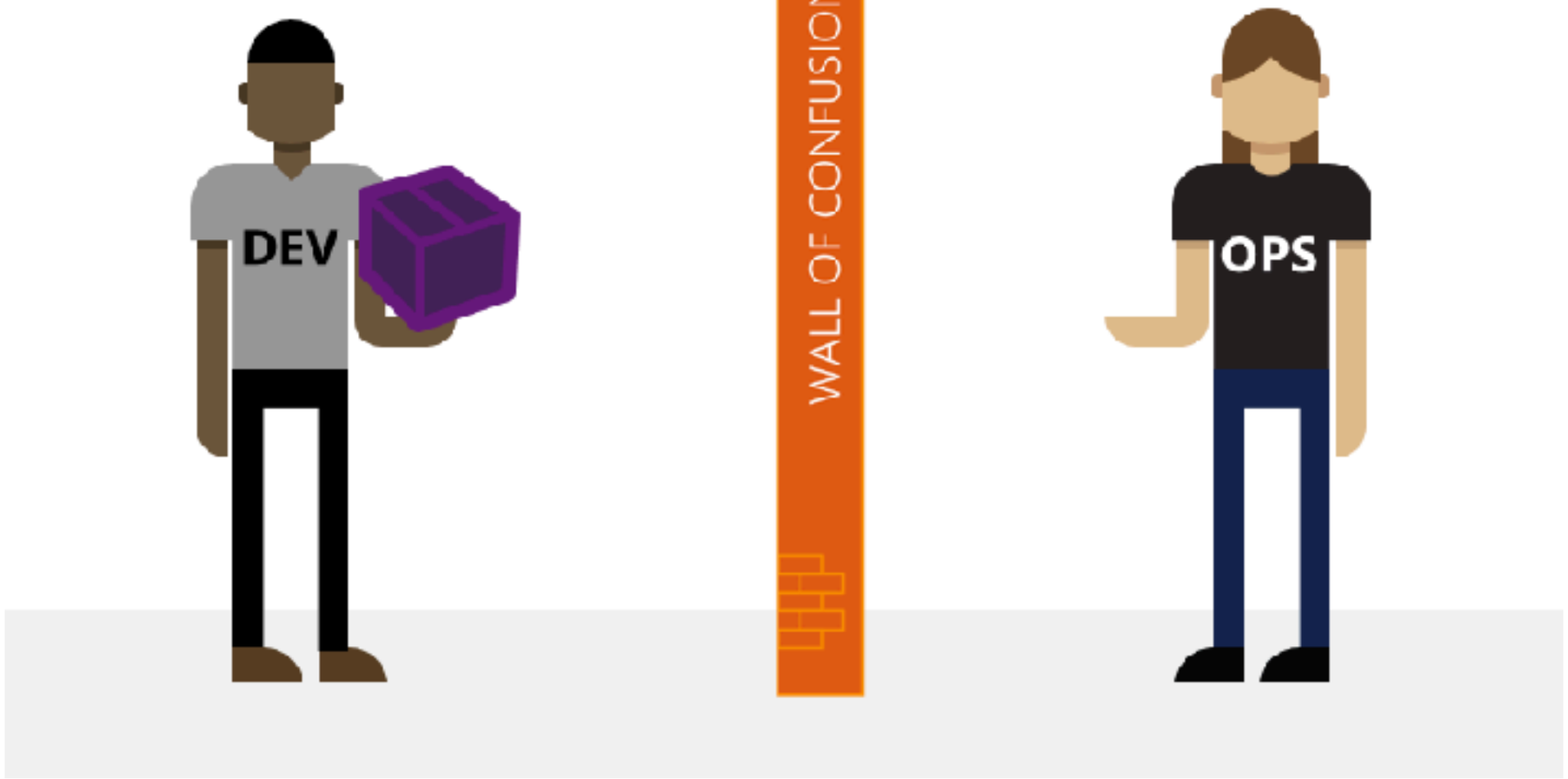


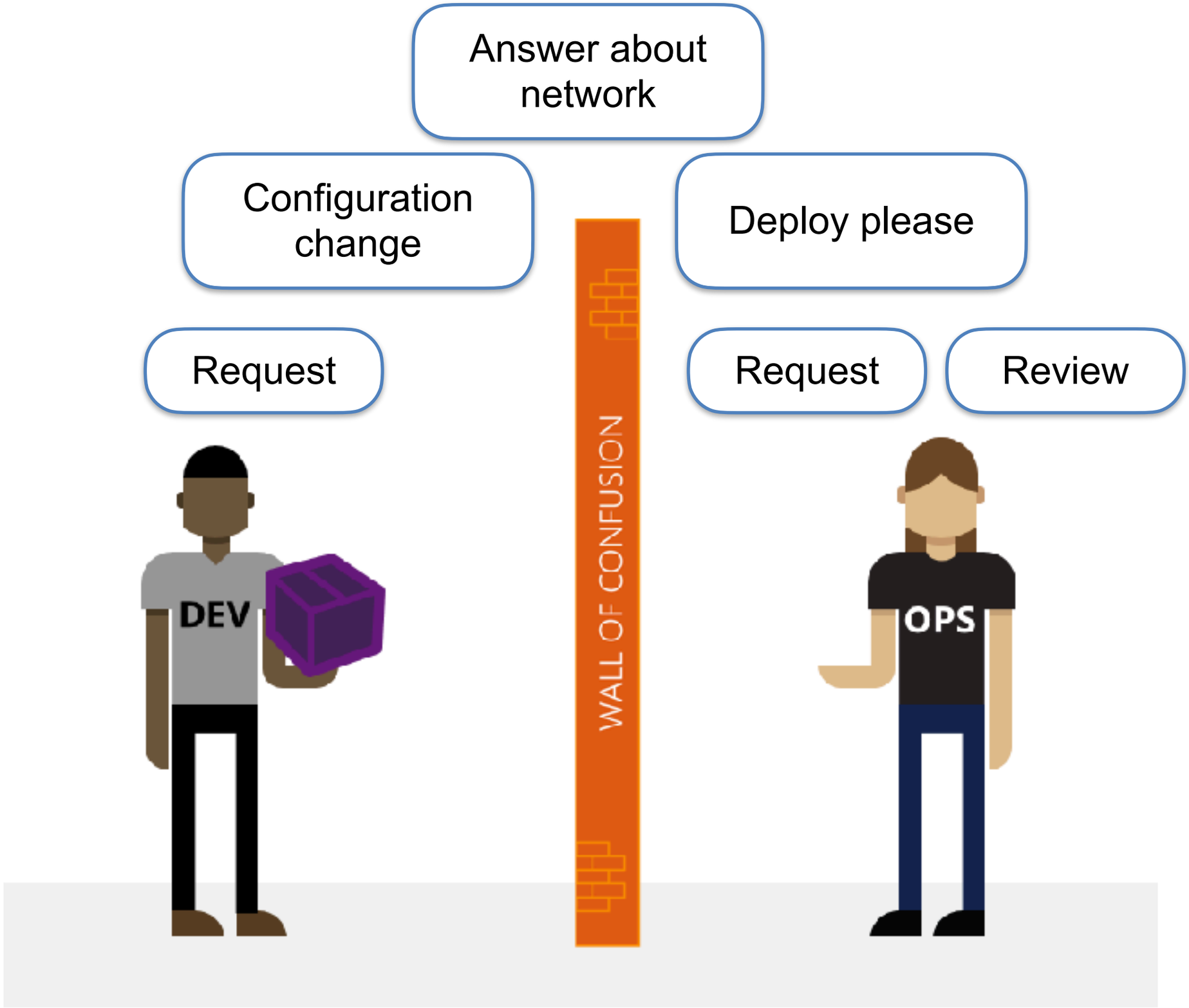
Development vs Operations



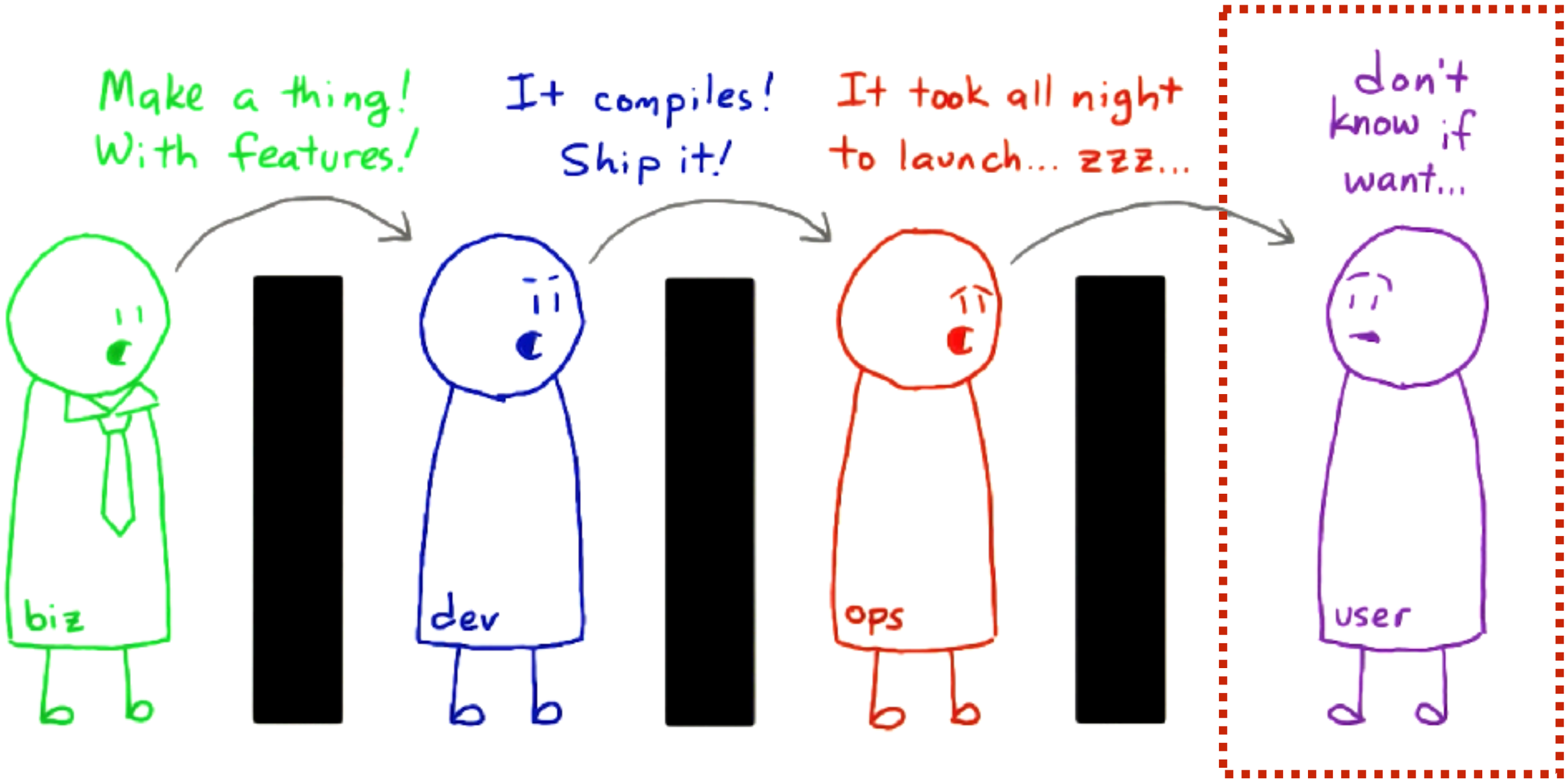
I want to change !

I want to stability !





Result ?



Problems ?

Deadline Driven Development

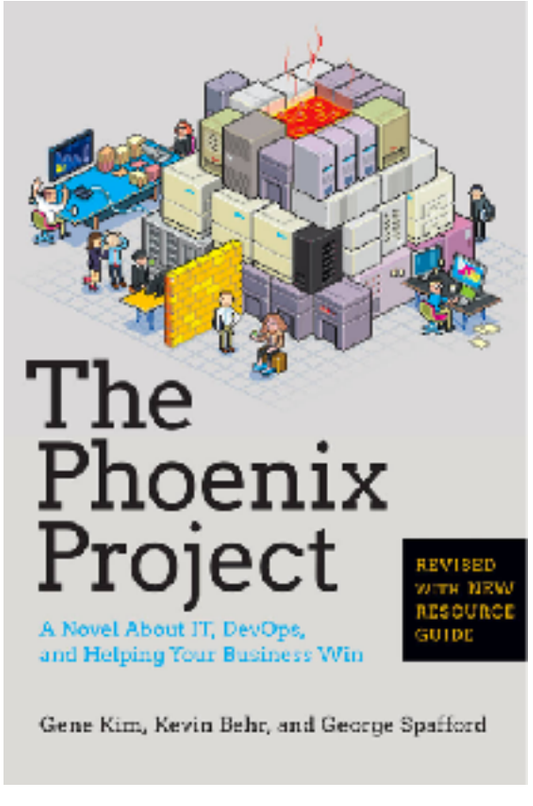
Delayed project/product deliver

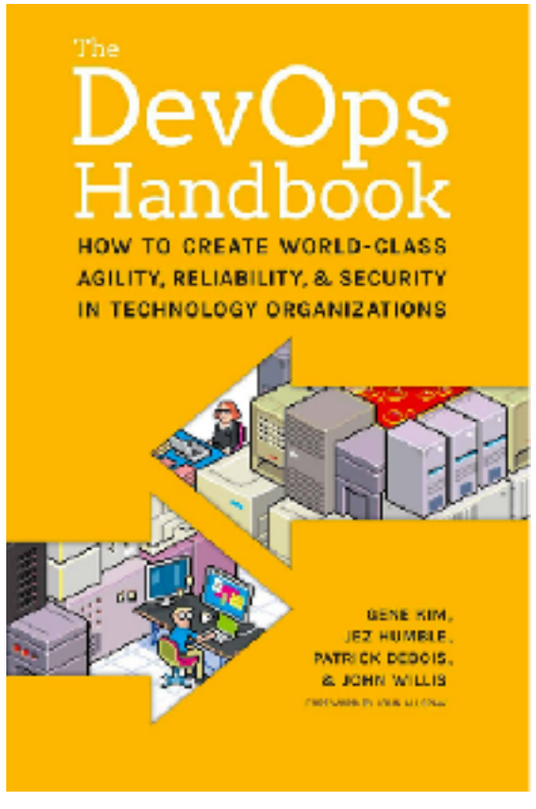
Bad quality

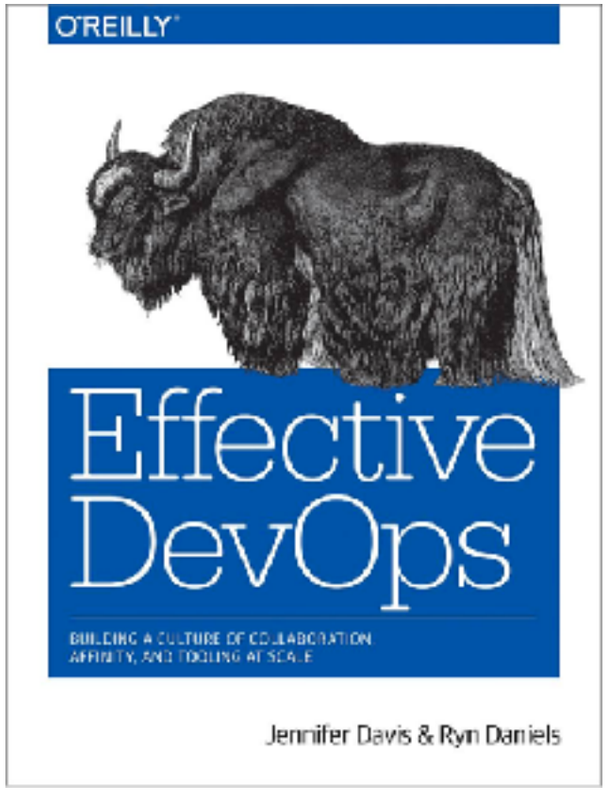
Low customer satisfaction

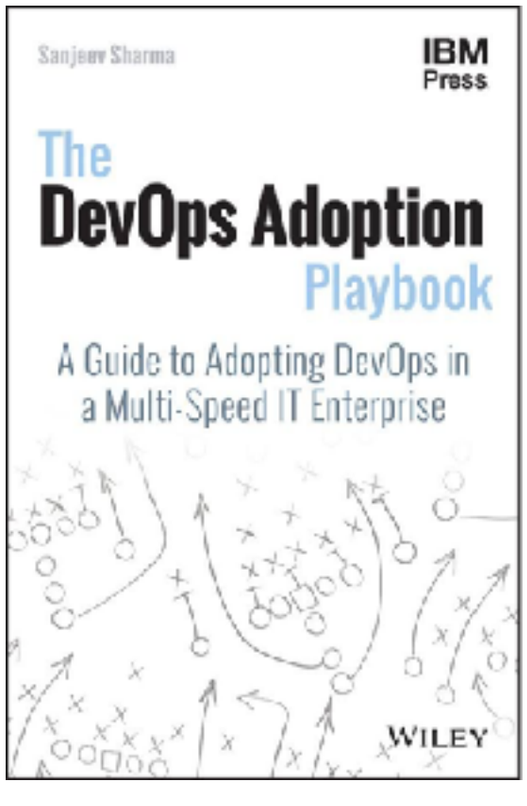
Rise of DevOps

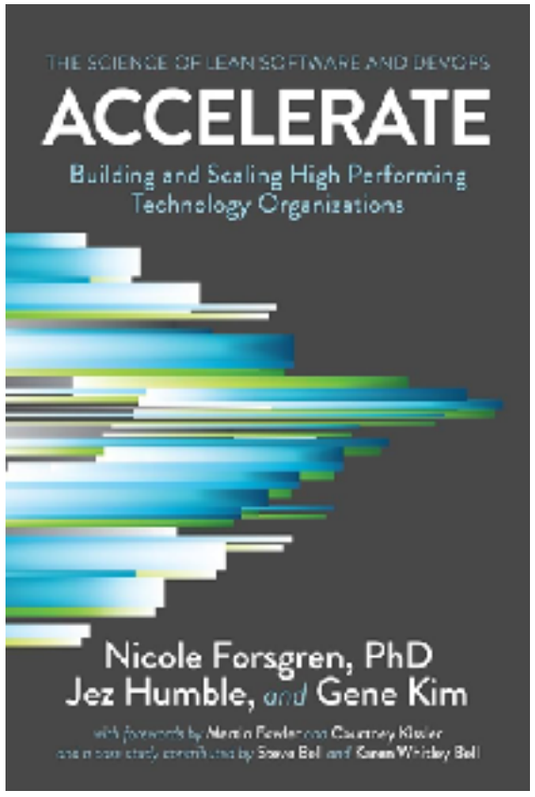
DevOps foundation

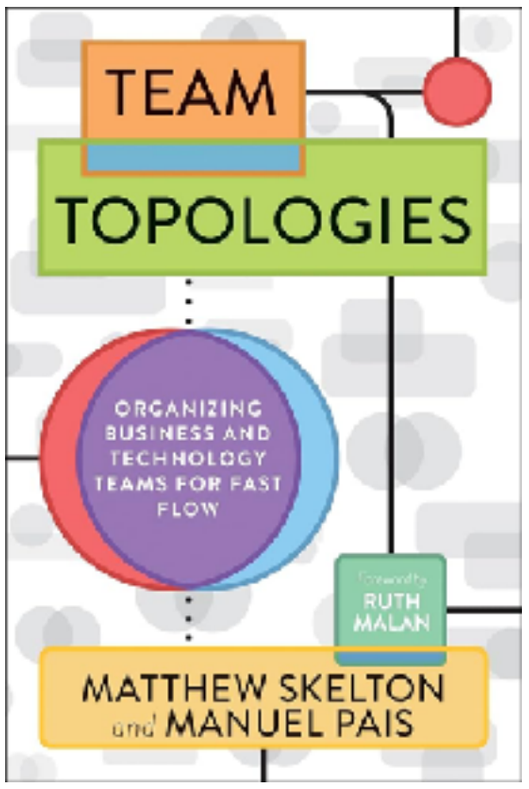


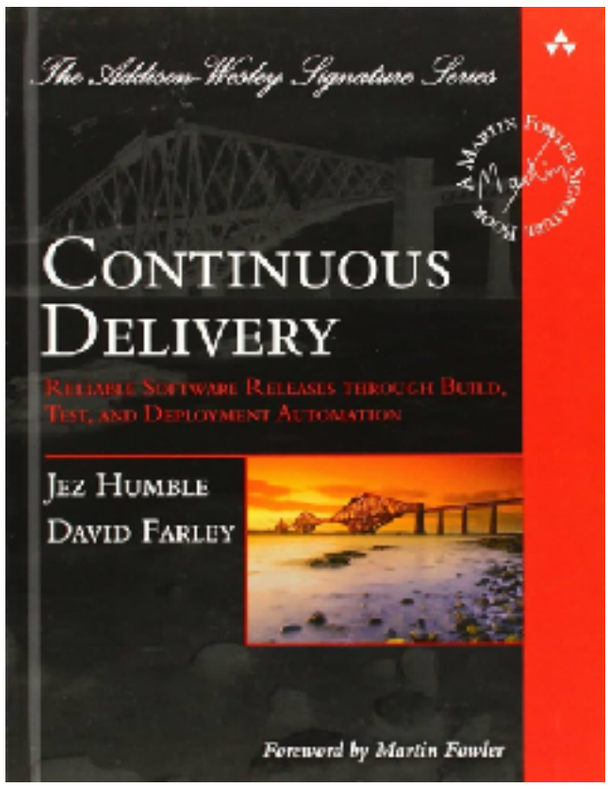


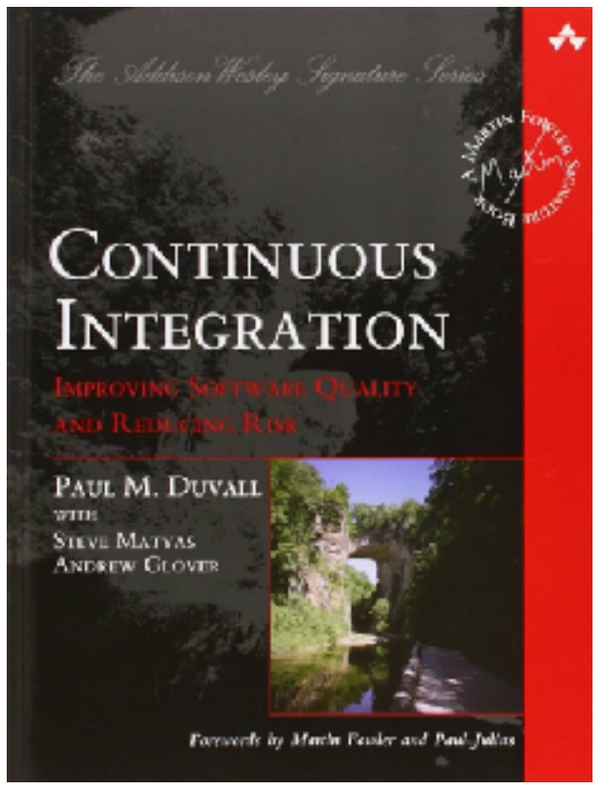




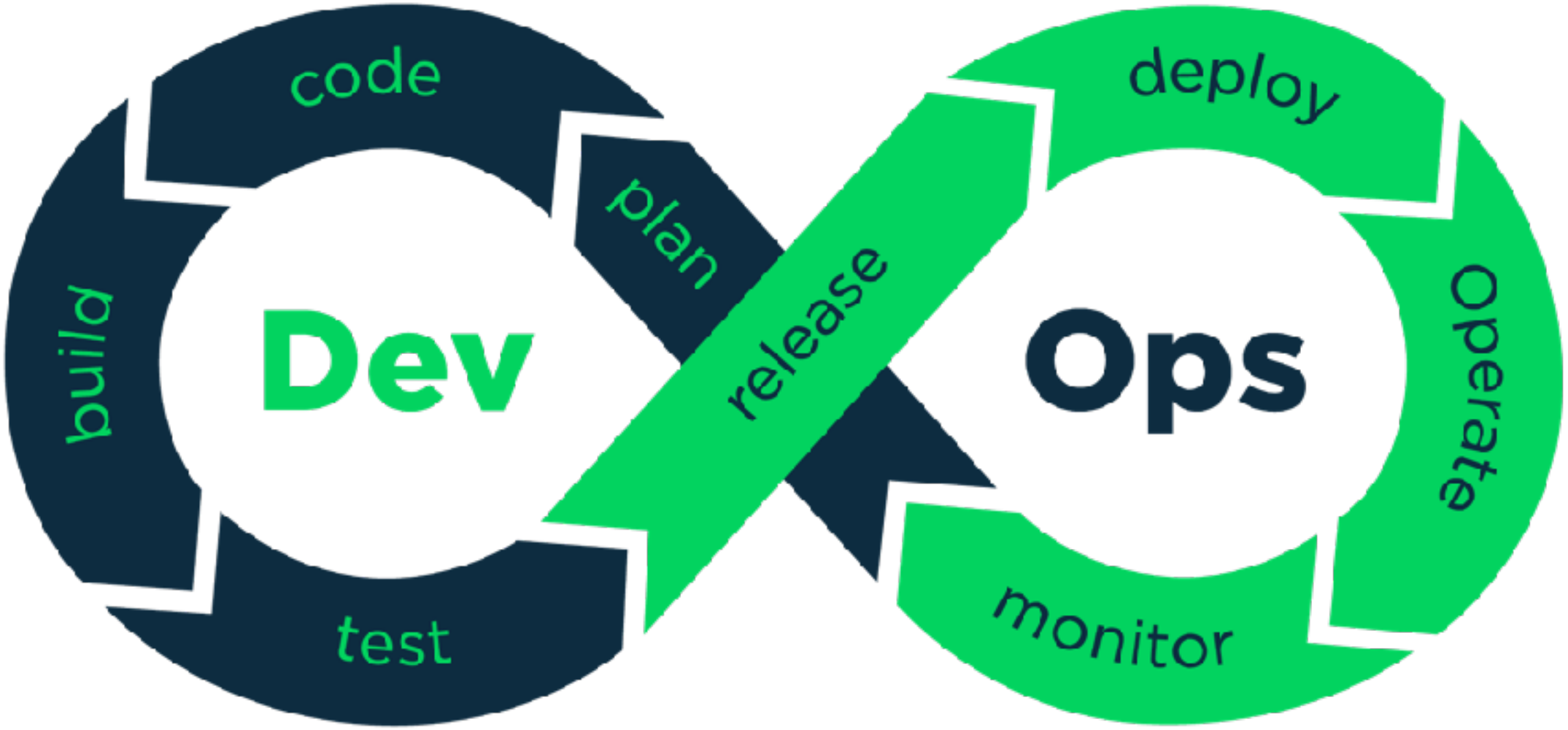




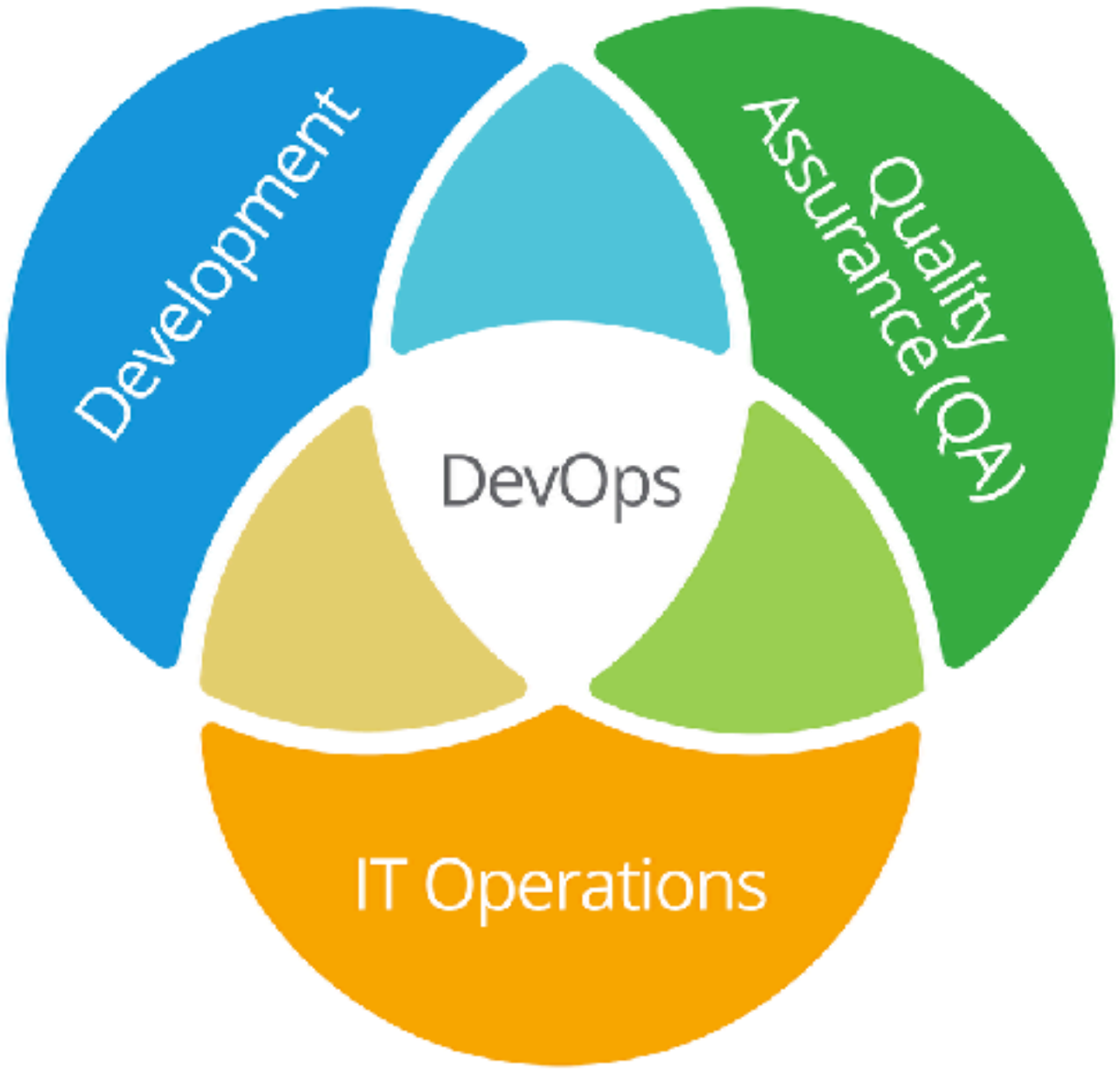




DevOps



DevOps

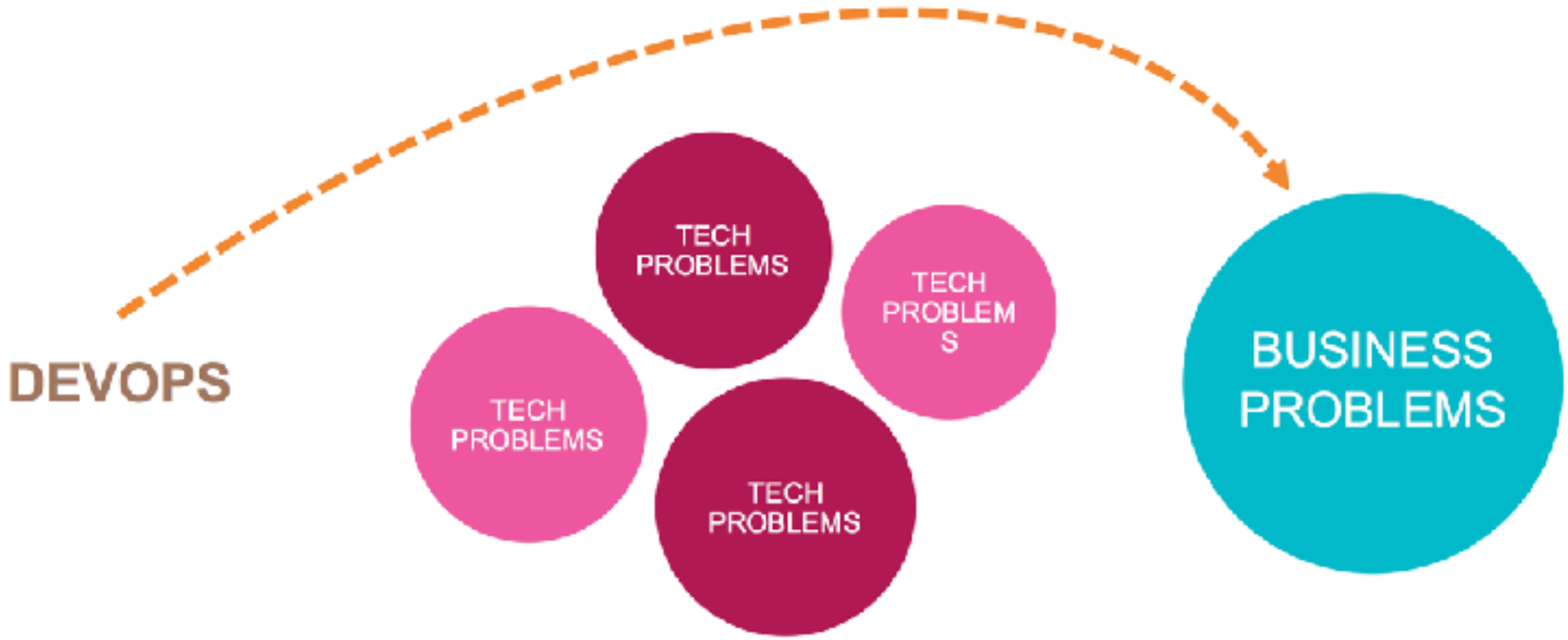


Devops isn't any single person's job.

It's everyone's job.

DevOps

Solve **business** problem first



State of DevOps Report

High-performing teams deploy more frequently and have much faster lead times.

200x more frequent deployments

2,555x shorter lead times

They make changes with fewer failures, and recover faster from failures.

3x lower change failure rate

24x faster recovery from failures

https://puppet.com/resources/whitepaper/state-of-devops-report

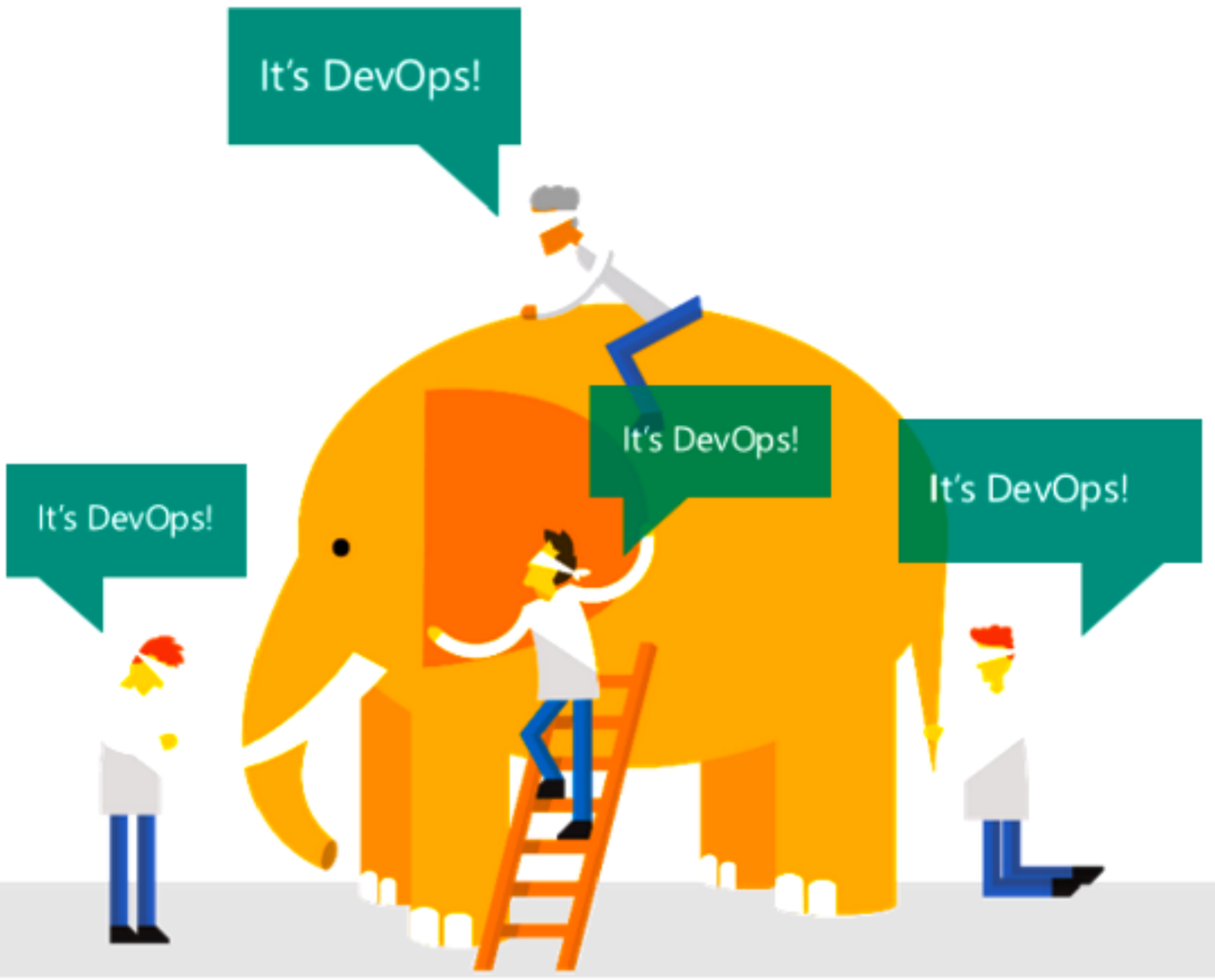
What is DevOps ?

What is DevOps ?

"**DevOps** is development and operations collaboration"

"**DevOps** is using automation"

"**DevOps** is small deployments"



"**DevOps** is treating your infrastructure as code"

"**DevOps** is feature switches"

"Kanban for Ops?"

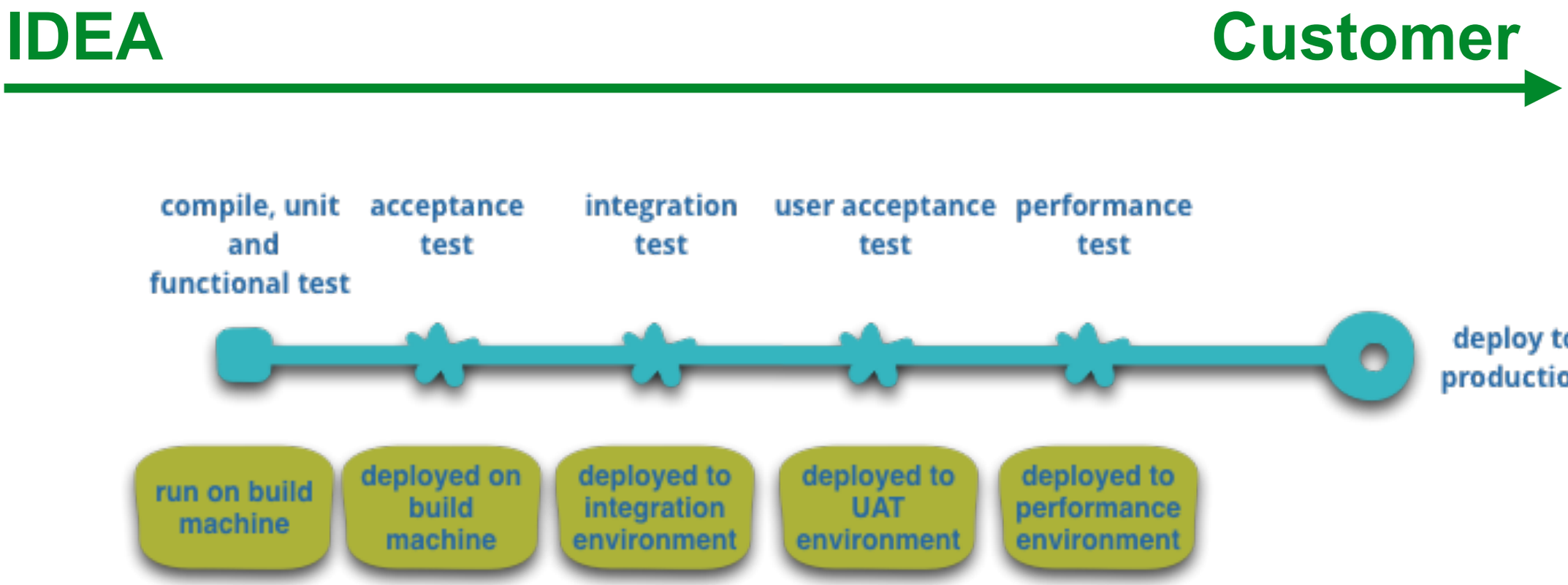
What is DevOps ?

DevOps is a **set of practices** intended to **reduce the time** between committing a change to a system and the change being placed into normal production, while ensuring **high quality**

https://en.wikipedia.org/wiki/DevOps#Definitions_and_history

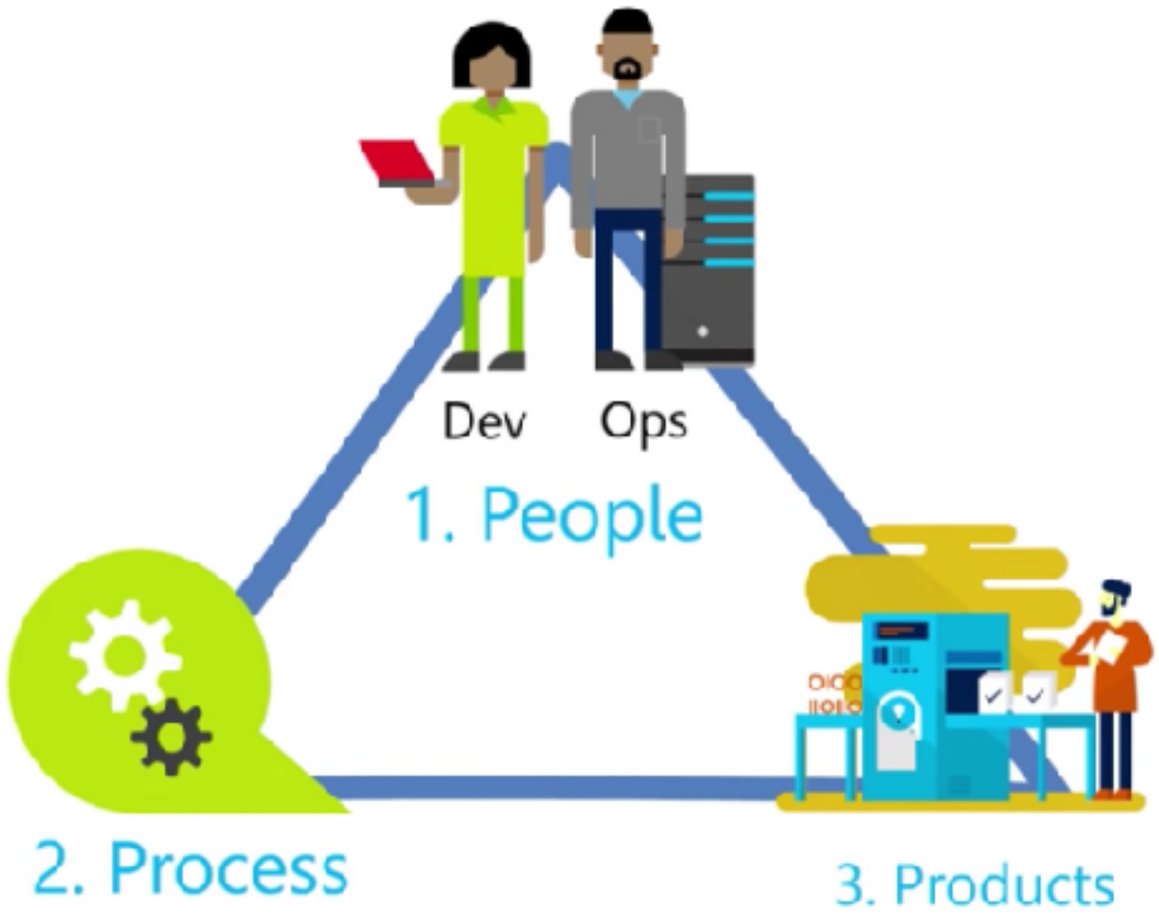
Goals of DevOps

Improve the delivery of value
for Customer and Business



Goals of DevOps

Enable the **continuous delivery** of value to customer and business



Continuous Delivery

The more often to deploy

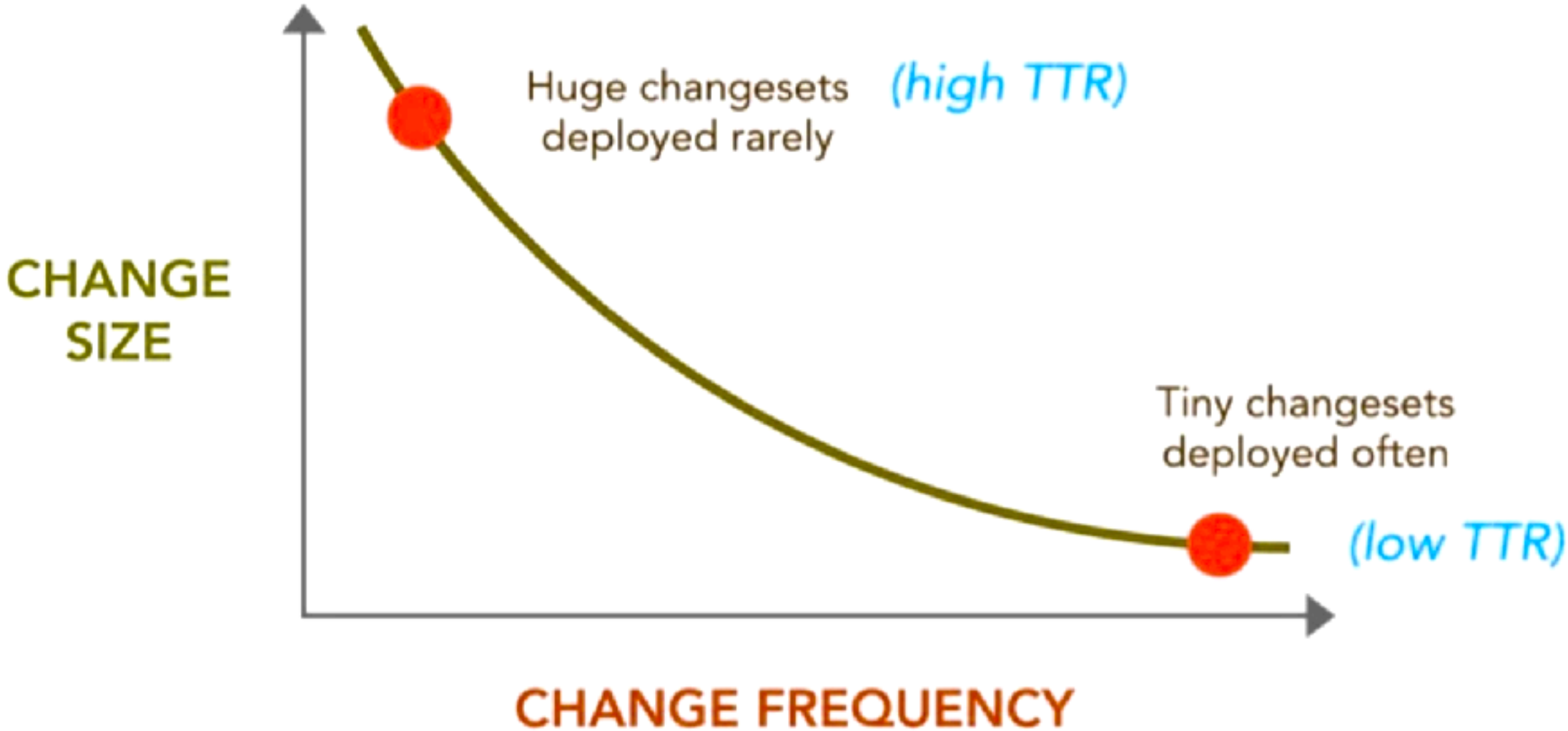
Small change

Automate it

Reduce **TTR** (Time to Repair/Recovery)

Learn and repeat

Continuous Delivery



DevOps is all about Human problems

DevOps Metrics

Lead time for changes

Change failure rate

Deployment frequency

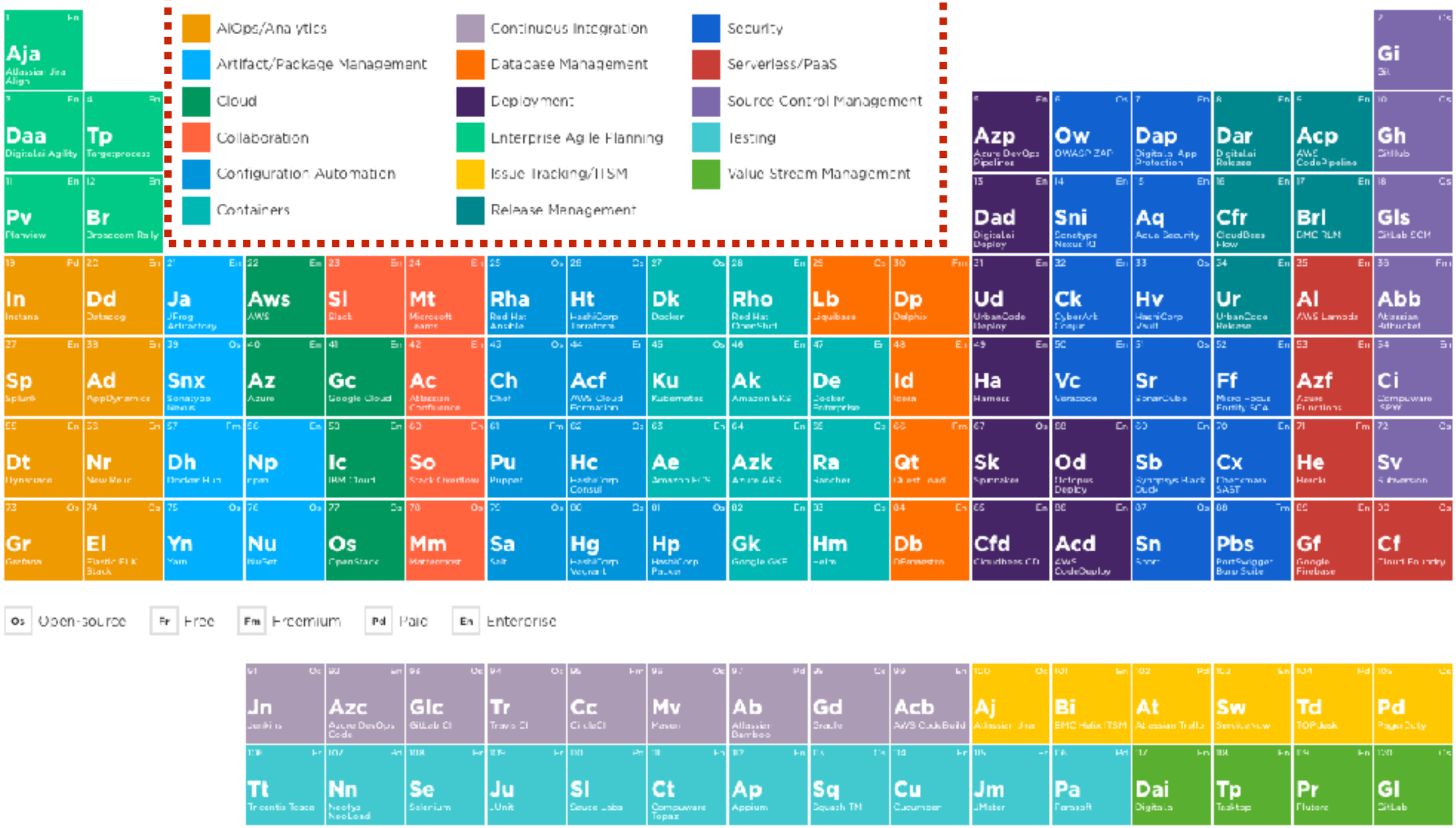
Mean time to recovery (MTTR)

Software Delivery Performance

Aspect of Software delivery performance	Elite	High	Medium	Low
Change lead time For the primary application or service you work on, what is your lead time for changes (i.e., how long does it take to go from code committed to code successfully running in production)?	Less than one day	Between one day and one week	Between one week and one month	Between one week and one month
Deployment frequency For the primary application or service you work on, how often does your organization deploy code to production or release it to end users?	On-demand (multiple deploys per day)	Between once per day and once per week	Between once per week and once per month	Between once per week and once per month
Change failure rate For the primary application or service you work on, what percentage of changes to production or released to users result in degraded service (e.g., lead to service impairment or service outage) and subsequently require remediation (e.g., require a hotfix, rollback, fix forward, patch)?	5%	10%	15%	64%
Failed deployment recovery time For the primary application or service you work on, how long does it generally take to restore service after a change to production or release to users results in degraded service (for example, lead to service impairment or service outage) and subsequently require remediation (for example, require a hotfix, rollback, fix forward, or patch)?	Less than one hour	Less than one day	Between one day and one week	Between one month and six months
Percentage of respondents	18%	31%	33%	17%

https://cloud.google.com/blog/products/devops-sre/announcing-the-2023-state-of-devops-report

DevOps Tools



https://digital.ai/periodic-table-of-devops-tools

- AiOps/Analytics
- Artifact/Package Management
- Cloud
- Collaboration
- Configuration Automation
- Containers
- Continuous Integration
- Database Management
- Deployment
- Enterprise Agile Planning
- Issue Tracking/ITSM
- Release Management
- Security
- Serverless/PaaS
- Source Control Management
- Testing
- Value Stream Management

https://digital.ai/periodic-table-of-devops-tools

Workshop

Design your delivery process

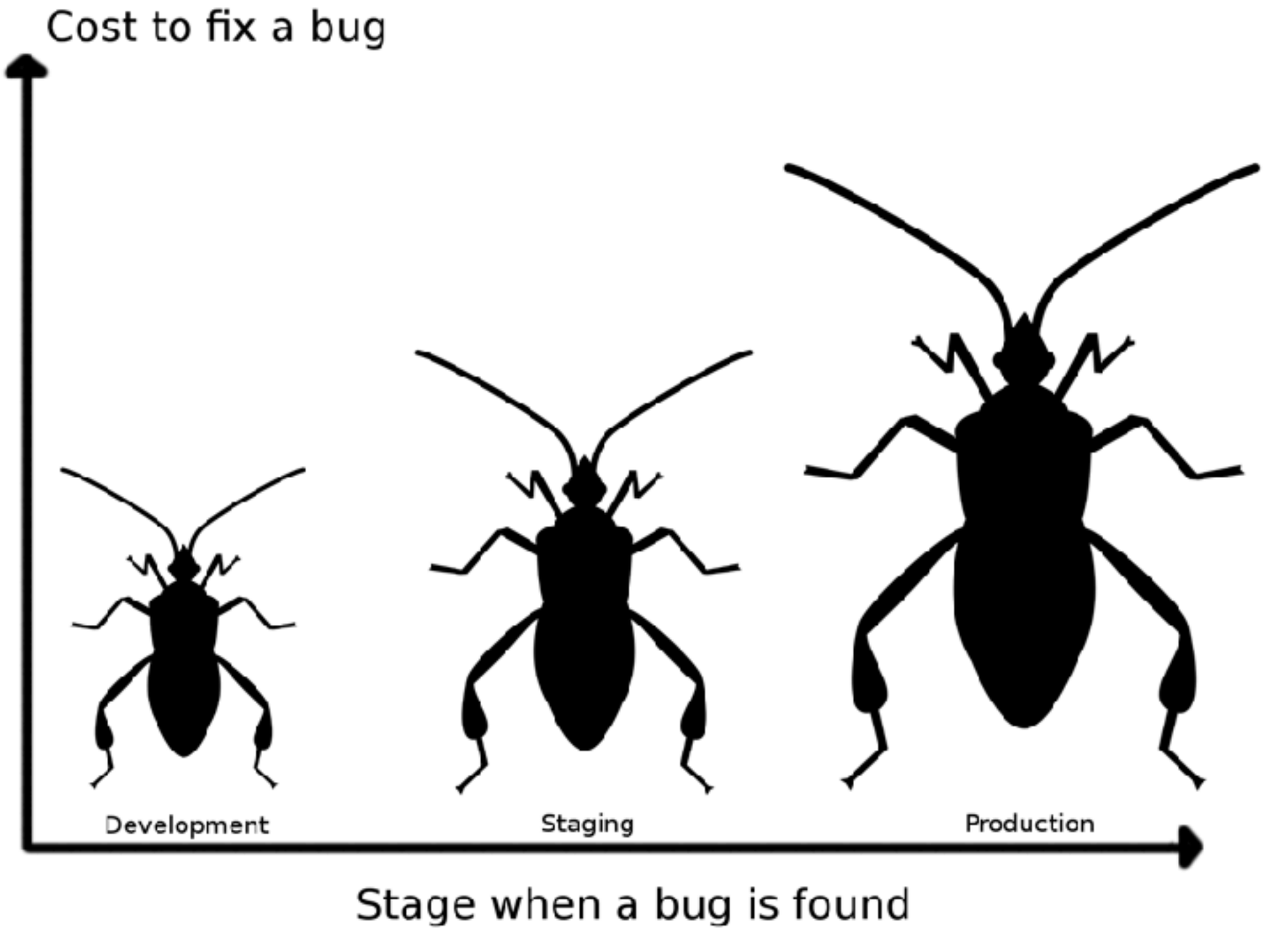
Continuous Integration
Continuous Delivery

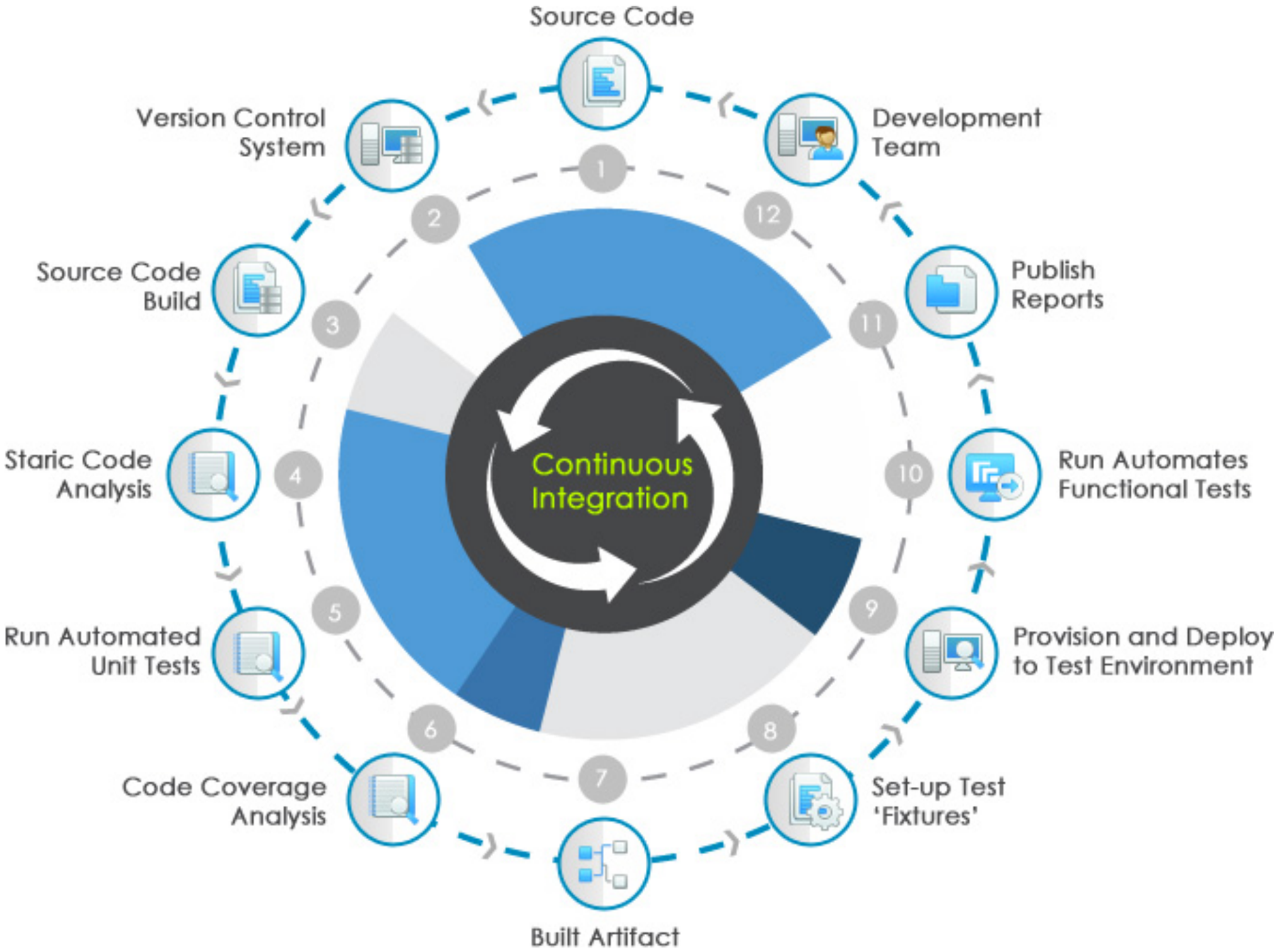
Why CI/CD ?

The cost of integration

1. Merging the code
2. Duplicate changes
3. Test again again !!
4. Fixing bugs
5. Impact on stability

The cost of integration





Jenkins

Bamboo

TeamCity

go

Visual Studio
Team Foundation Server

Hudson

Travis

wercker

circleci

CI is about what people do not about what tools they use

Continuous Integration

Discipline to **integrate frequently**

Continuous Integration

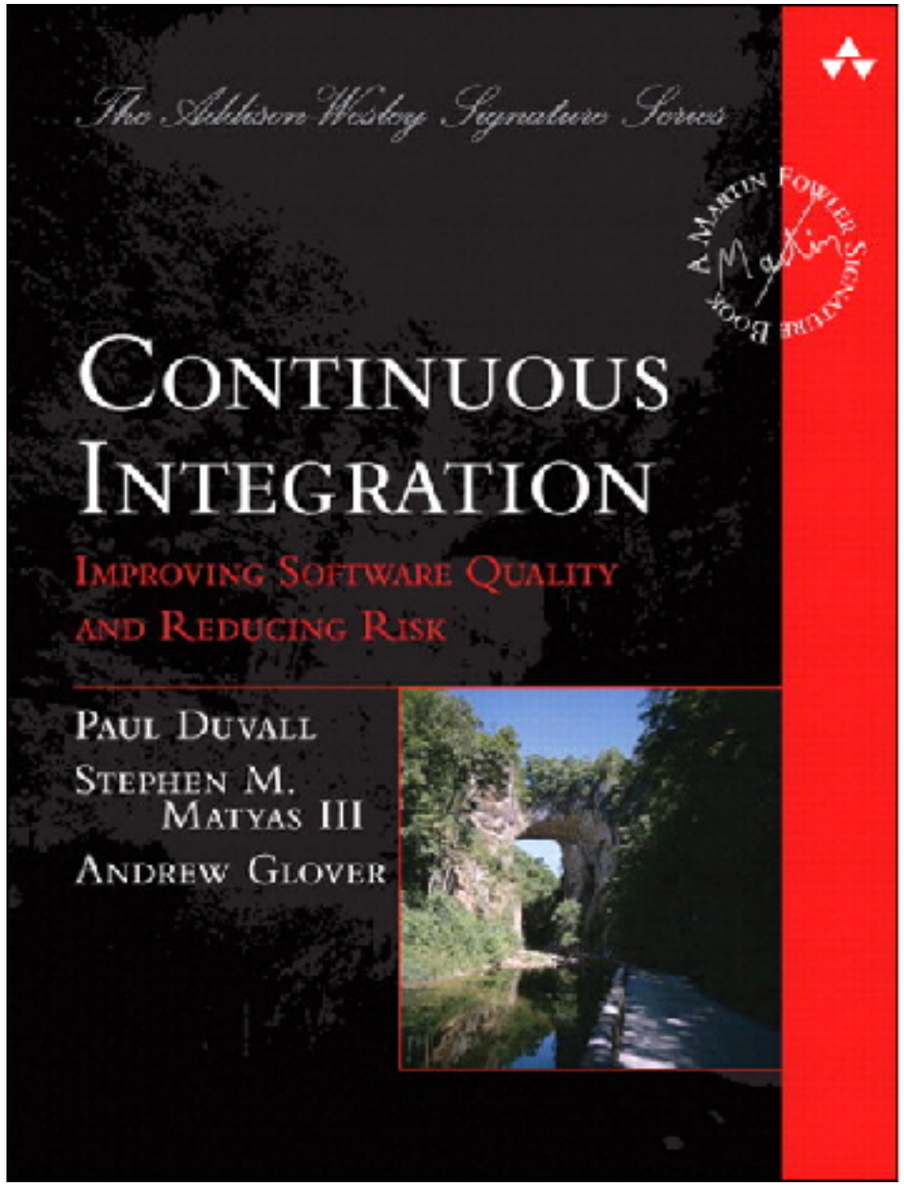
Strive to make **small change**

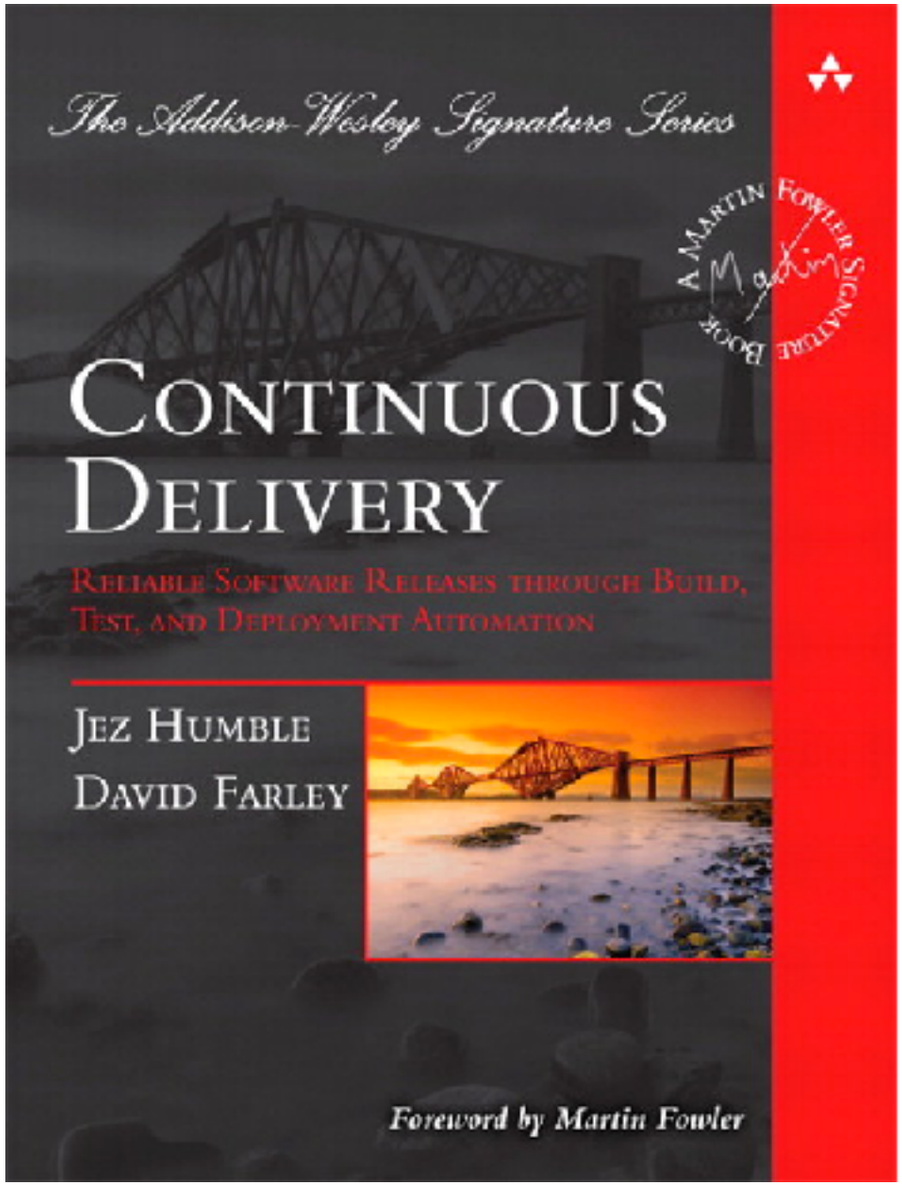
Continuous Integration

Strive for **fast feedback**

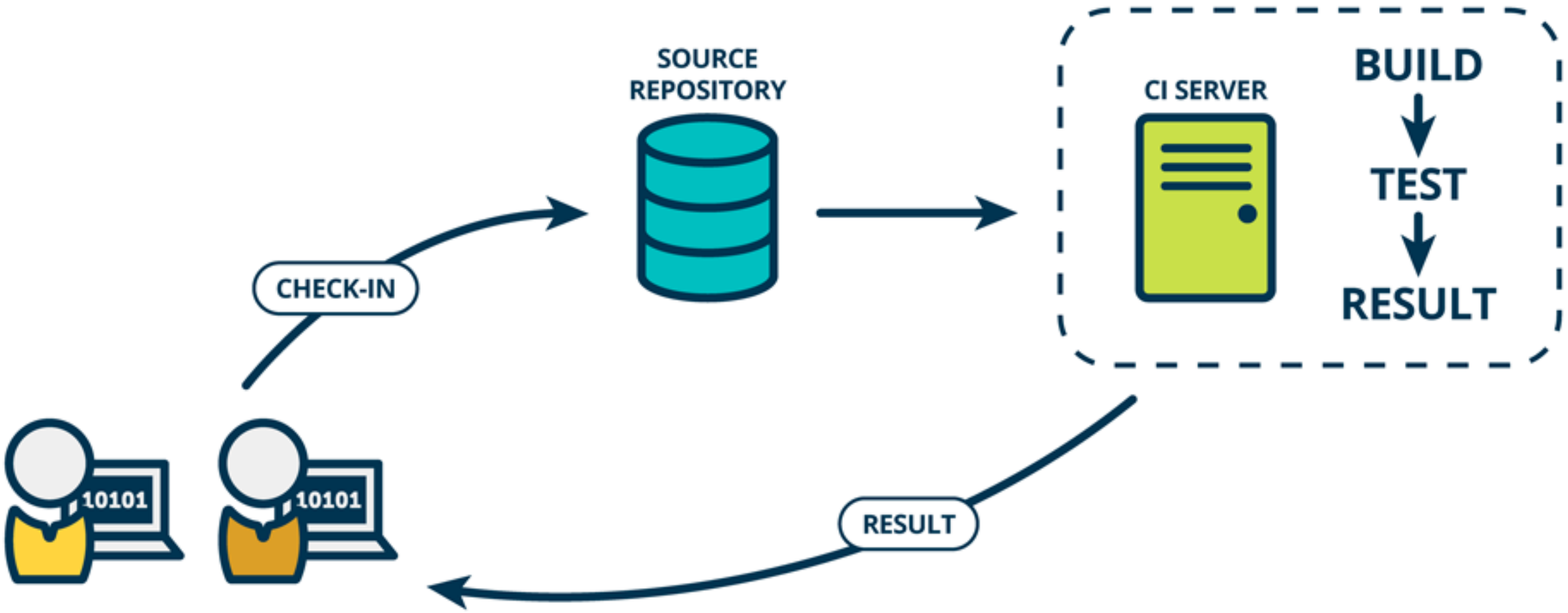
Practices of Continuous Integration

Improve quality and reduce risk

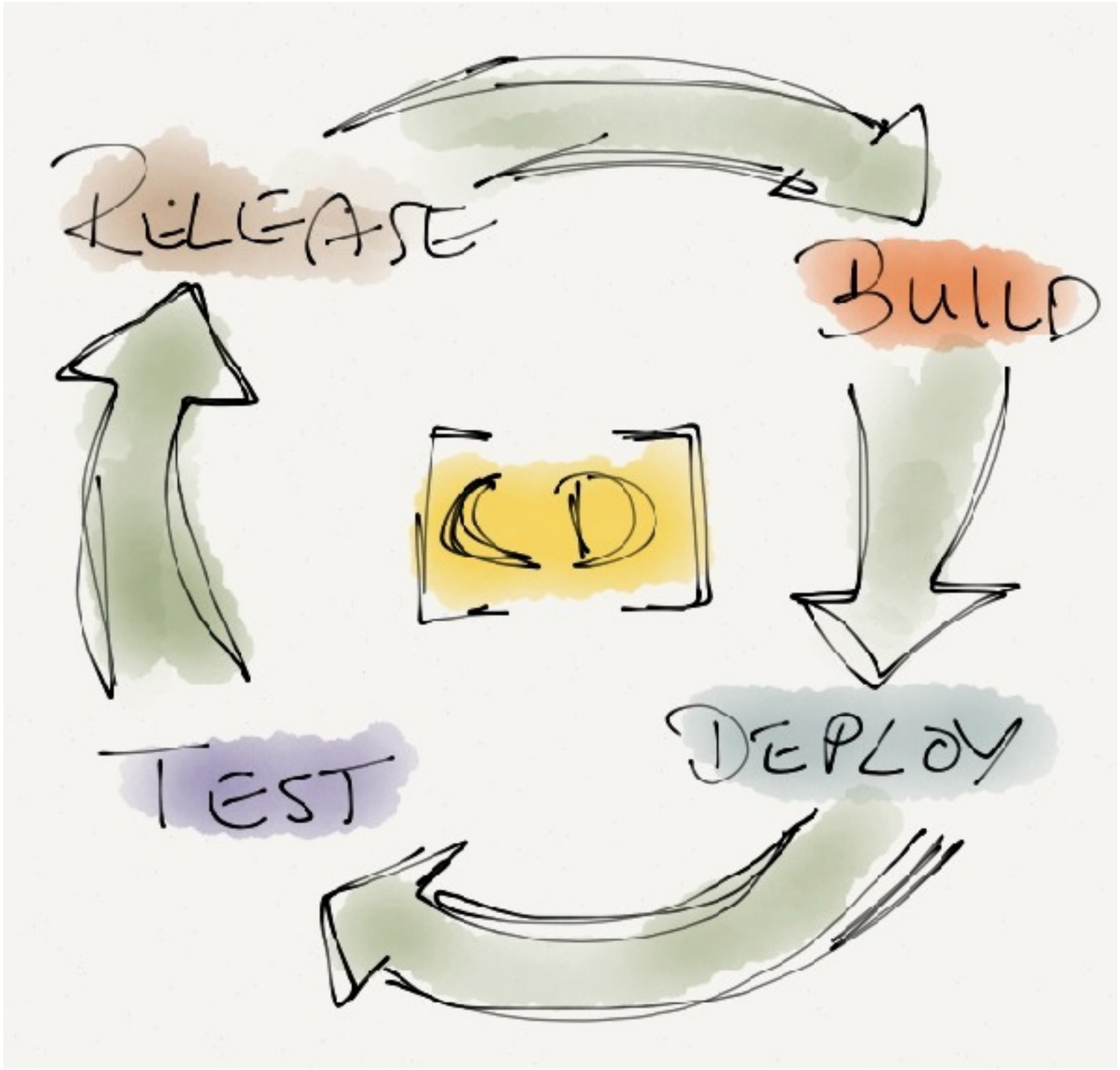




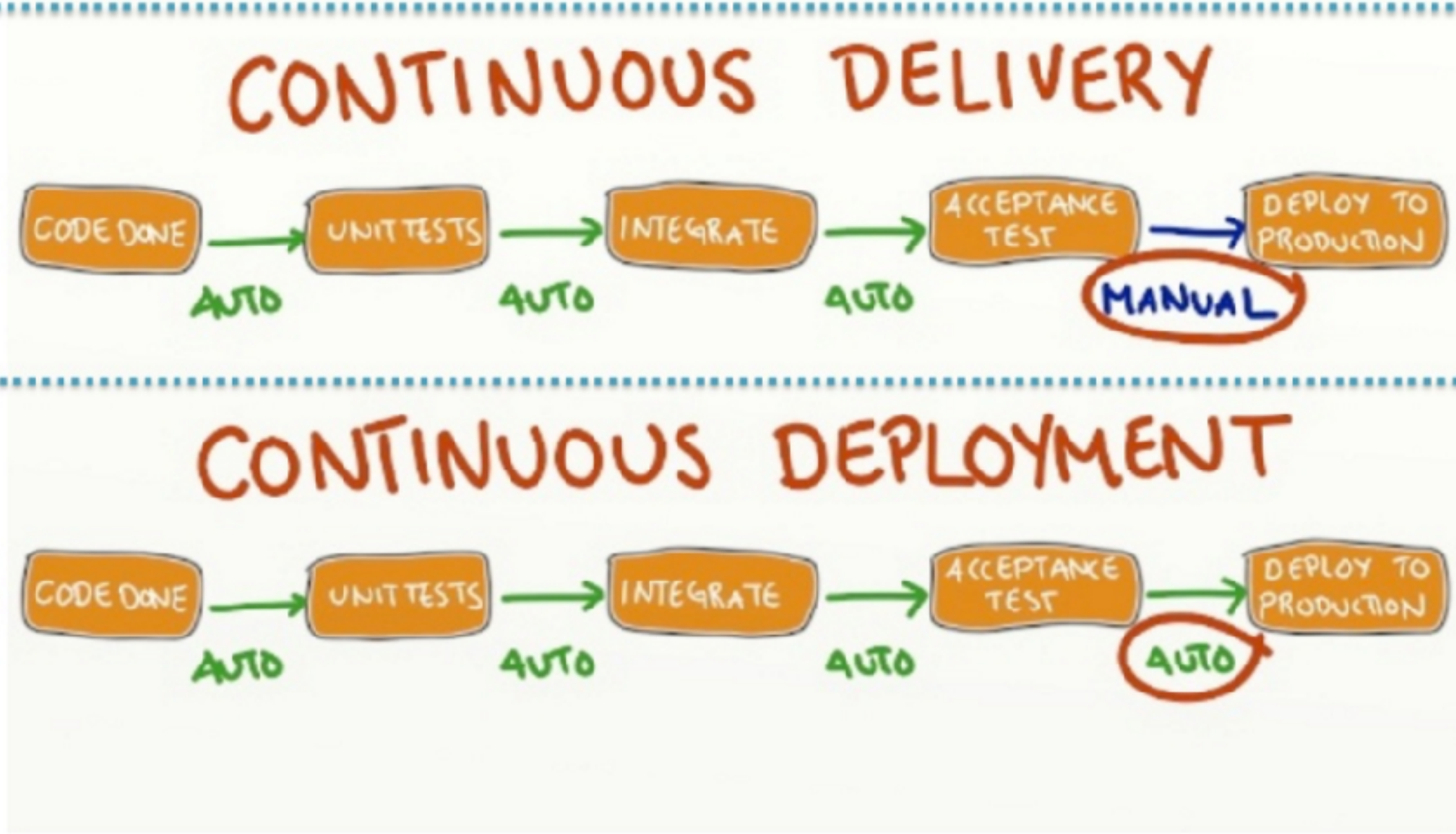
Continuous Integration



CD ?

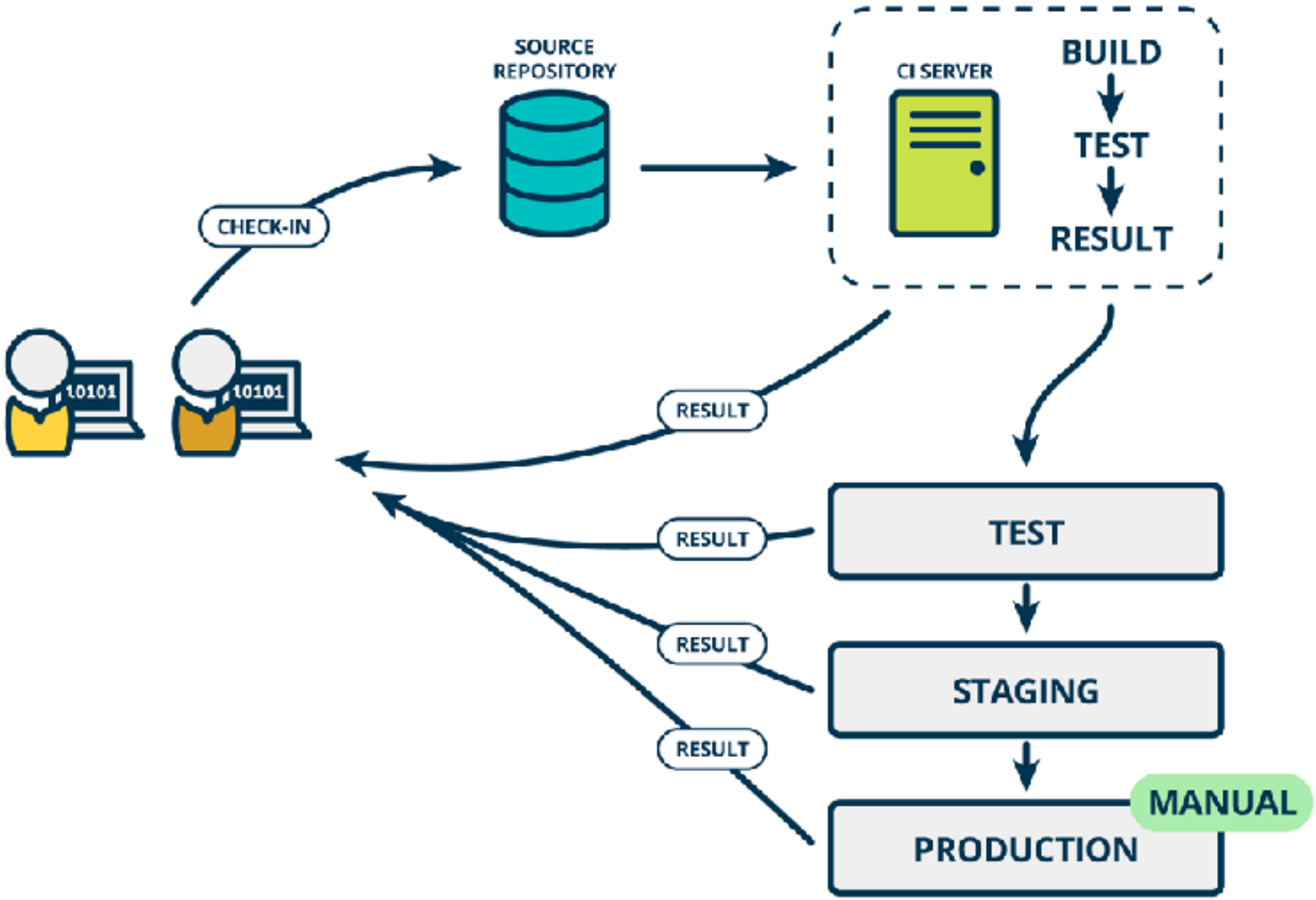


CD ?

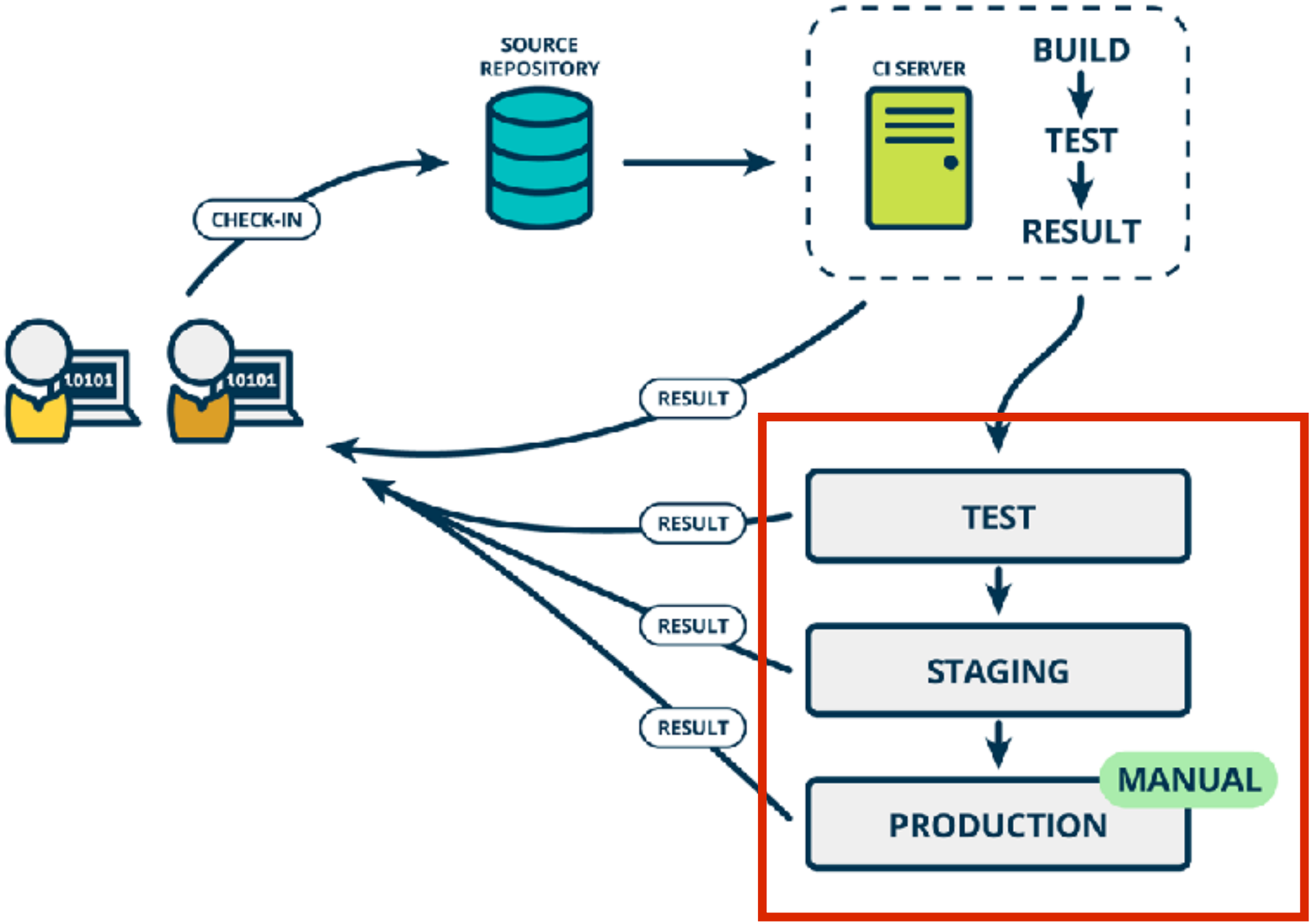


http://blog.crisp.se/2013/02/05/yassalsundman/continuous-delivery-vs-continuous-deployment

Continuous Delivery



Rise of DevOps



Continuous Integration

is a Software development **practices**

Practice 1

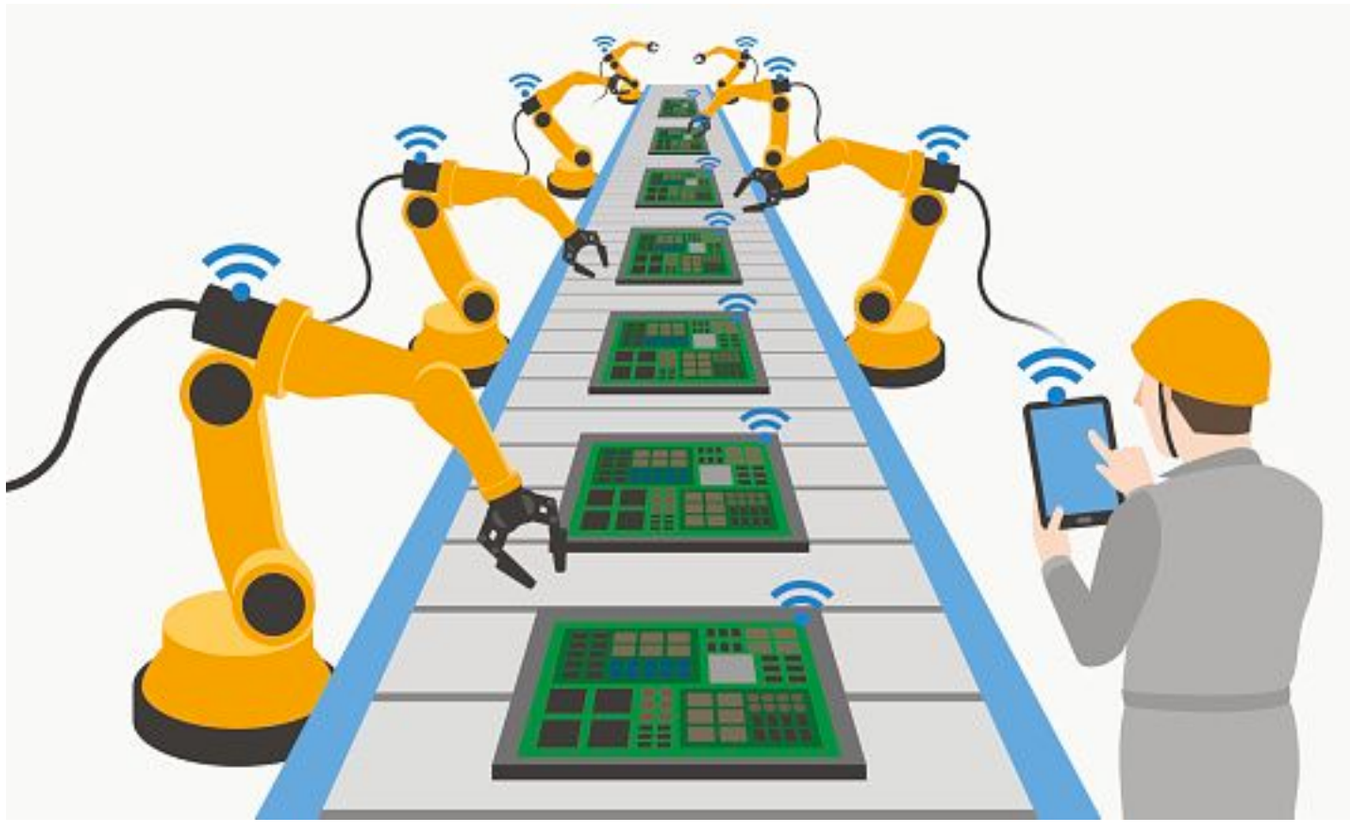
Maintain a single source repository

In general, you should store in source control everything you need to build anything

Practice 2

Automated the build

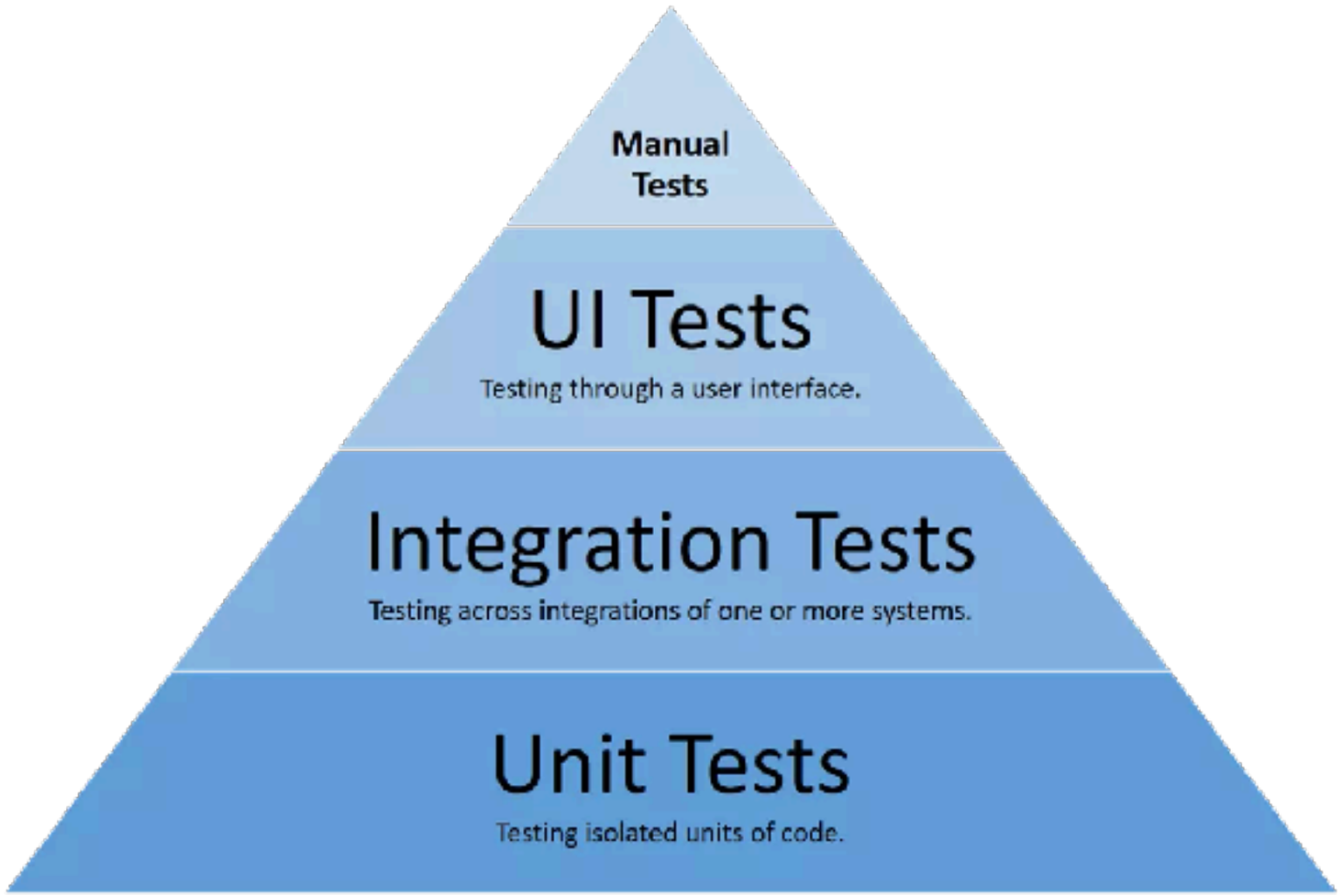
Automated environment for builds



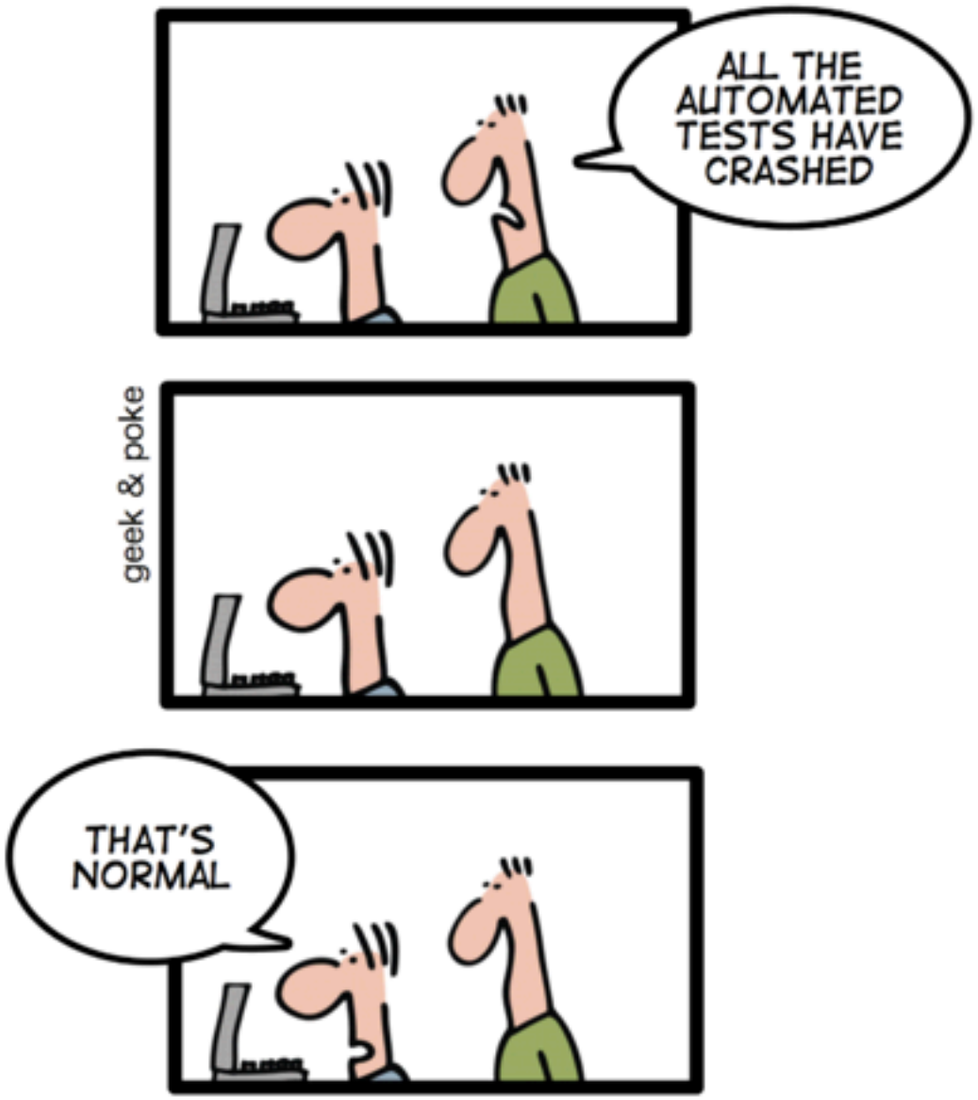
Practice 3

Make your build **self-testing**

Build process => compile, linking and **testing**



TODAY: CONTINUOUS INTEGRATION GIVES YOU THE COMFORTING FEELING TO KNOW THAT EVERYTHING IS NORMAL



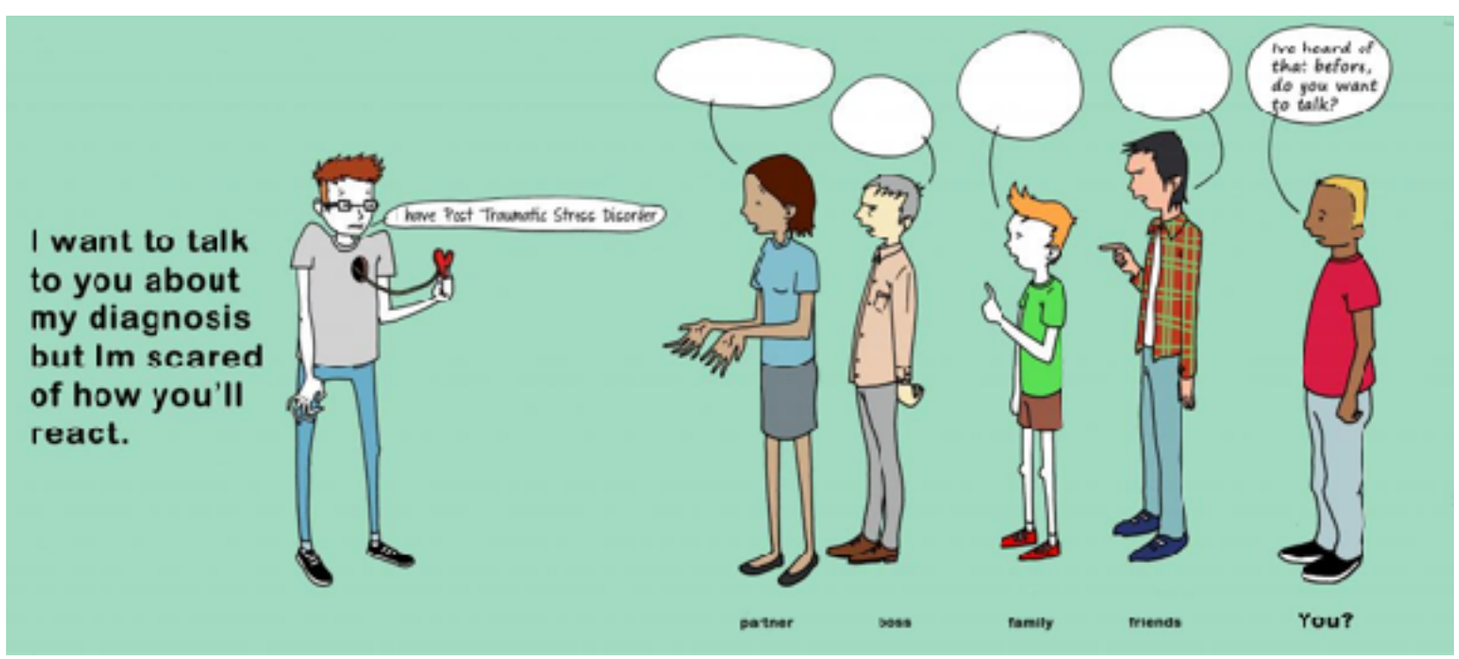
http://geekandpoke.typepad.com/

Practice 4

Everyone **commits** to the mainline everyday

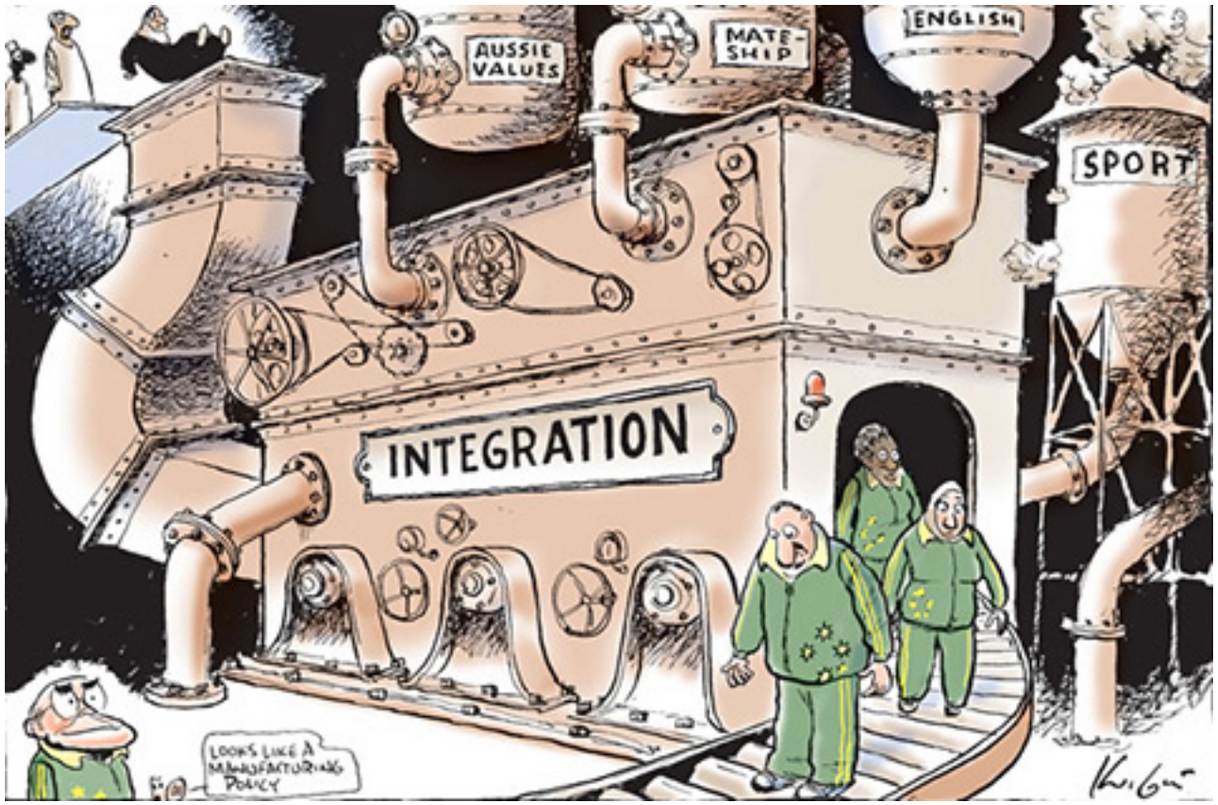
Integration is about **communication**

Integration allows developers to **tell** other developers



Practice 5

Every commits should build the mainline on an **Integration machine**



Nightly build is not enough for Continuous Integration

Practice 6

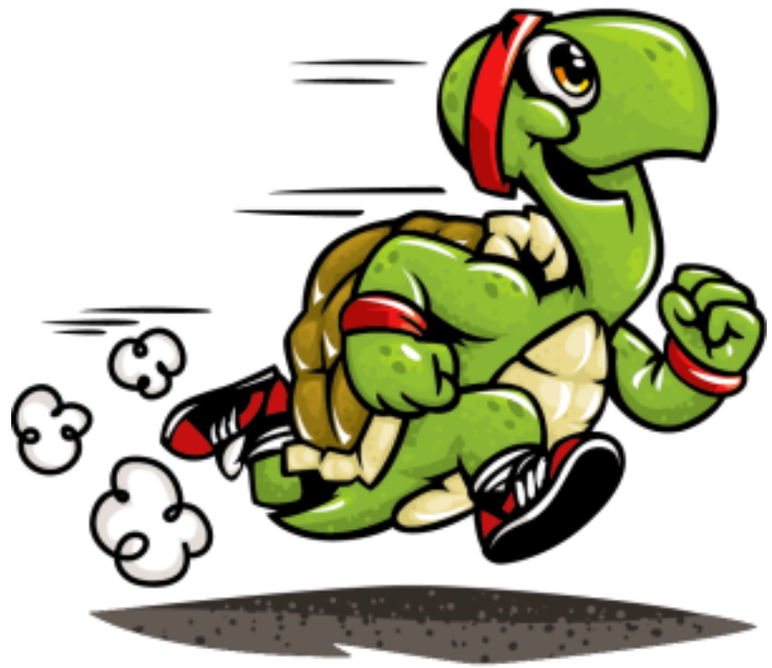
Fix broken builds **immediately**

“Nobody has a higher priority task than fixing the build”

Practice 7

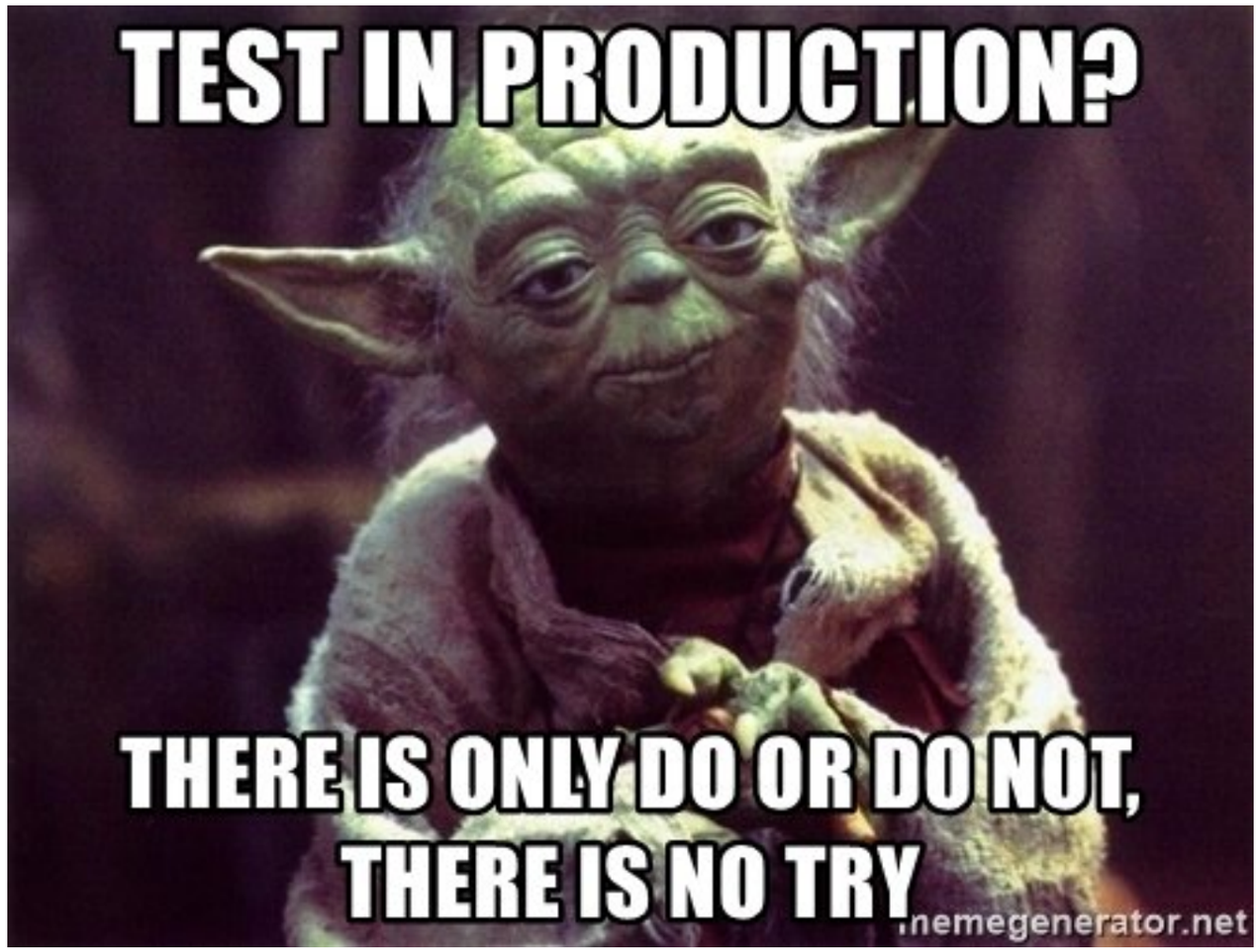
Keep the build **fast**

Continuous Integration is to provide rapid feedback



Practice 8

Test in clone of the **Production** environment



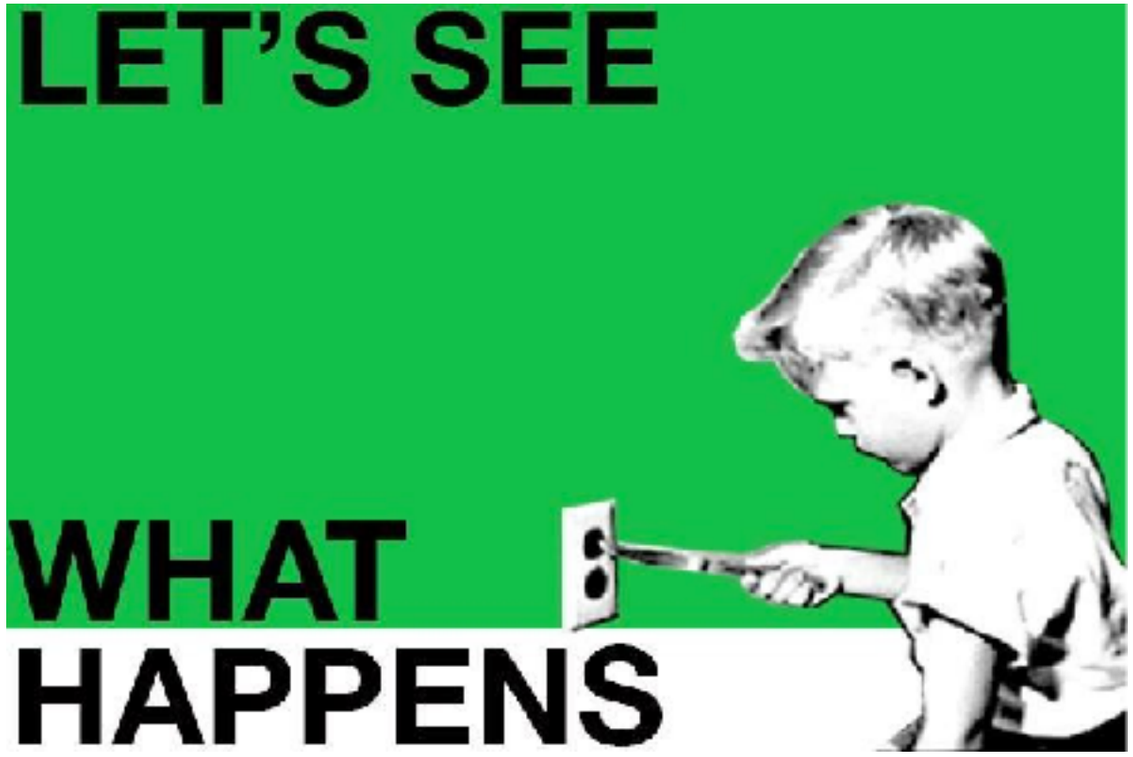
Practice 9

Make it easy for anyone to get
the latest executable

Make sure well known place where people can find

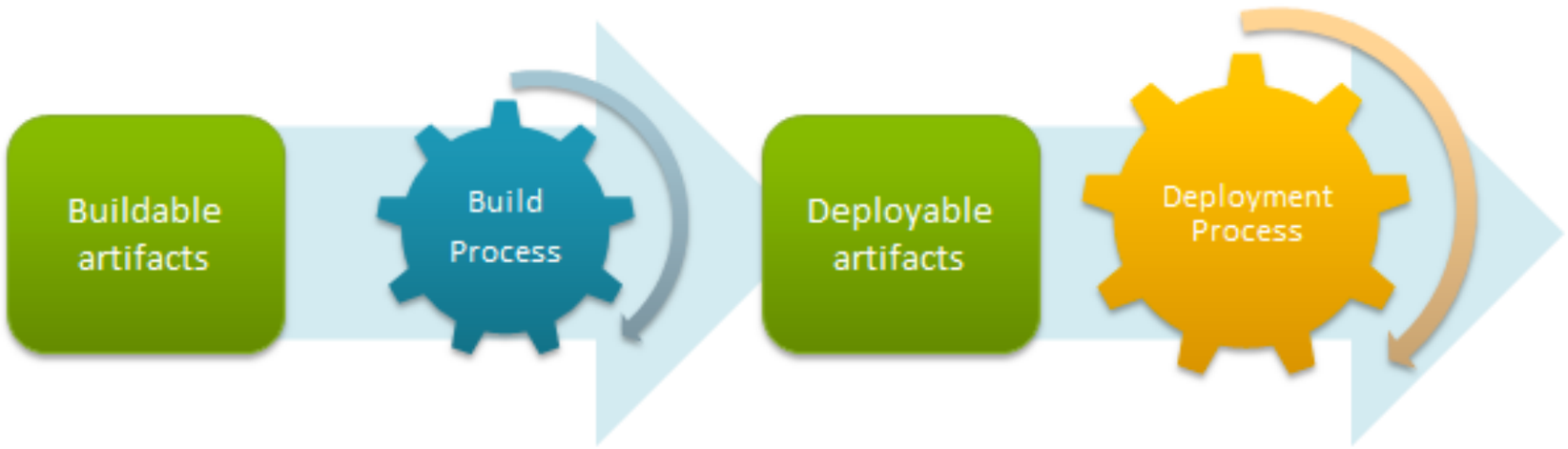
Practice 10

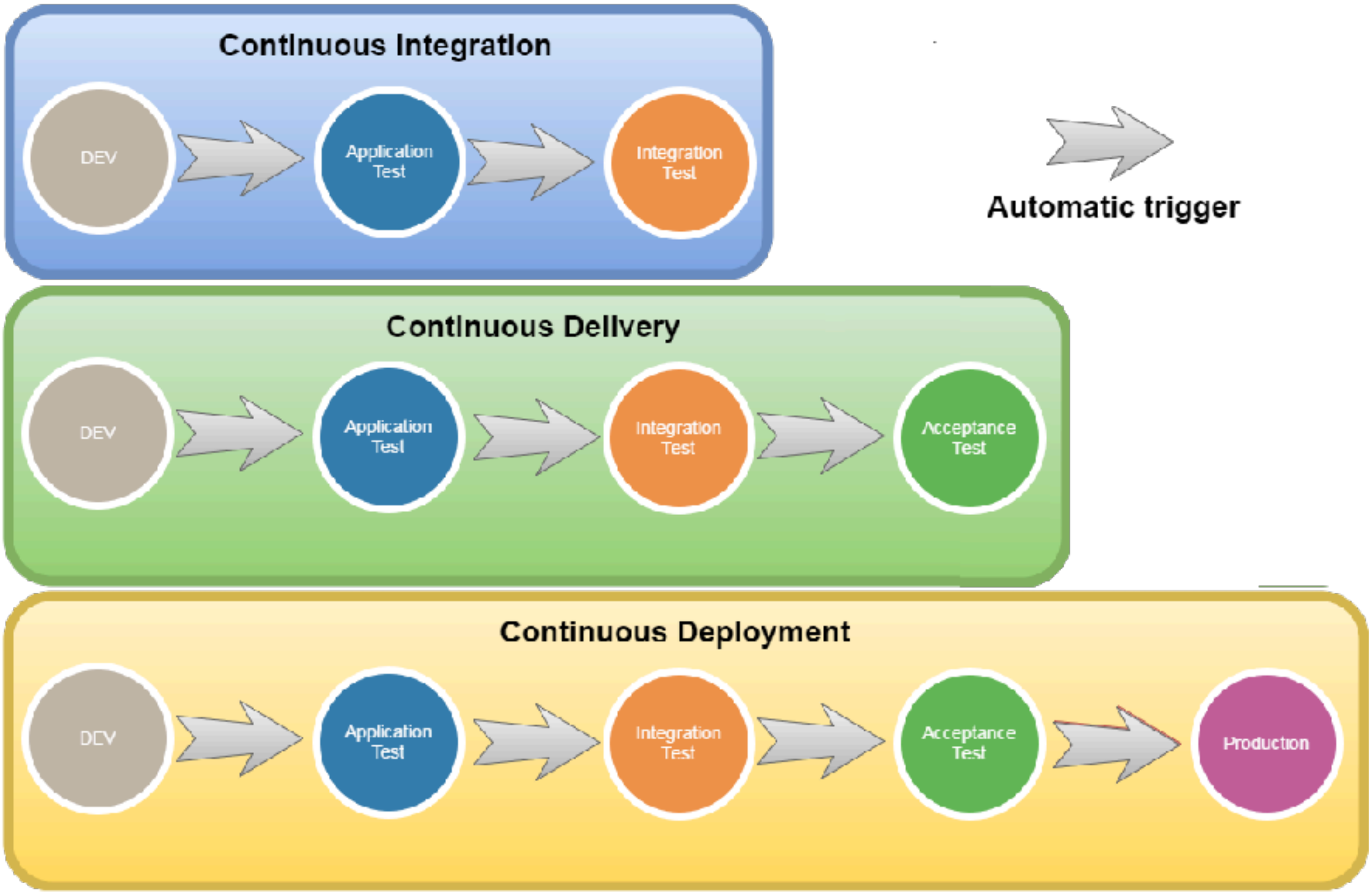
Everyone can see what's happening
Easier to see the state of the system and changes
Show the good information



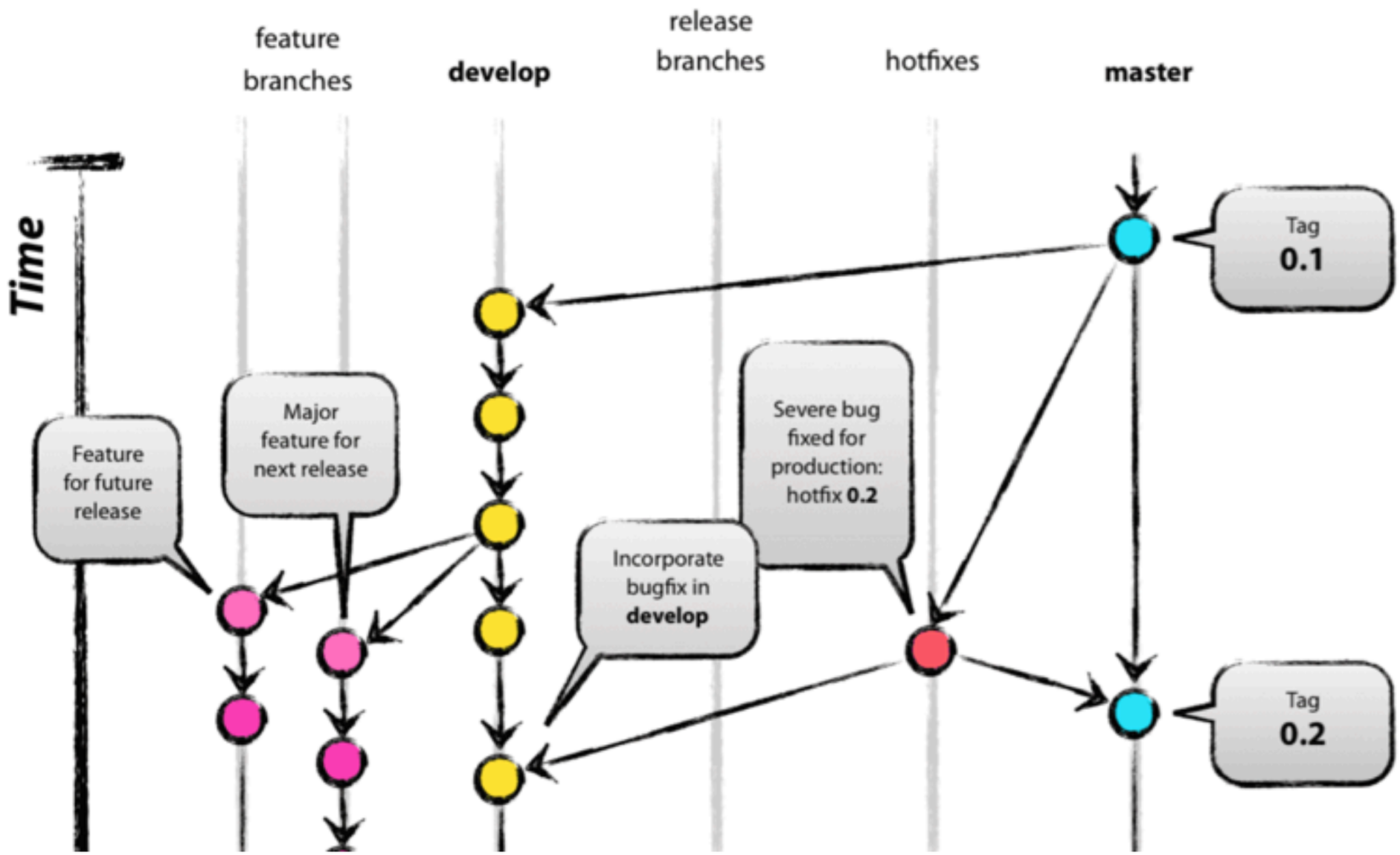
Practice 11

Automated deployment





Git Branch Model



https://nvie.com/posts/a-successful-git-branching-model/

Suggestion

Keeping branches short-lived, merge conflicts are keep to as few as possible

Commit message

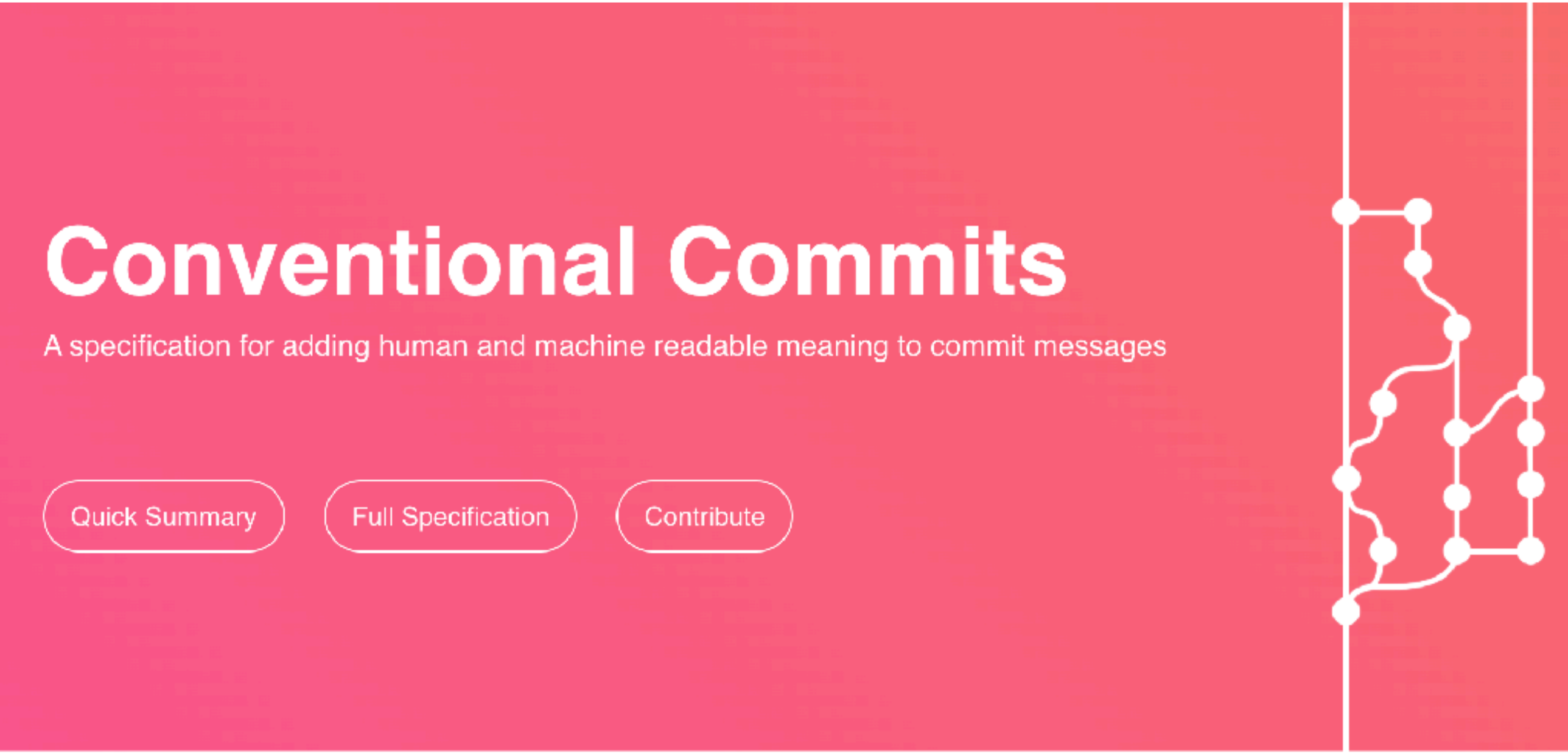
Update

TODO

fixed bug

add feature

Commit Message Format ?

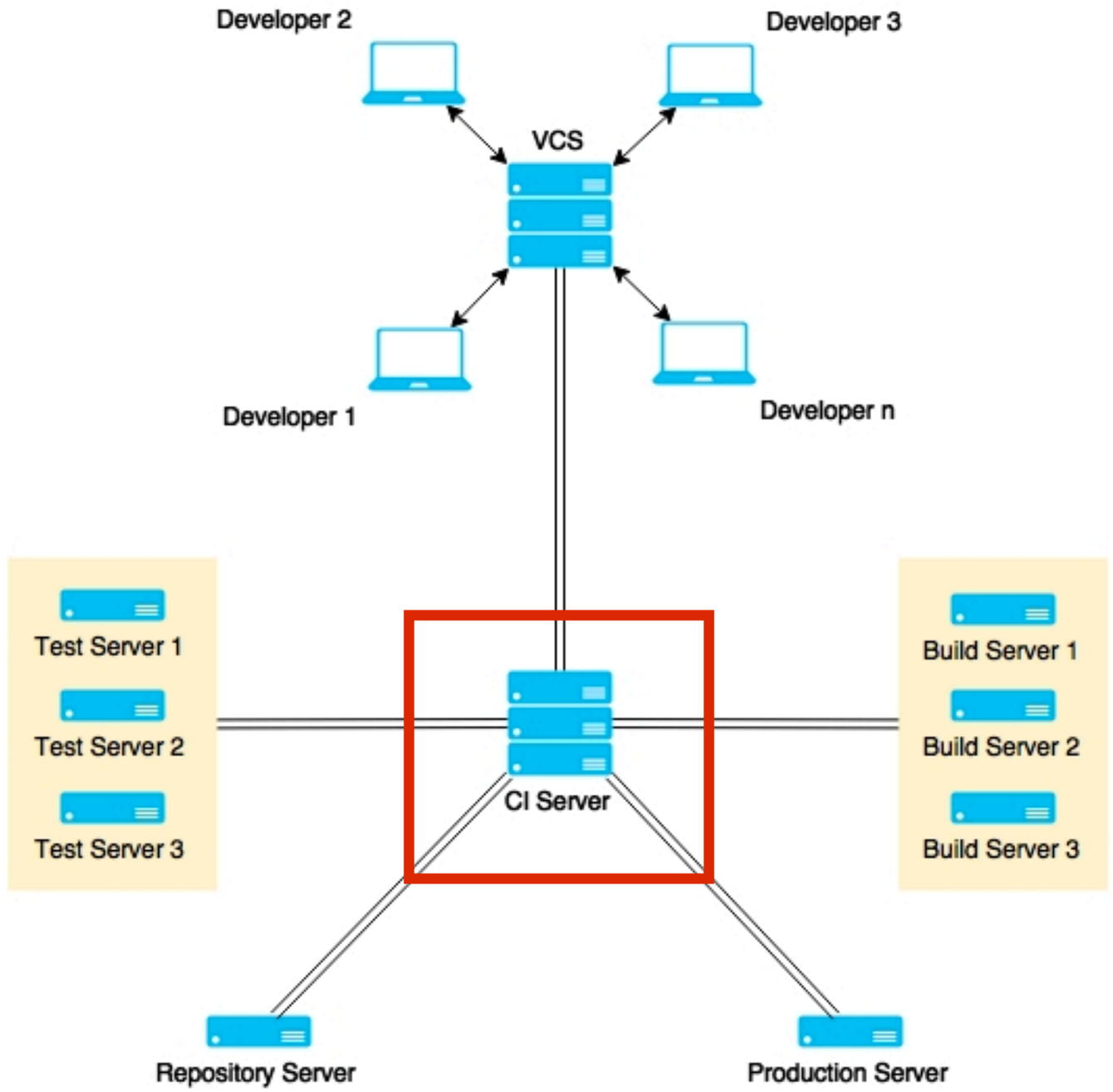


https://www.conventionalcommits.org/

“Added a user object to the database.
currently only has a name and email.
no authentication yet ”

“Fixed the bug that would add something to everything. Turn to not add something to everything”

3. Use good CI tools



CI/CD Tools

Jenkins

Bamboo

TeamCity

go

Visual Studio Team Foundation Server

Hudson

Travis

wercker

circleci

More …

Use static code analysis

Automated testing

Automated deployment

People discipline/habit

“Behind every successful agile project, there is a

Continuous Integration Server”

Let’s start with CI/CD

Application and framework to manage and monitor the executable of **repeated tasks**

Jenkins

https://jenkins.io/

Centralize Continuous Integration Server

Jenkins

https://jenkins.io/

Why Jenkins ?

Easy !!

Extensible

Scalable

Opensource

Large community

Lot of plugins

Who use Jenkins ?

We thank the following organizations for their major commitments to support the Jenkins project.

Microsoft · cloudbees · OSL OPEN SOURCE LAB

rackspace redhat

We thank the following organizations for their support of the Jenkins project through free and/or open source licensing programs.

Atlassian Datadog JFrog Mac Cloud PagerDuty XMission

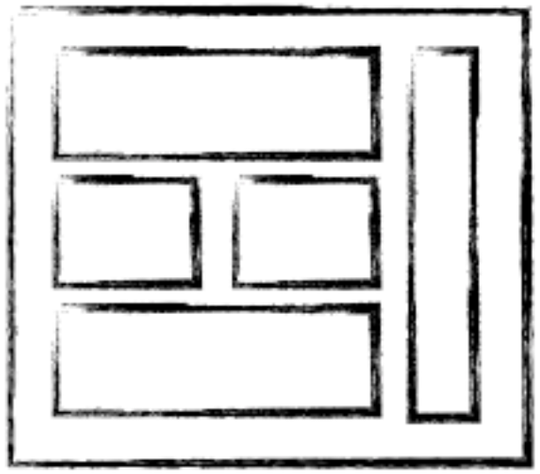
https://jenkins.io/

Workshop with Jenkins

Manage containers

Why docker ?

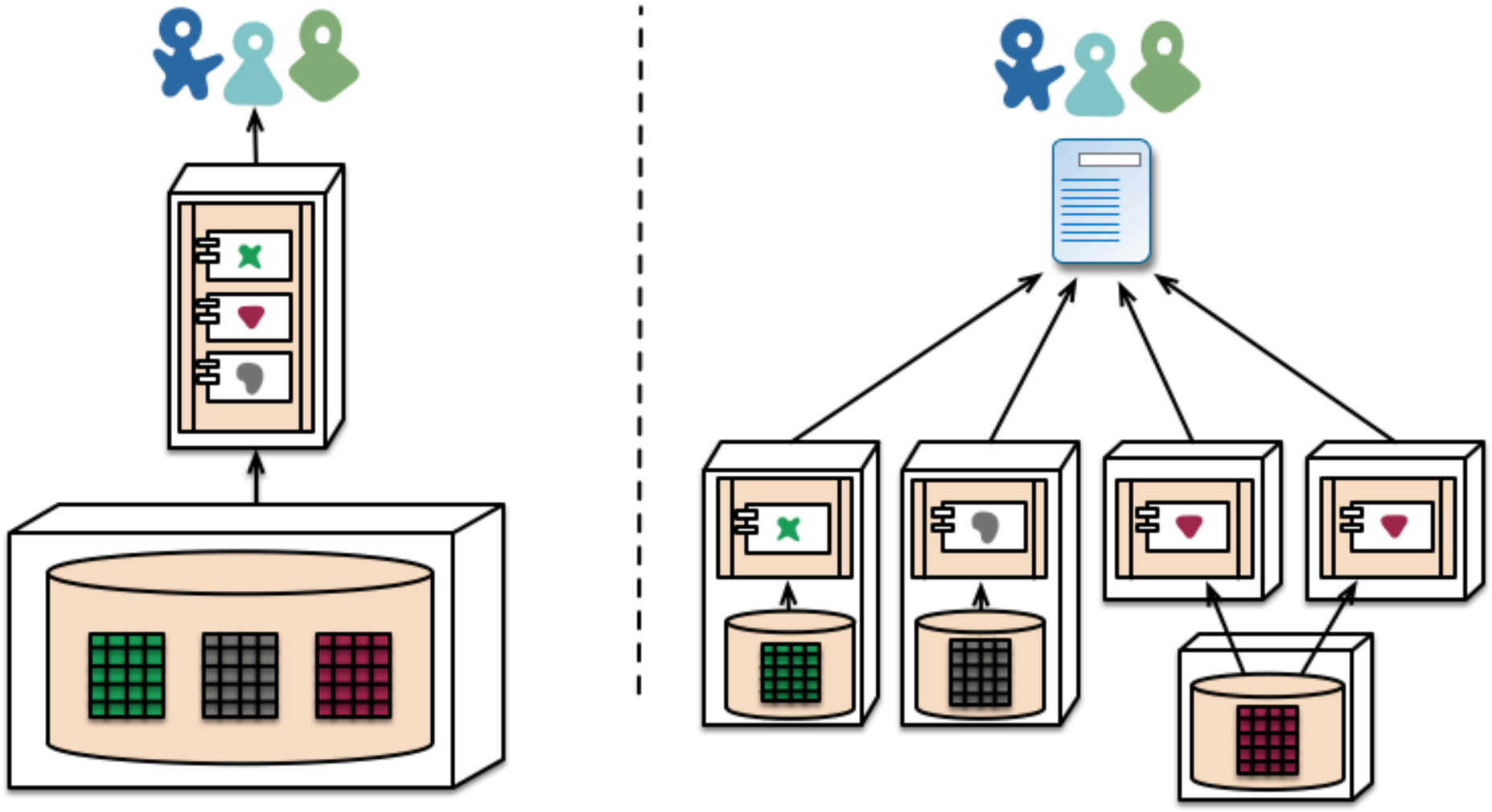
Software industry has changed !!



MONOLITHIC/LAYERED

MICRO SERVICES

Microservice architecture



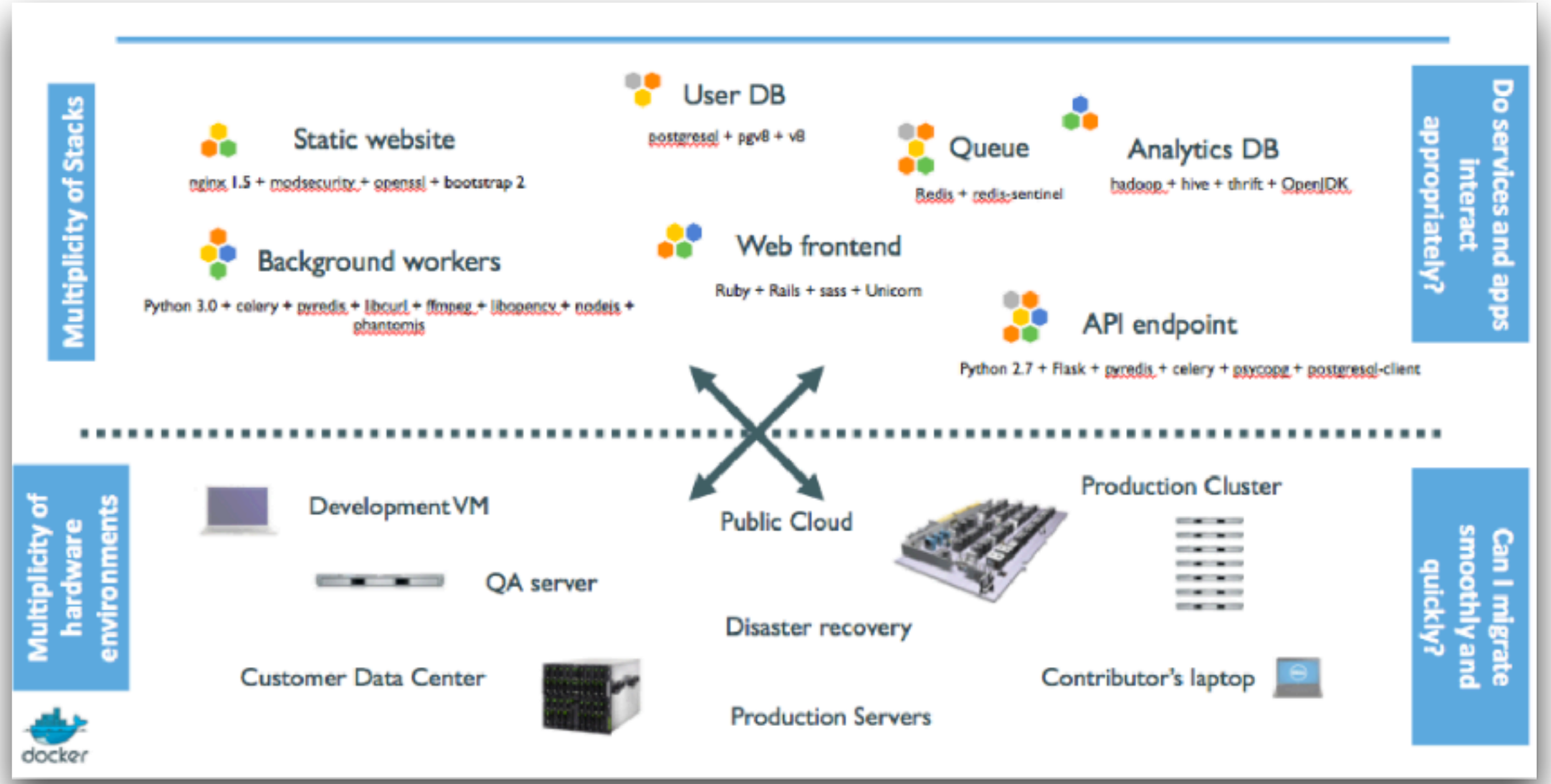
monolith - single database

microservices - application databases

https://martinfowler.com/articles/microservices.html

Why docker ?

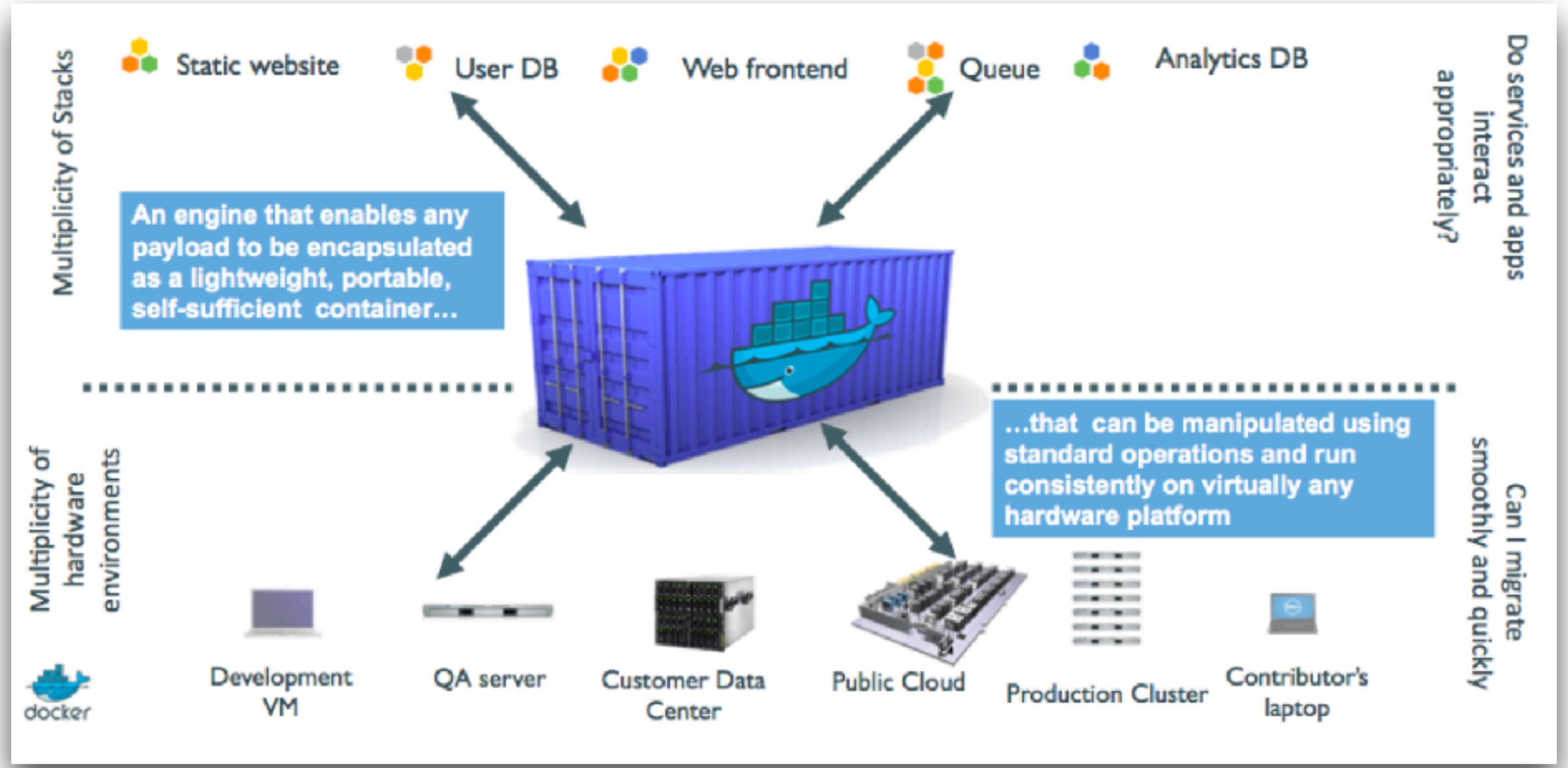
We have problem !!



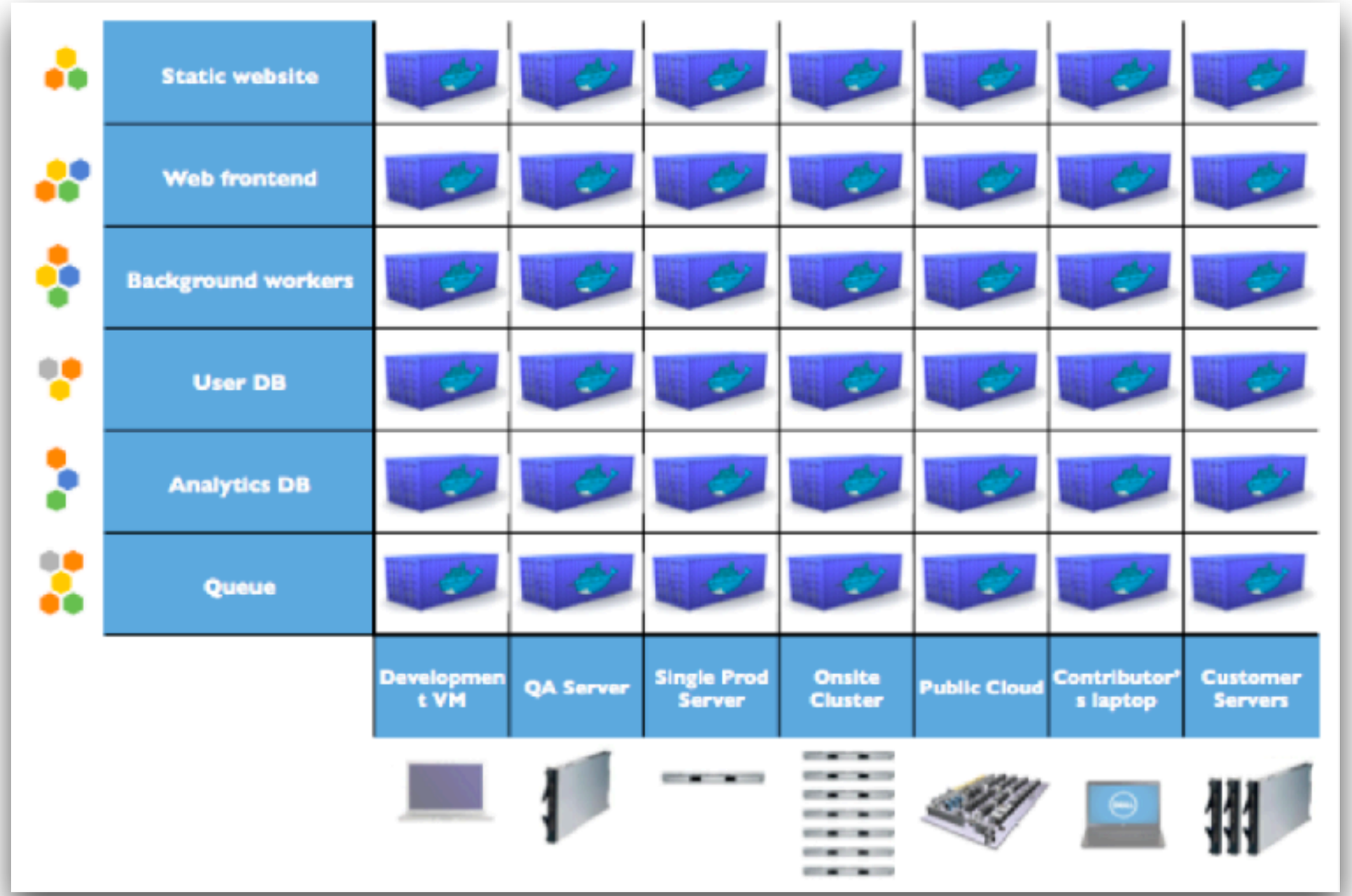
Problem ?

Static website	?	?	?	?	?	?	?
Web frontend	?	?	?	?	?	?	?
Background workers	?	?	?	?	?	?	?
User DB	?	?	?	?	?	?	?
Analytics DB	?	?	?	?	?	?	?
Queue	?	?	?	?	?	?	?
	Developmen t VM	QA Server	Single Prod Server	Onsite Cluster	Public Cloud	Contributor' s laptop	Customer Servers

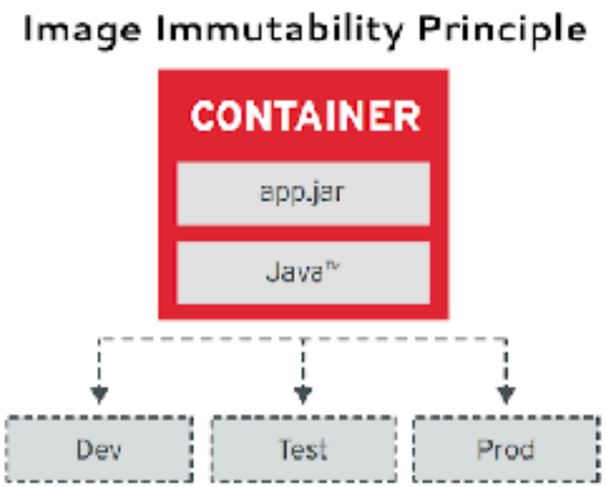
Solution ?

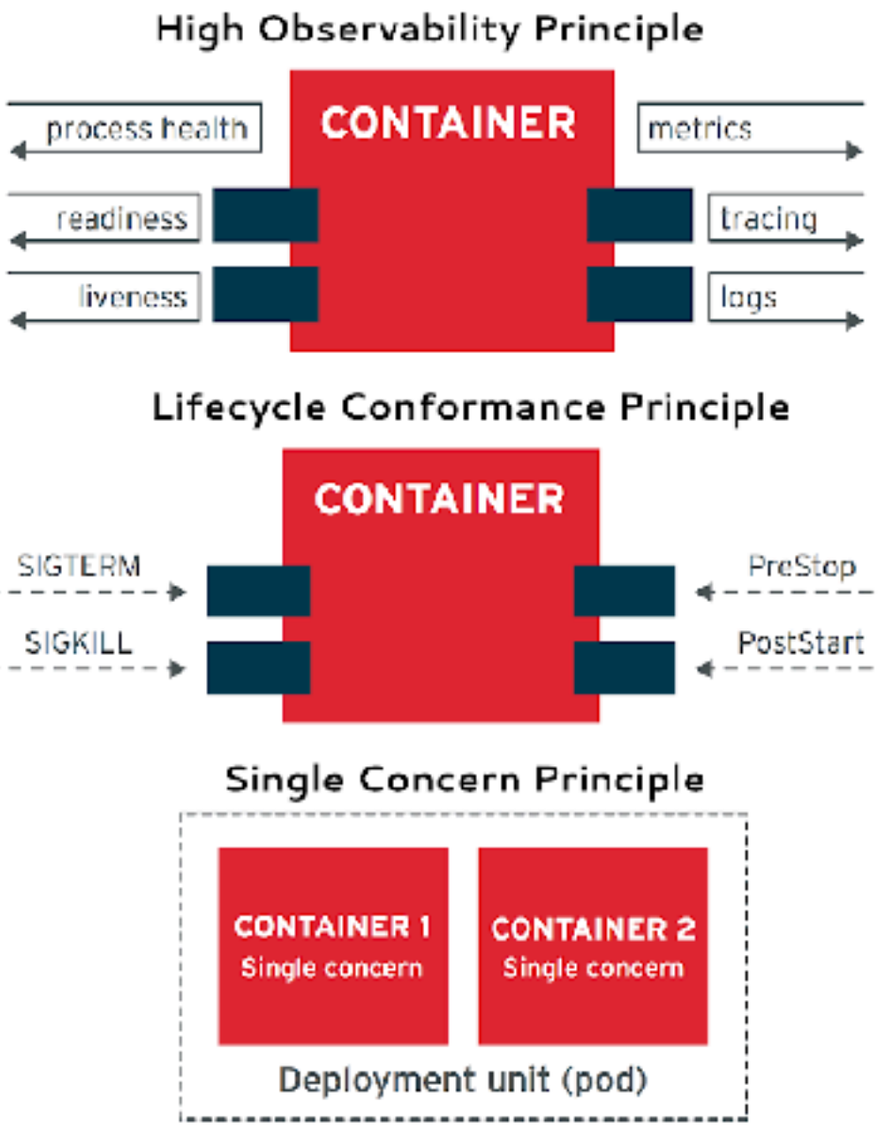


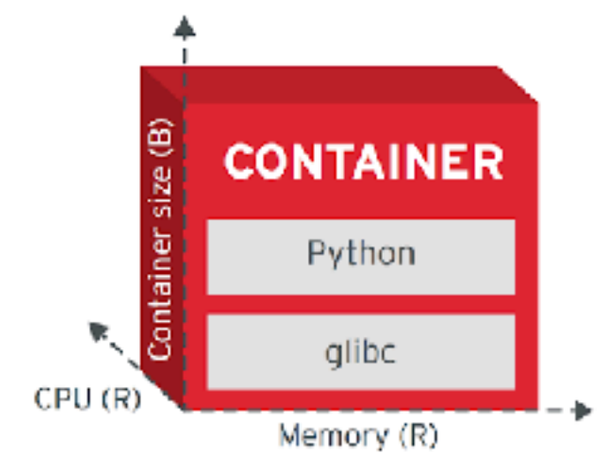
Solution ?

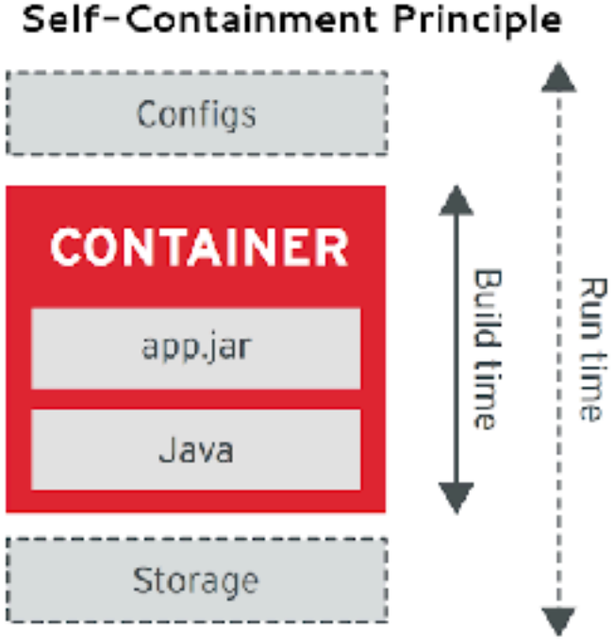


Container Design Principles



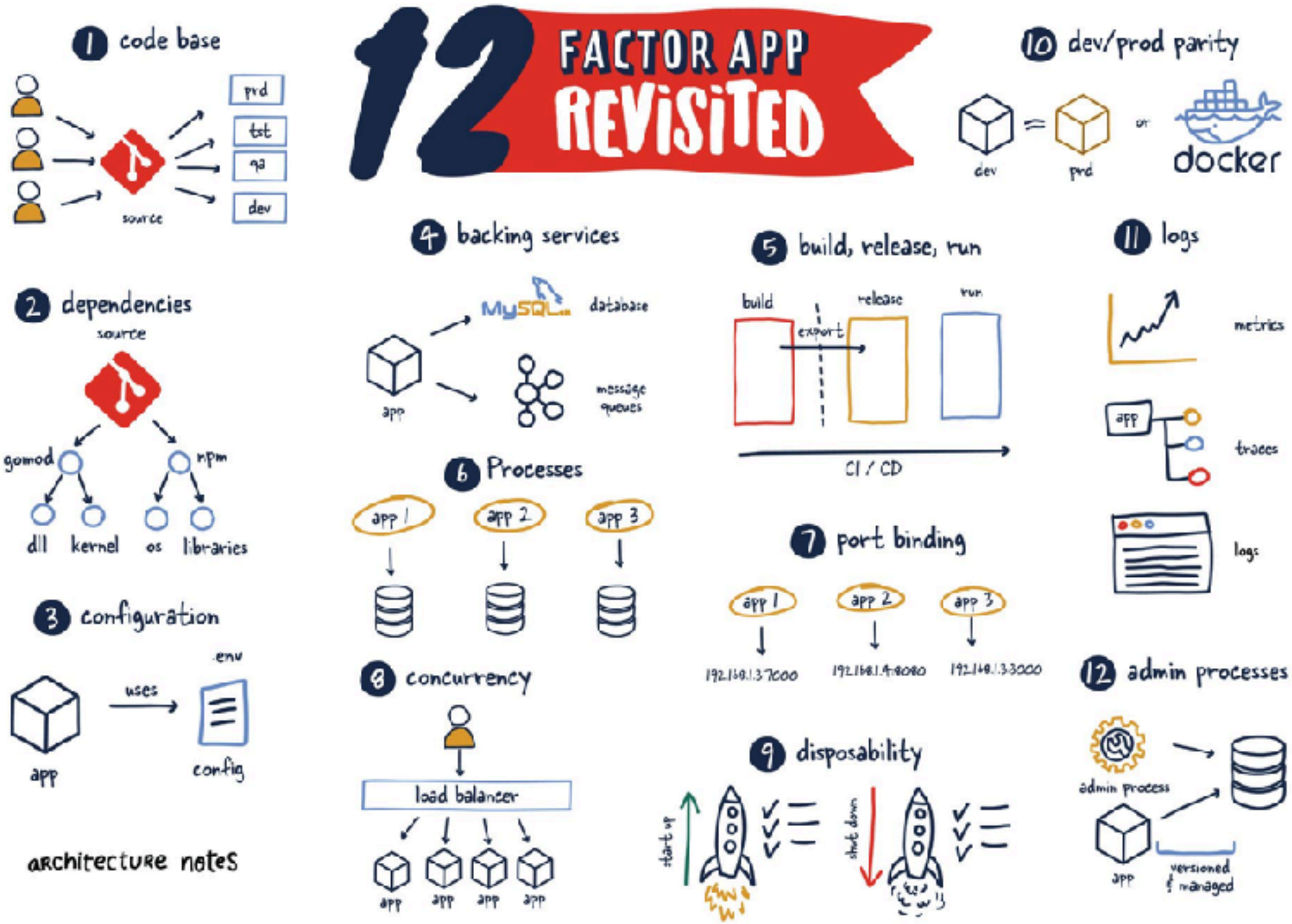






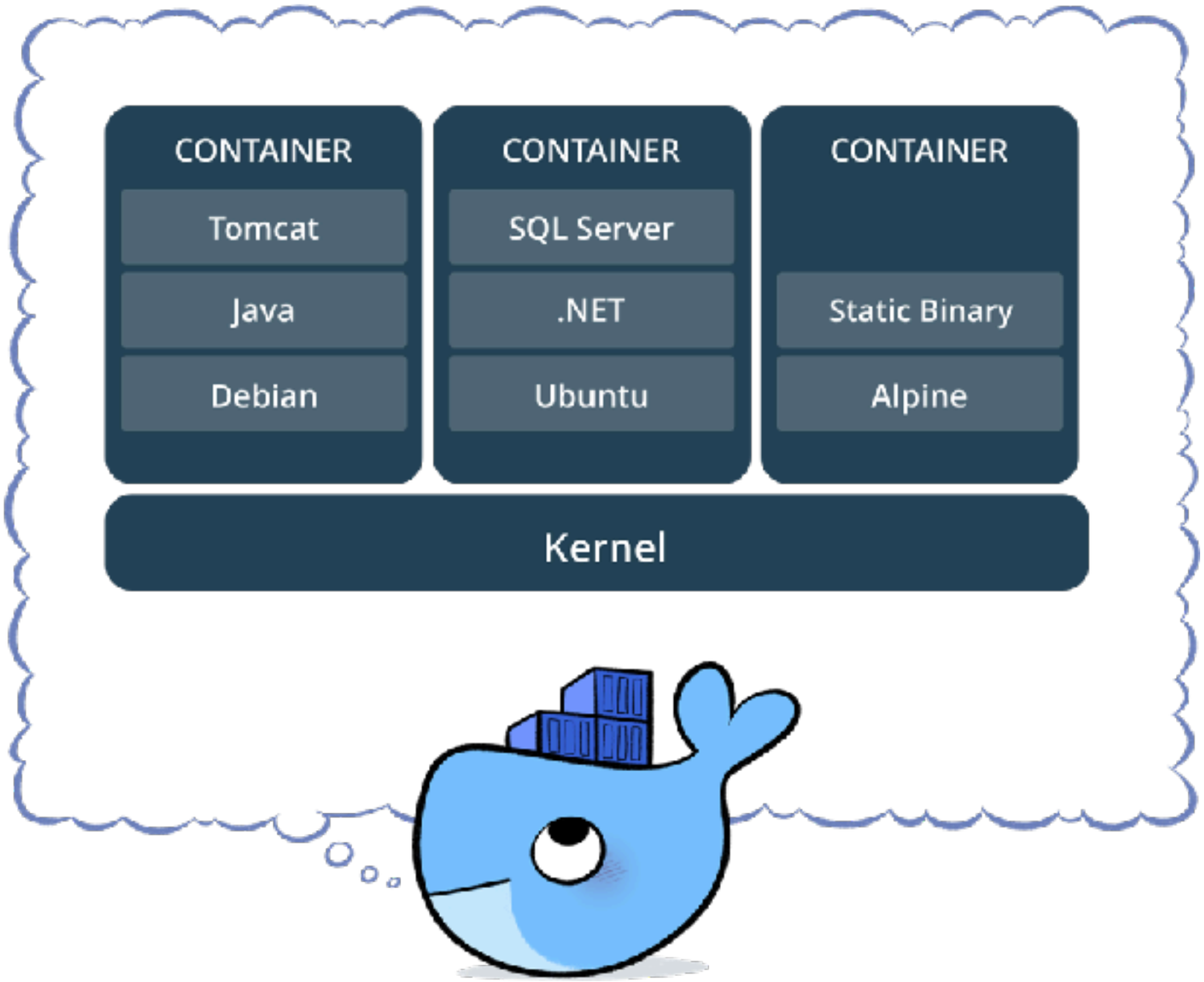
https://kubernetes.io/blog/2018/03/principles-of-container-app-design/

The Twelve-Factor App



https://12factor.net/

Container with Docker



Basic of Docker

Image
Container
Dockerfile
Registry
Volume and network

Image vs Container

Image like template/blueprint
Containers are created from image

Docker image

Collection of files and some meta data

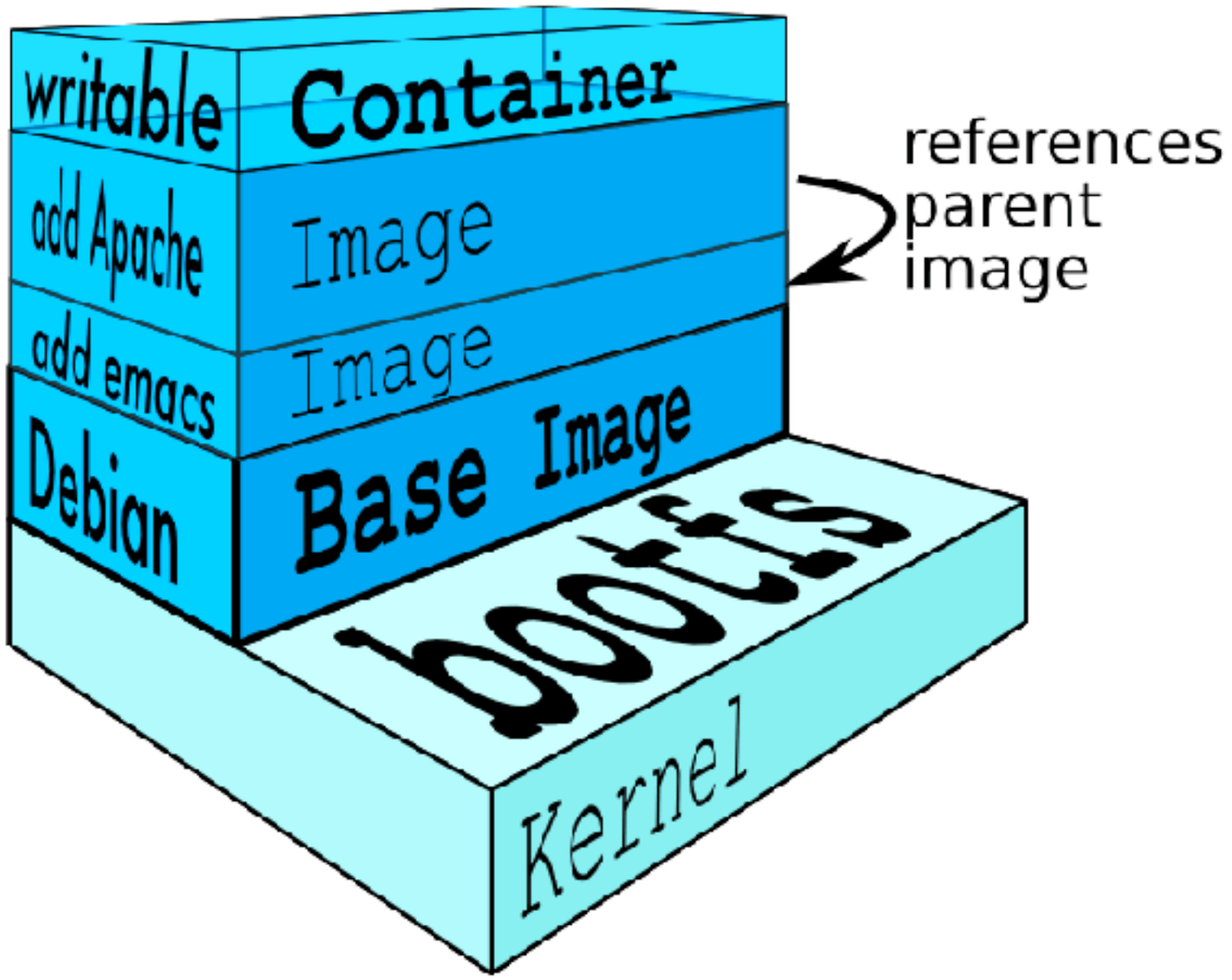
Made of **layers**

Each layer can add/change/remove files

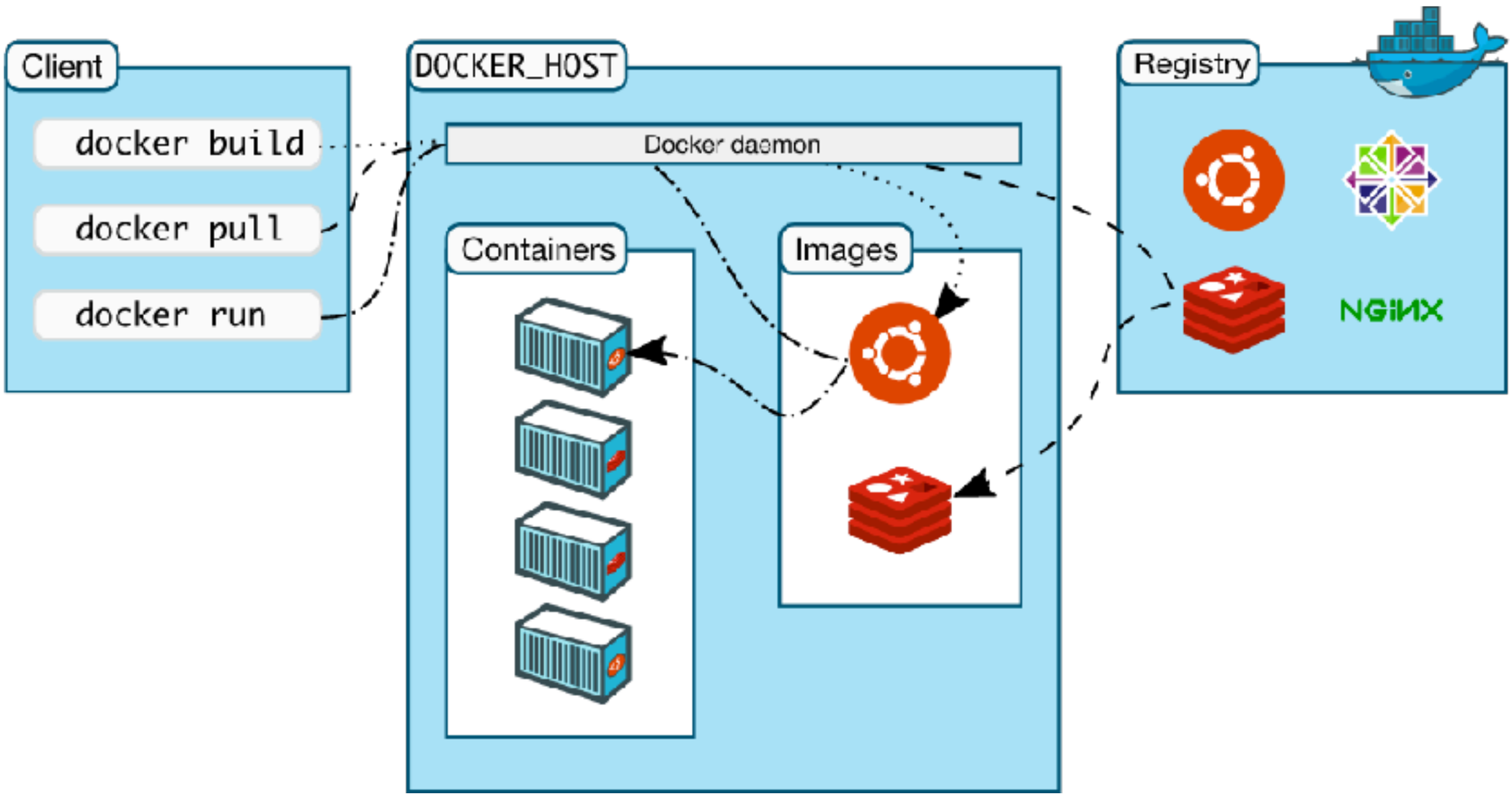
Image can share layers

Read-only file system

Docker image layer

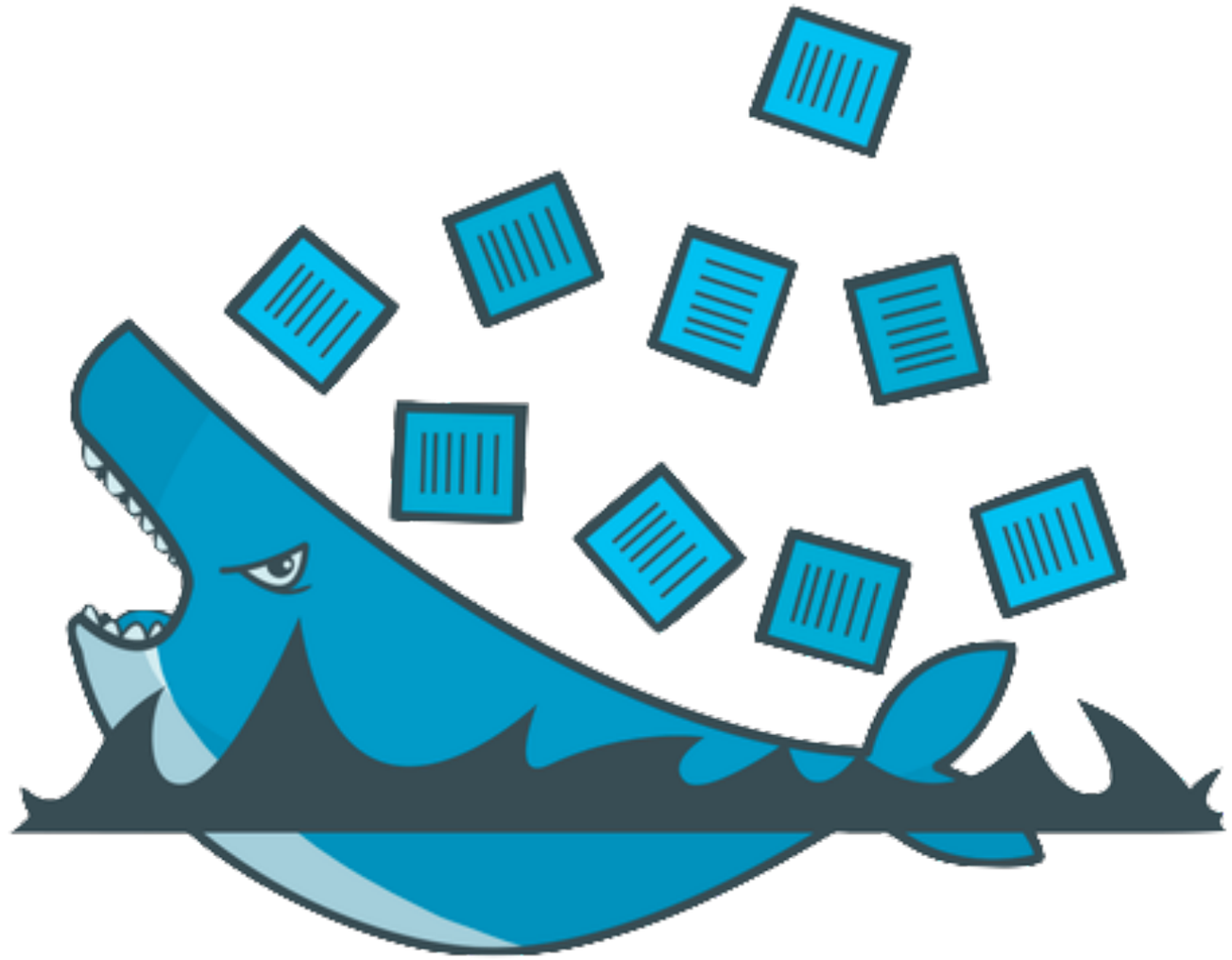


How docker works ?



https://docs.docker.com/get-started/overview/

Workshop



Working with Docker compose

https://docs.docker.com/compose/

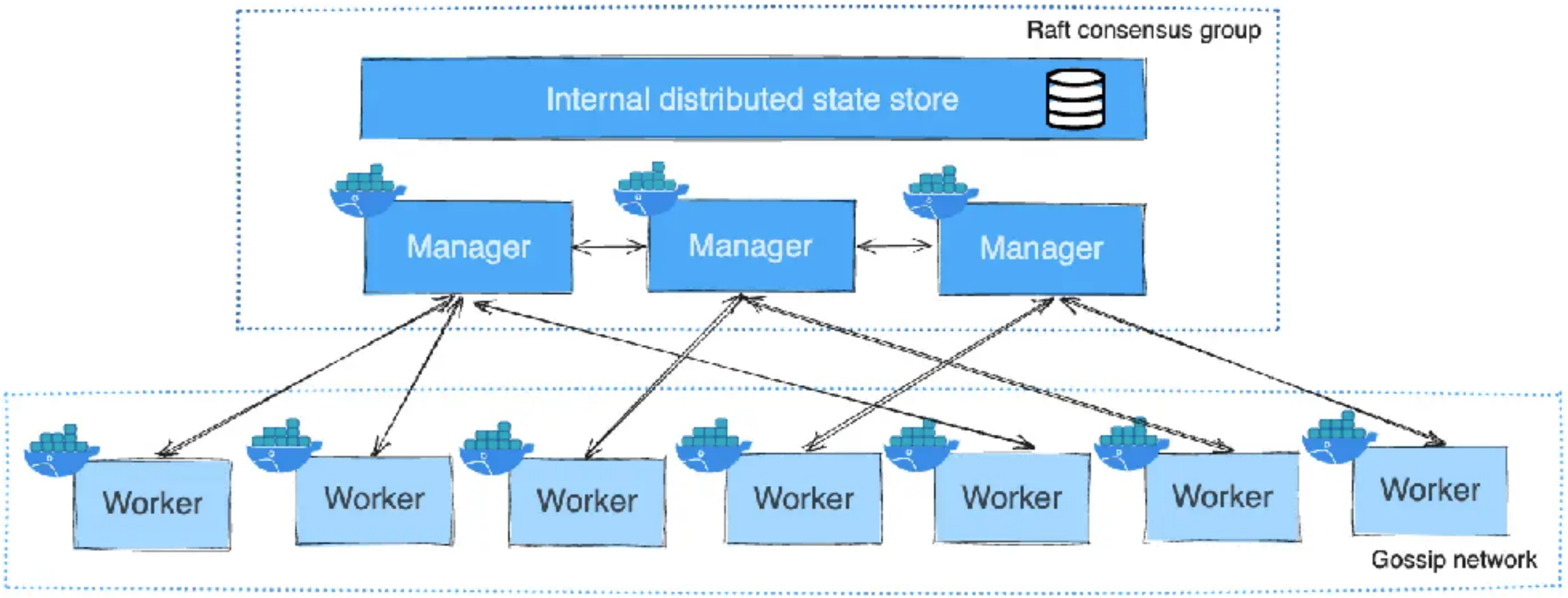
Multilple containers app!!

Difficult to create and manage !!

Mutilple containers app!!

Build images from Dockerfile
Pull images from Hub/private/cache
Configuration and create containers
Start and stop containers
Stream their logs

Docker Swarm



https://docs.docker.com/engine/swarm/how-swarm-mode-works/nodes/

Introduction

Why we need Kubernetes ?

Managing containers for production is challenging
Need something to manage beyond a container engine !!

Key capabilities was missing !!

Using multiple containers with **shared** resources

Monitoring running containers

Handling **dead containers**

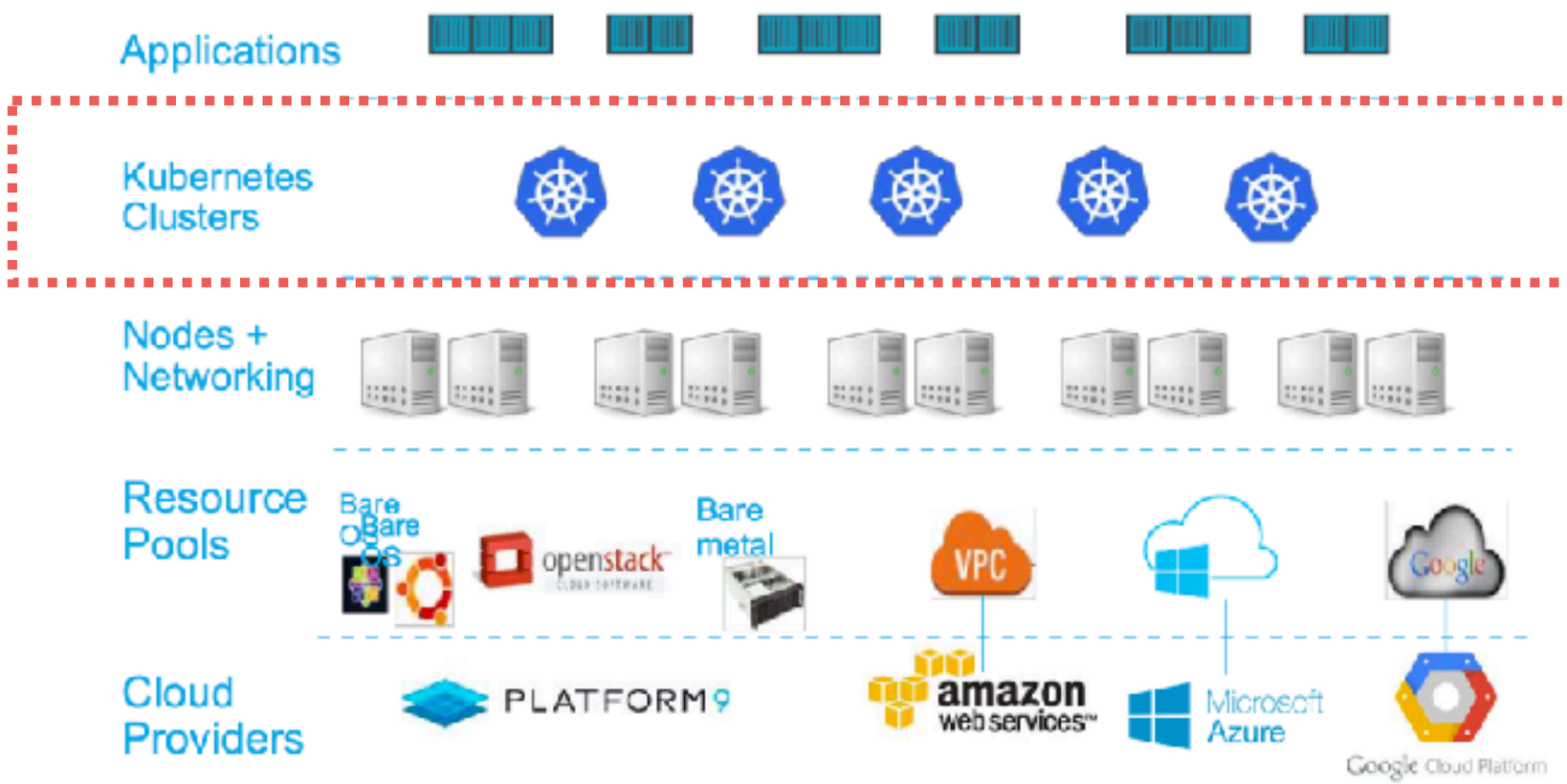
Autoscaling container instances to handle load

Making the container services **easily** accessible

Handling **dead connecting** containers to a variety of external data sources

Write once, run anywhere

Eliminate infrastructure lock-in
Use containers
Provides management for containers



Deployment, not infrastructure

Software deployment is hard
Infrastructure provisioning/re-provisioning
Configuration networking and load balance
Redundancy (scale-out)
Lifecycle management (Software update)

K8s support for deployment

Scale-out service

Rolling update for new version

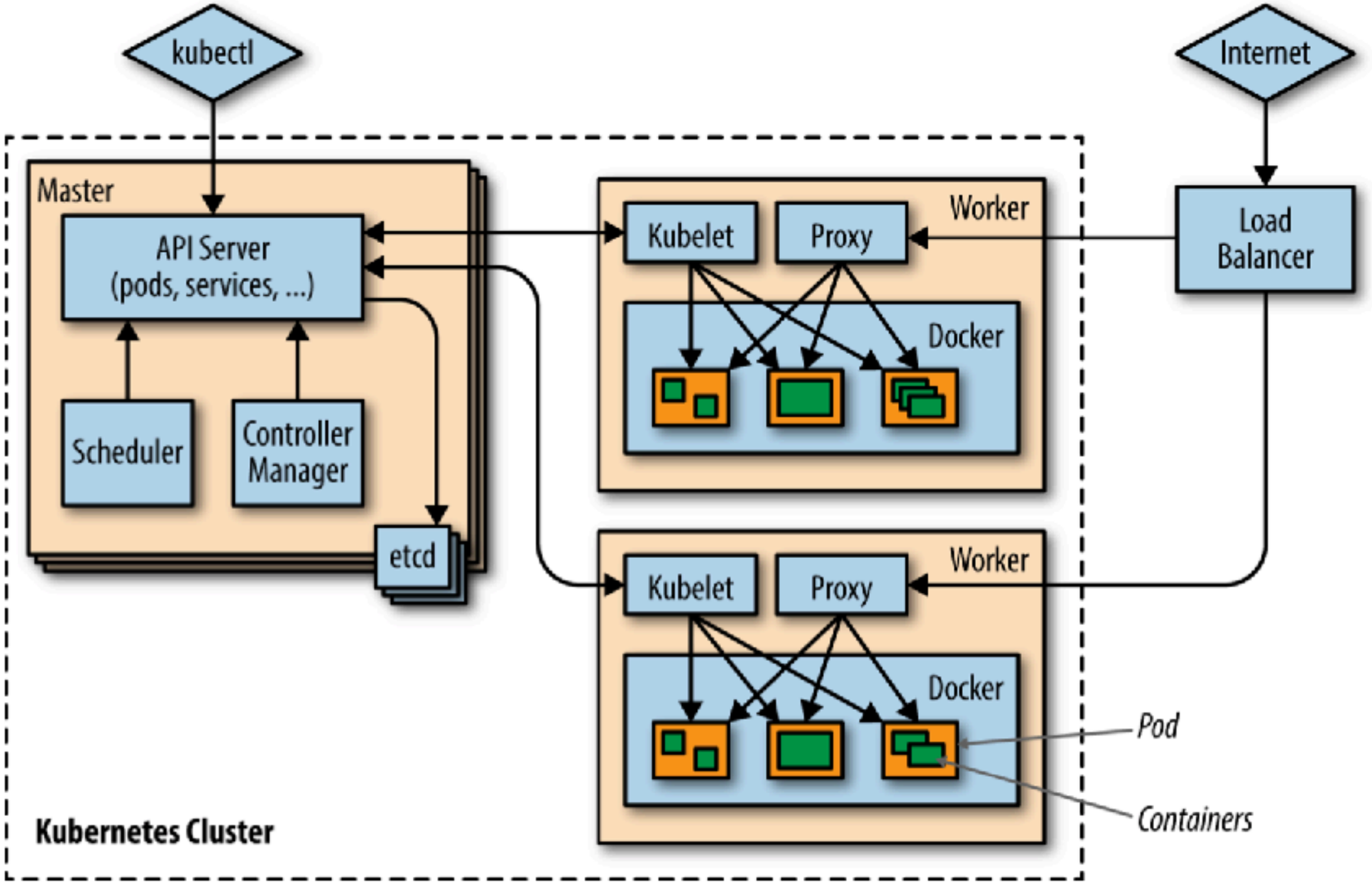
Rollback to a previous version

Pause and resume a deployment

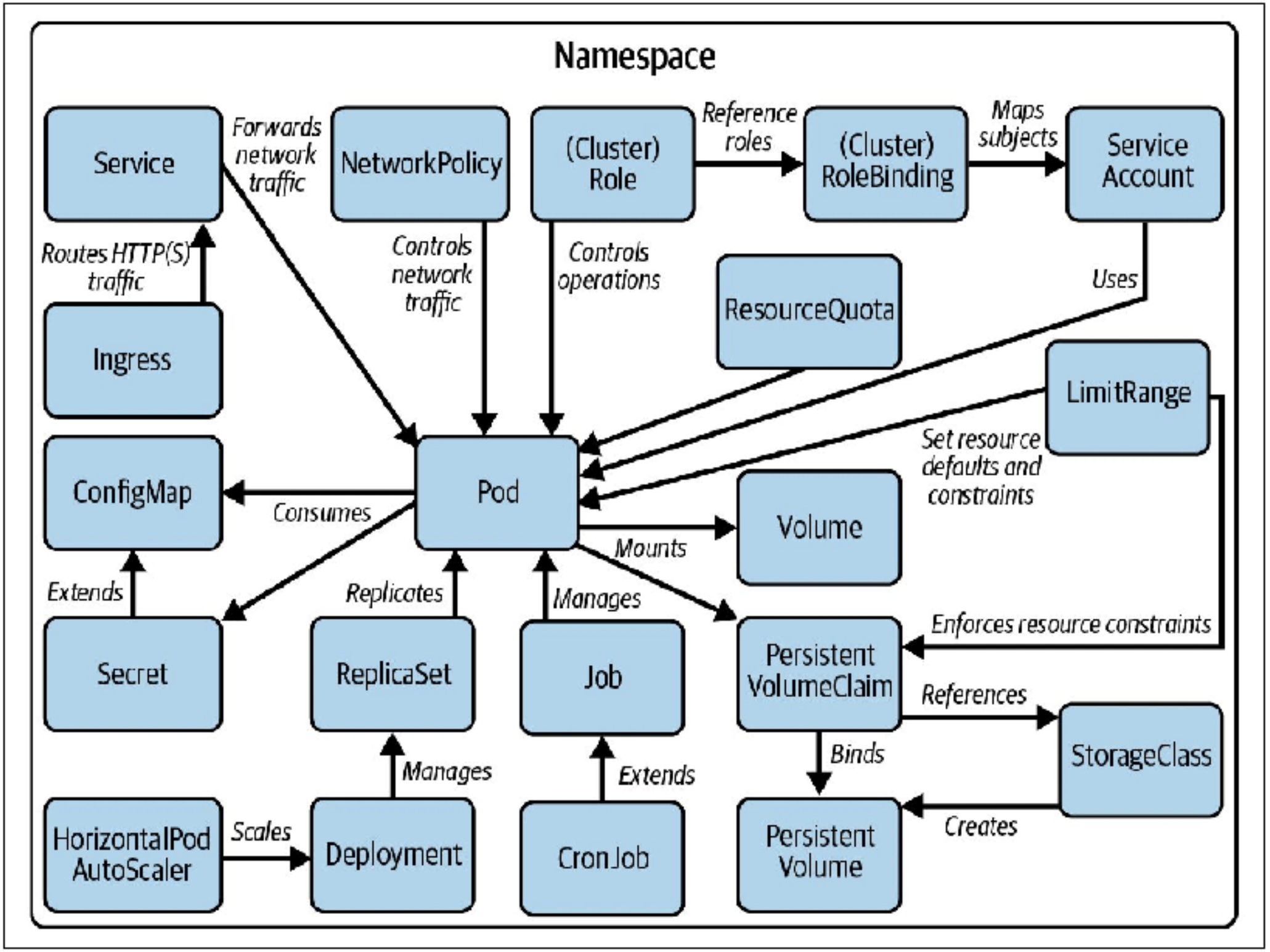
Horizontal auto-scaling

Canary deployment

Kubernetes Architecture



K8s components

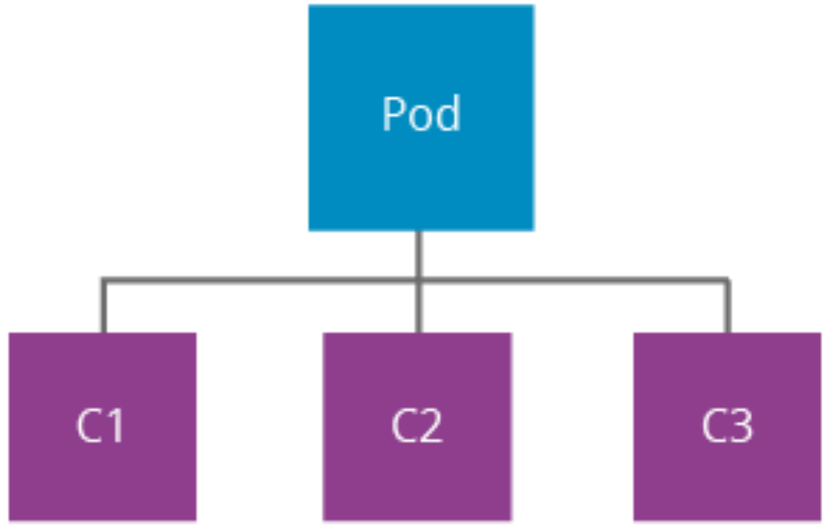


K8s components

Pods
Services
Deployments
ConfigMap and Secret
Volumes
StatefulSets

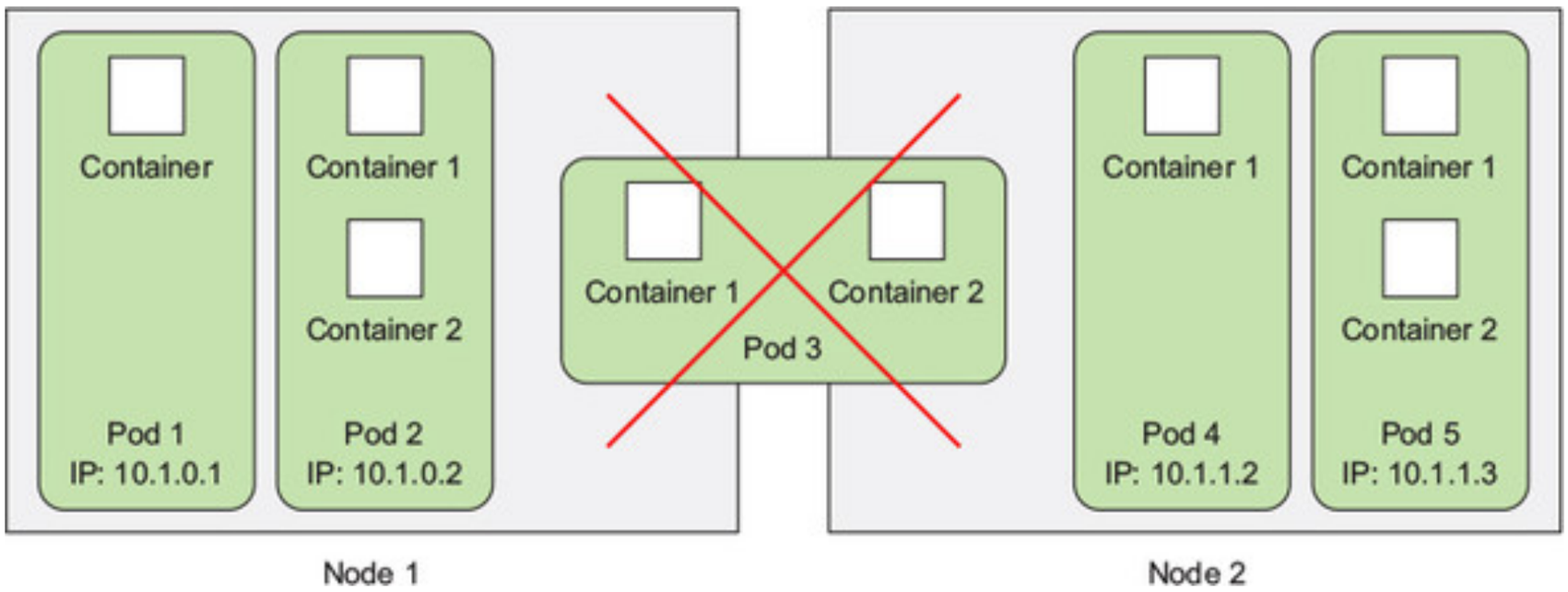
Pods

Small group of co-located containers
Optionally shared volume between containers
Basic deployment unit in Kubernetes



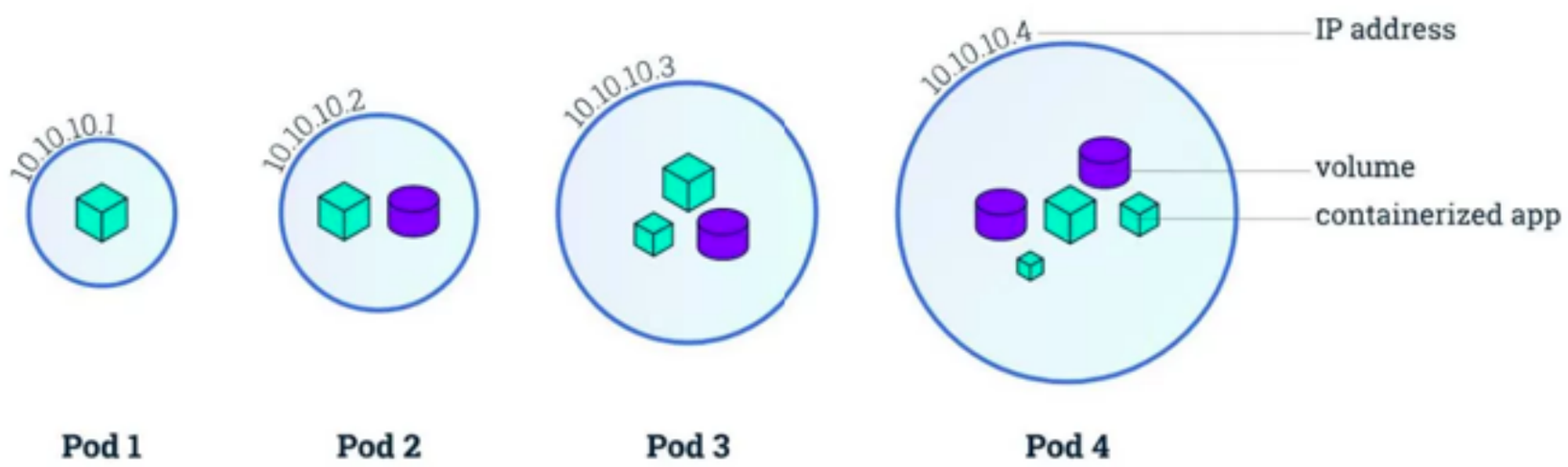
Pods

1 pods = 1 container
1 pods = N containers



All containers in same Pods

Share process ID
Share network interface
Share Hostname/IP/Port
Share Unix Time Sharing (UTS)

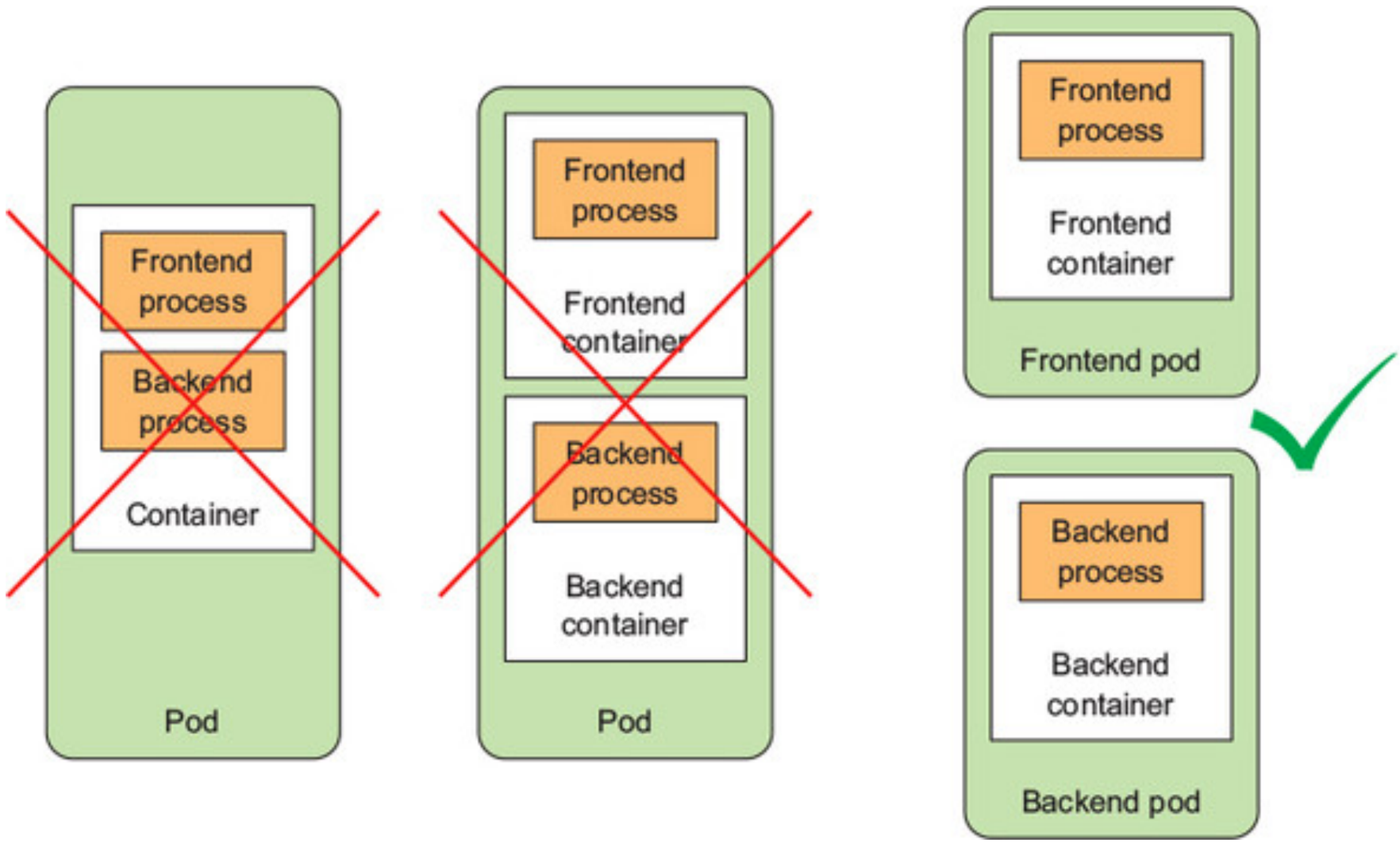


Pods Lifecycle

Phase	Description
Pending	Accepted by kubernetes but container not created yet
Running	Pods bound to the node, all containers created and at least one container is running/starting/ restarting
Successed	Containers exited with status 0
Failed	All containers exit and at least one exited with non-zero status
Unknow	State of Pods can’t be determined due to communication issues with its node

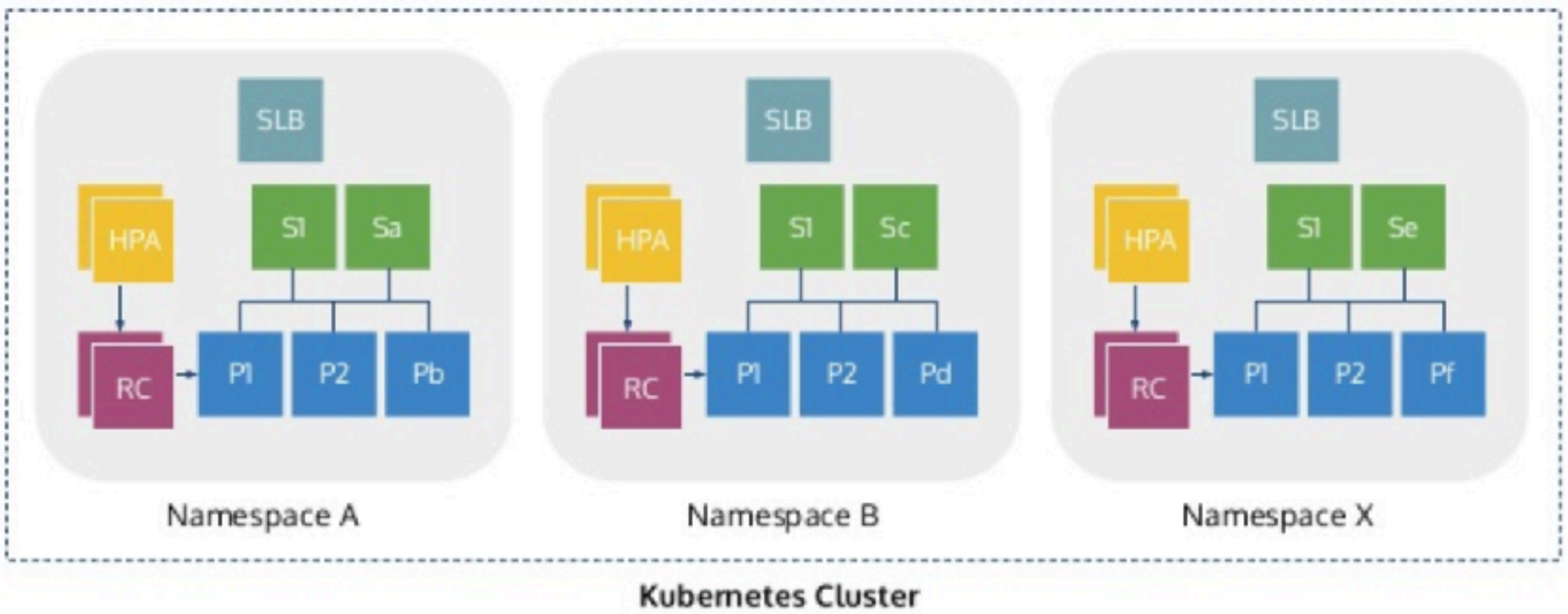
Organize container across Pods

Split multi-tiers app into multiple pods



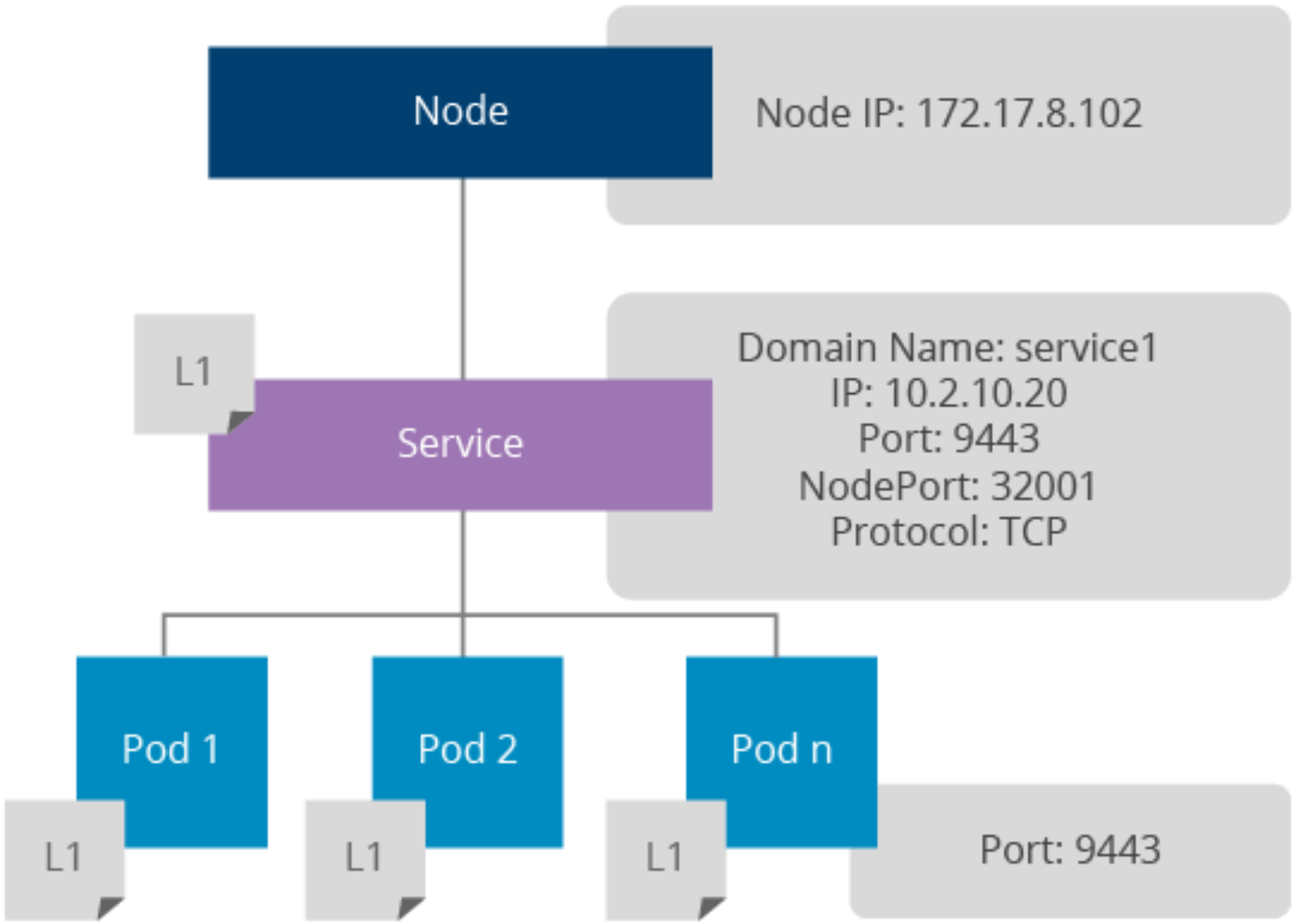
Pods Namespaces

Allow different teams to use the same cluster



Services discovery and Load balancing

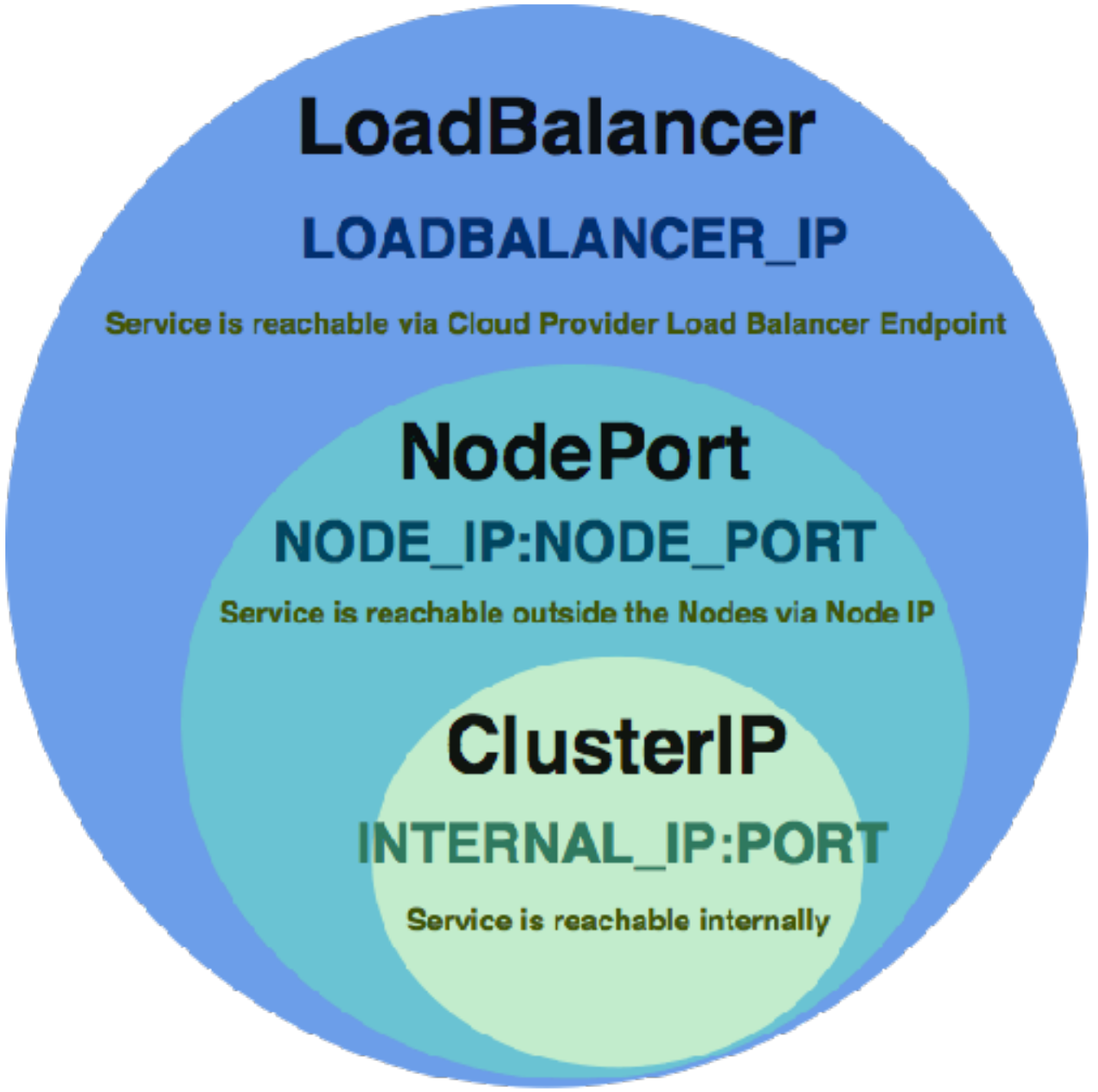
Services



Services

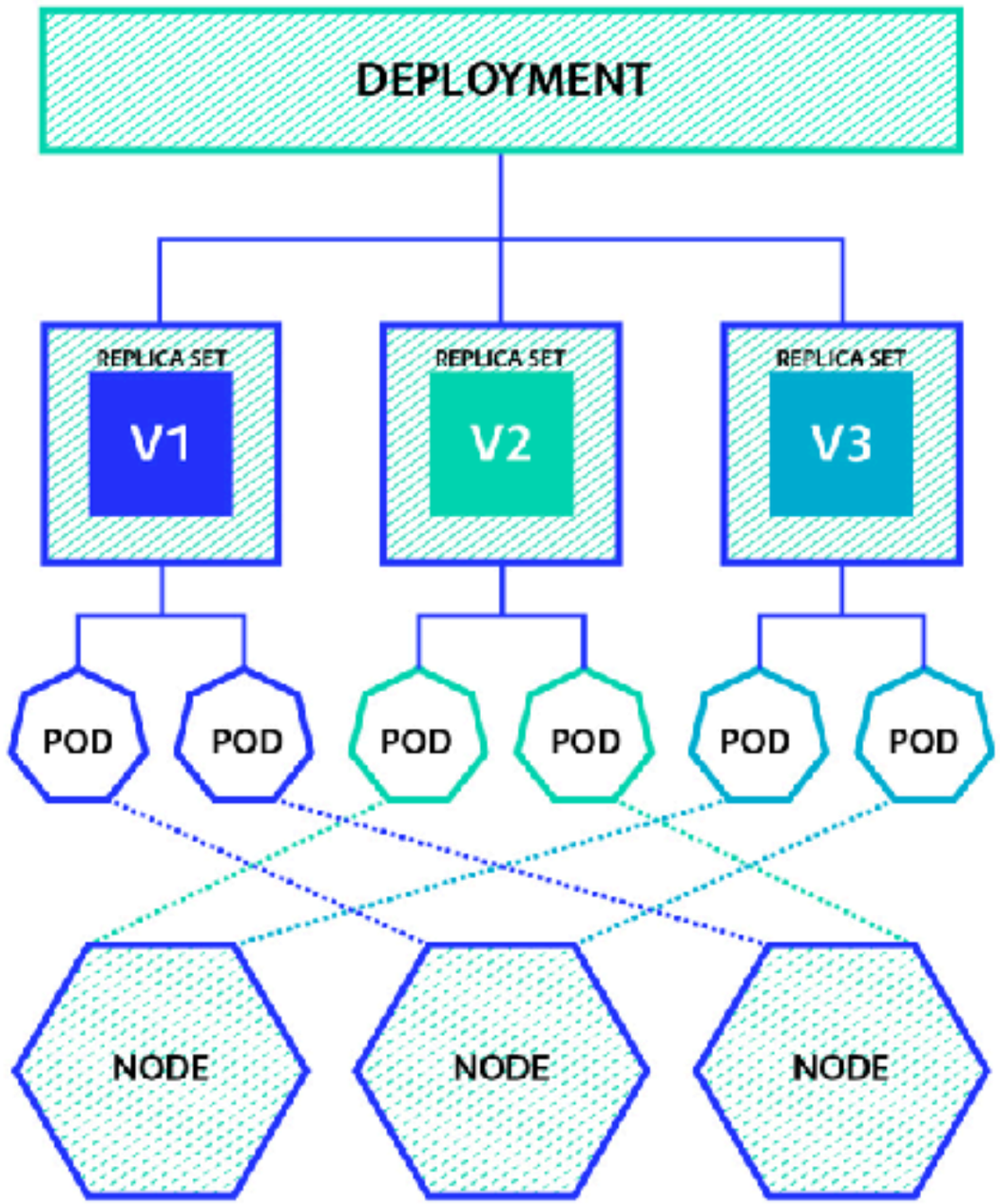
Independent from Pods
Abstraction layer of Pods
Provide load balance
Expose access Pods/Load balance
Find Pods by label selector

3 types of services



Deployment ReplicaSet (RS)

Deployment and ReplicaSet



Deployment and ReplicaSet

Next-generation of Replication Controller

Provide function to maintain versioning of Pods

Update new version (Rollout)

Revert to old version (Rollback)

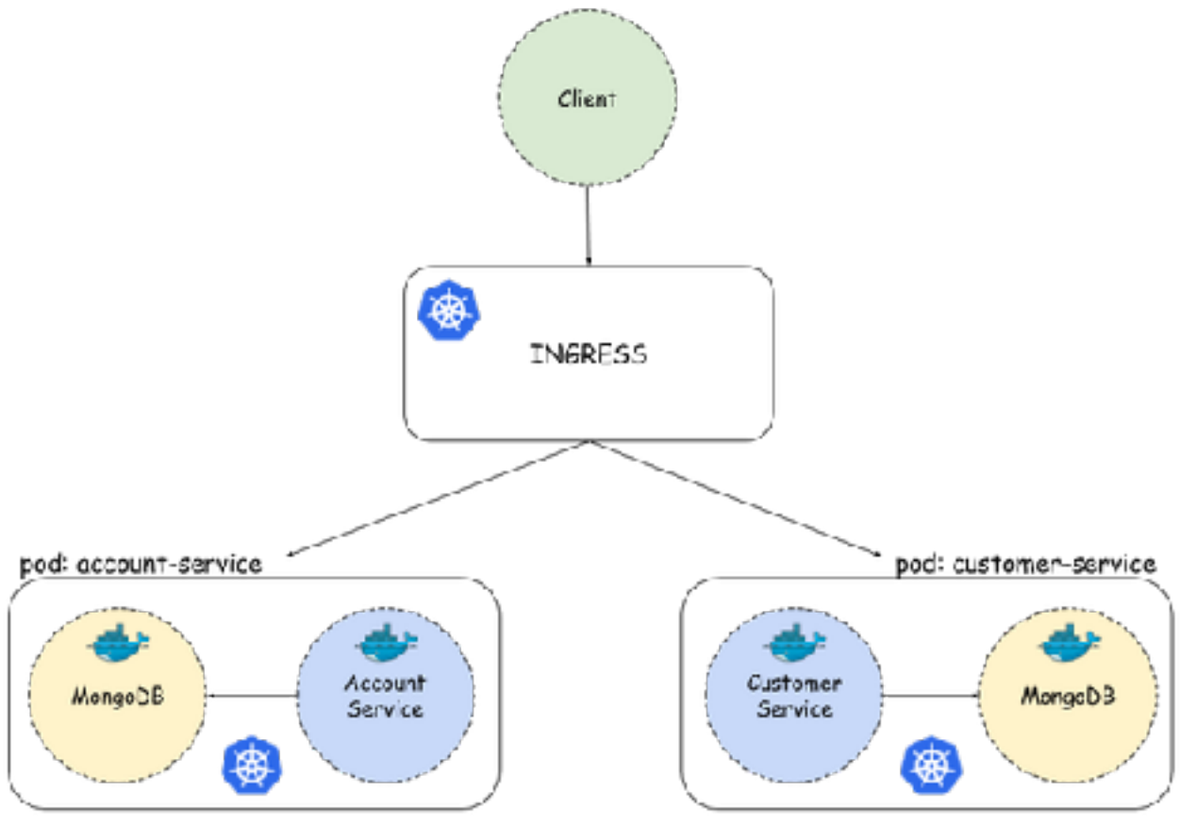
Scale a deployment

Pause/Resume process

Ingress network

Ingress Network

How to handle multiple services in same port ?
How to limit protocol to access ?



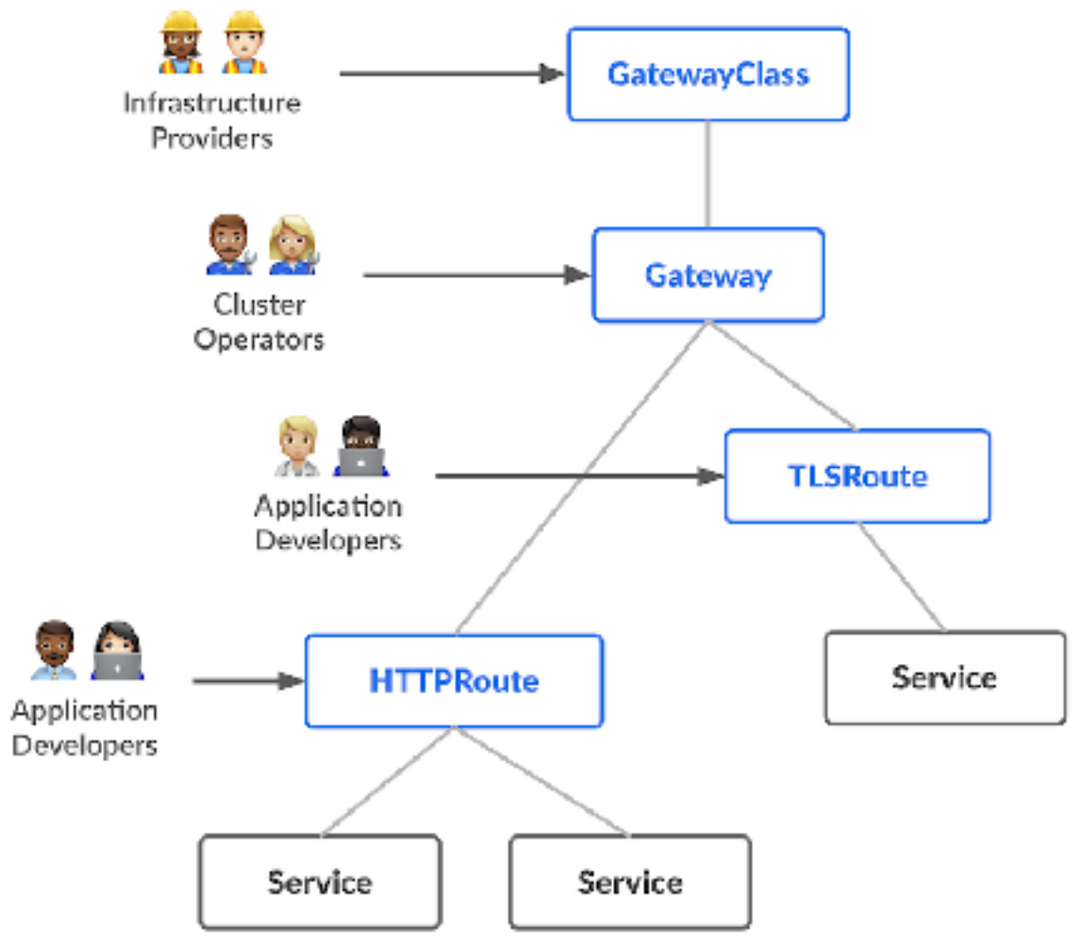
https://kubernetes.io/docs/concepts/services-networking/ingress/

Gateway API

https://kubernetes.io/docs/concepts/services-networking/gateway/

Gateway API

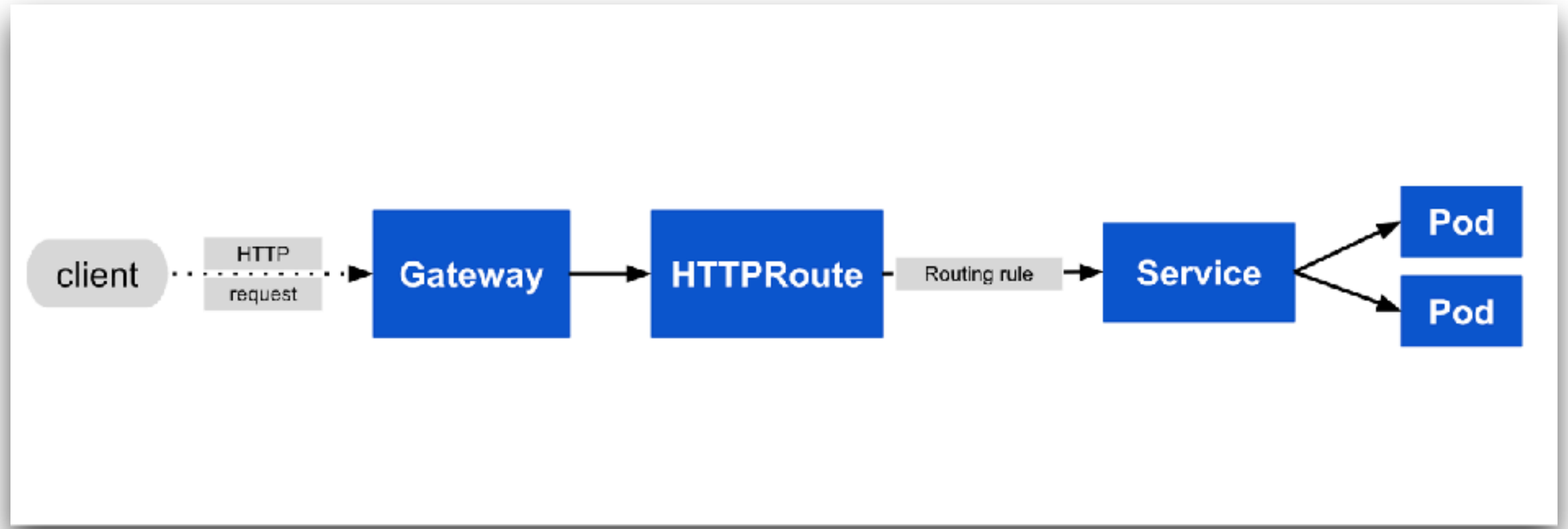
Dynamic infrastructure provisioning
Advance traffic routing



https://gateway-api.sigs.k8s.io/

Gateway API

Request flow of HTTP traffic



https://gateway-api.sigs.k8s.io/

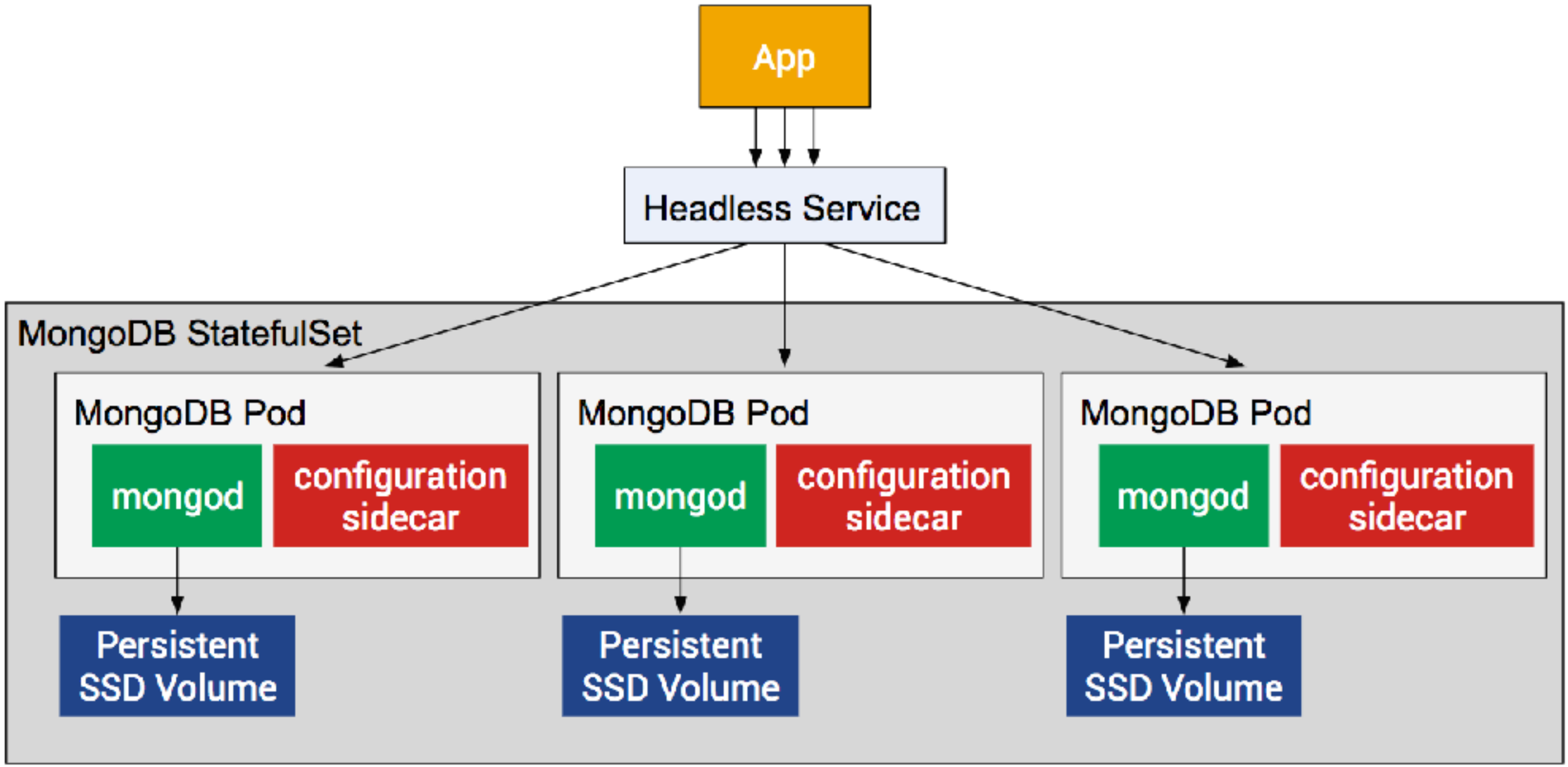
StatefulSet

StatefulSet

Bringing the concept of ReplicaSets to **stateful** Pods
Enable running Pods in **cluster** mode
Ideal for deploy **highly** available database **workload**

Pods are created **sequentially**
Pods are terminated in **LiFo** (Last in, First out)

StatefulSet

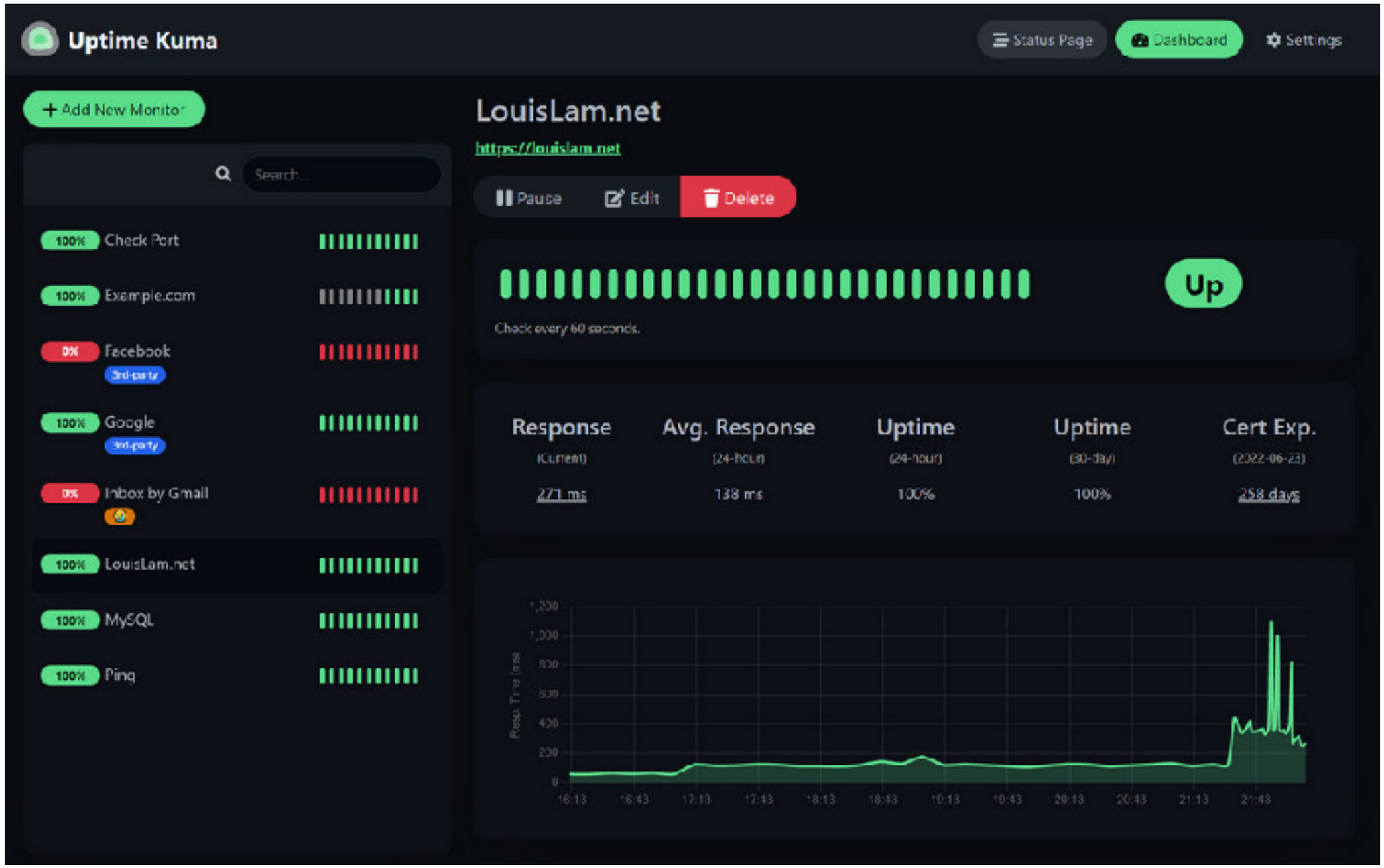


Workshop

kubernetes

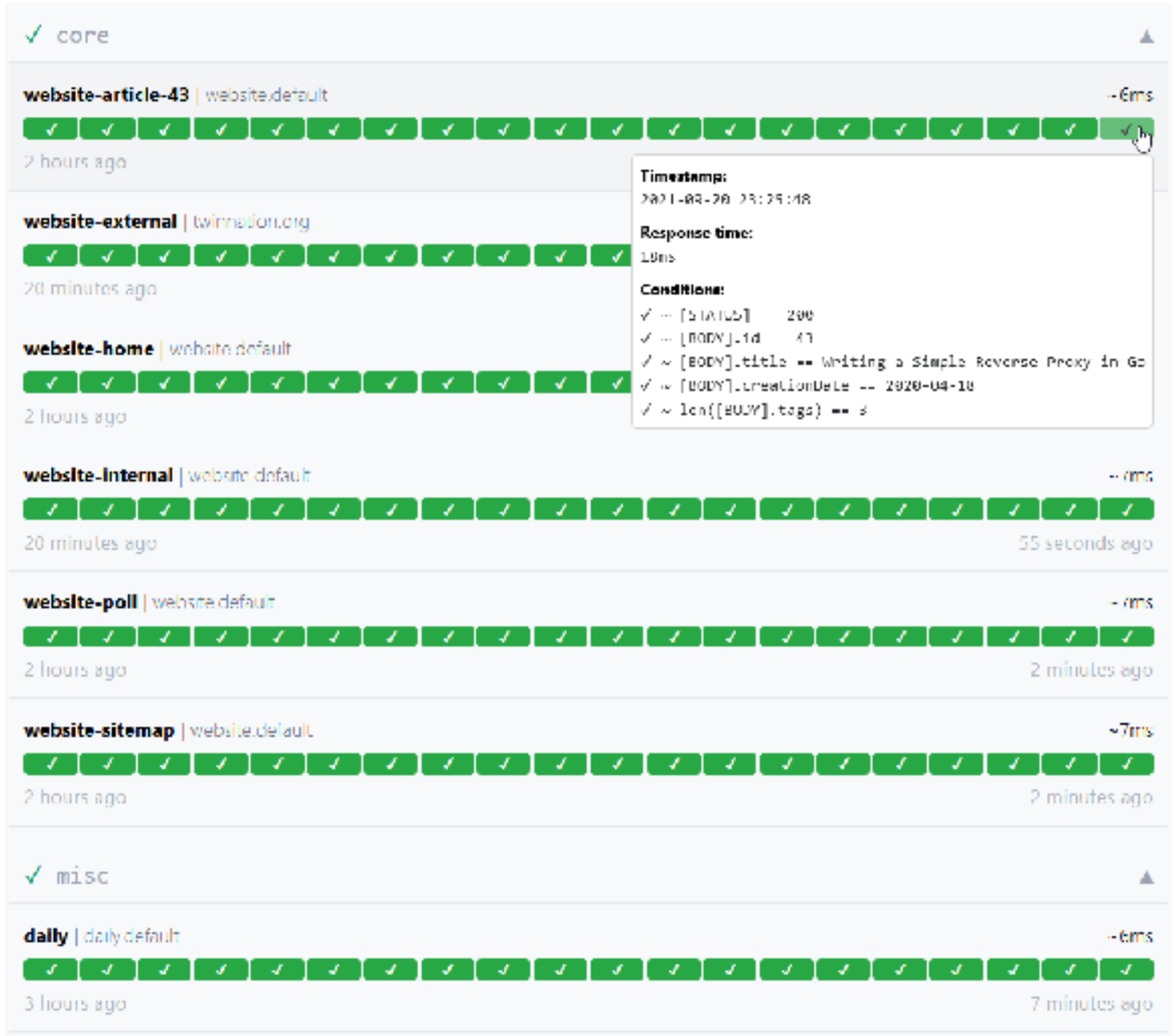
Observability Monitoring

Uptime Kuma



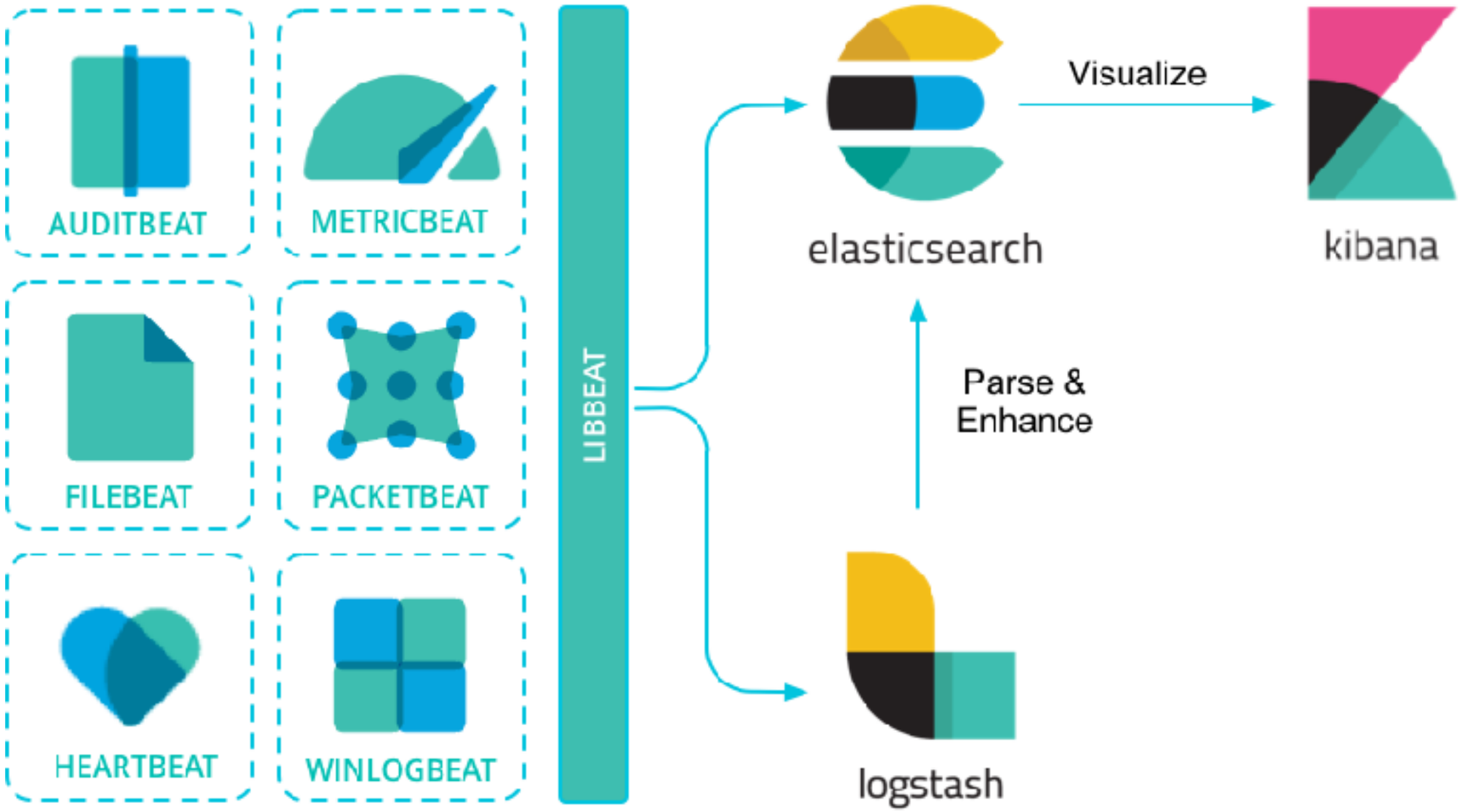
https://github.com/louislam/uptime-kuma

Gatus



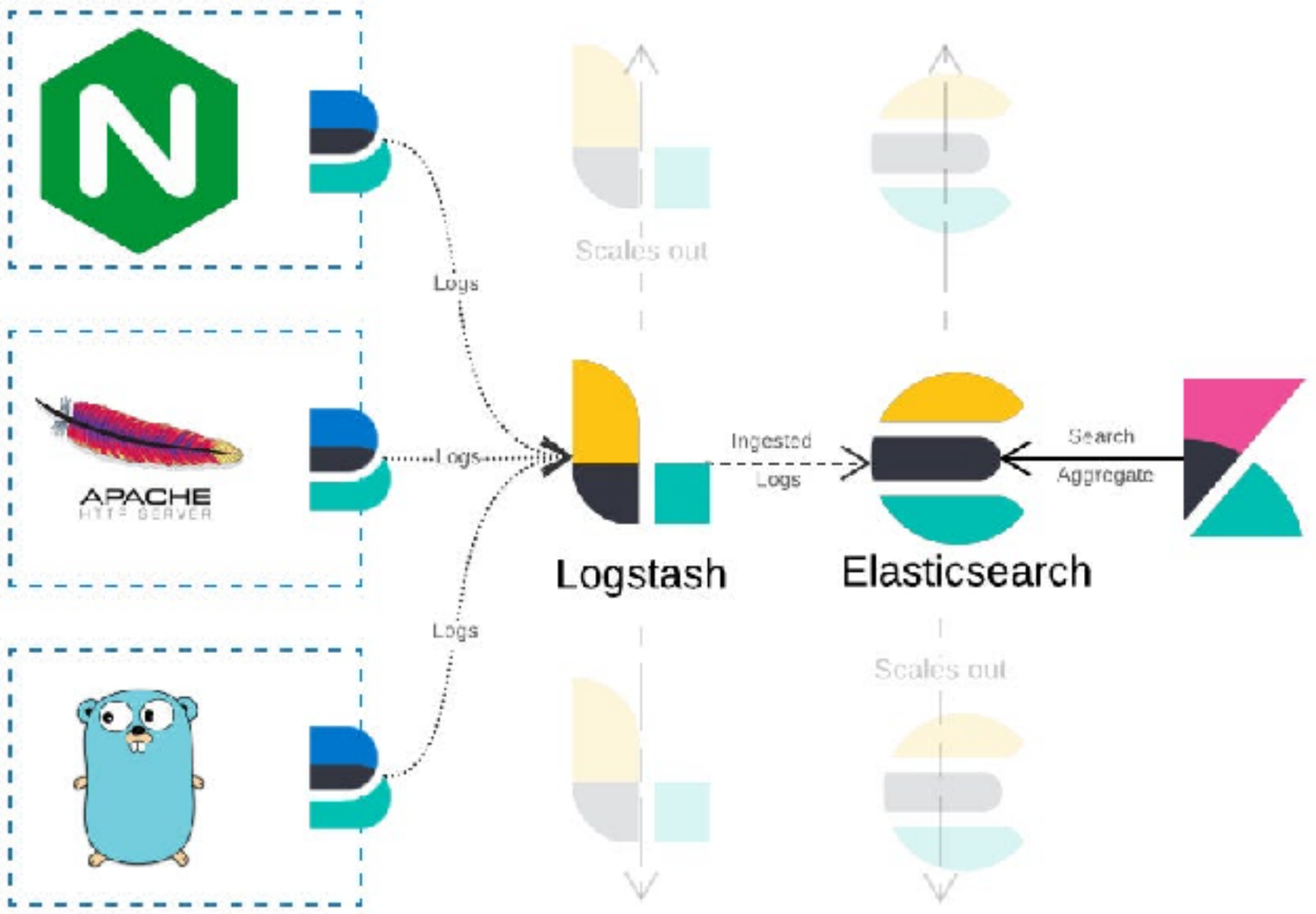
https://gatus.io/

ELK Stack



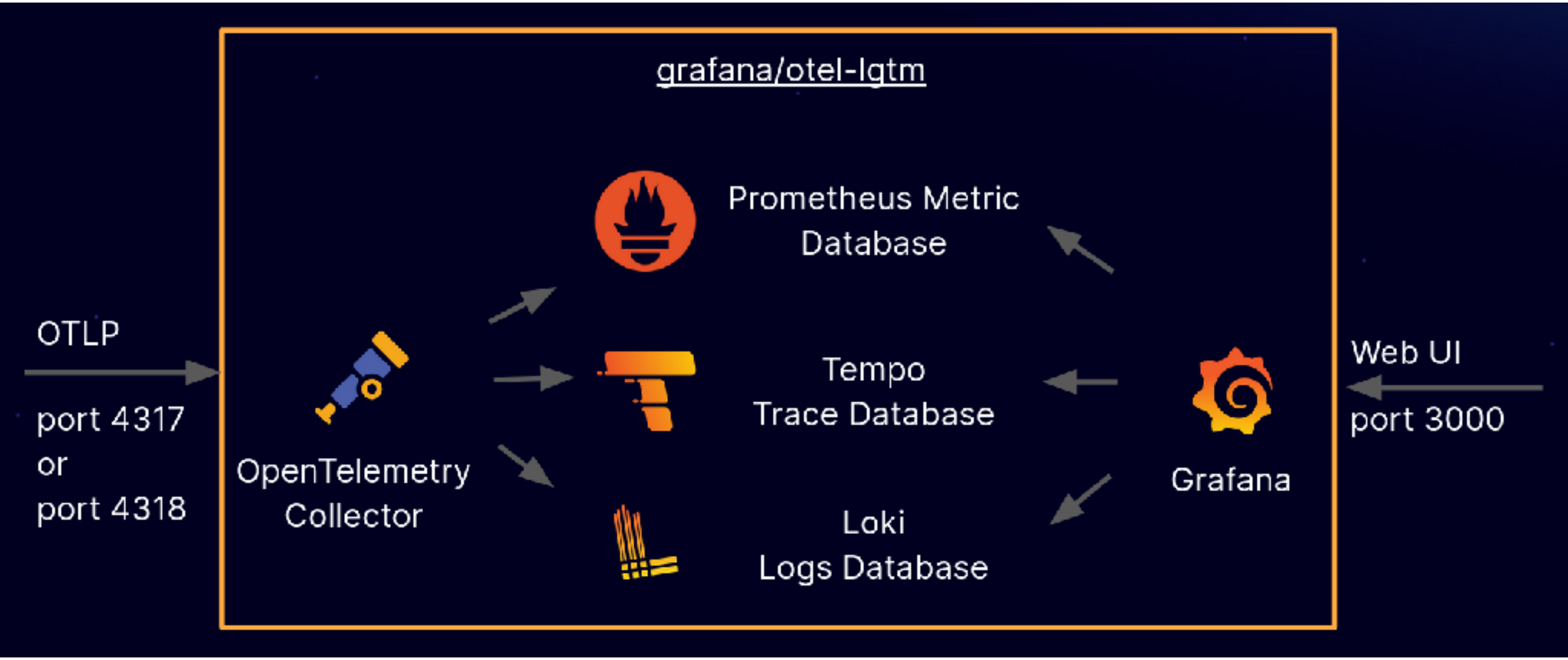
https://www.elastic.co/elastic-stack

ELK Stack



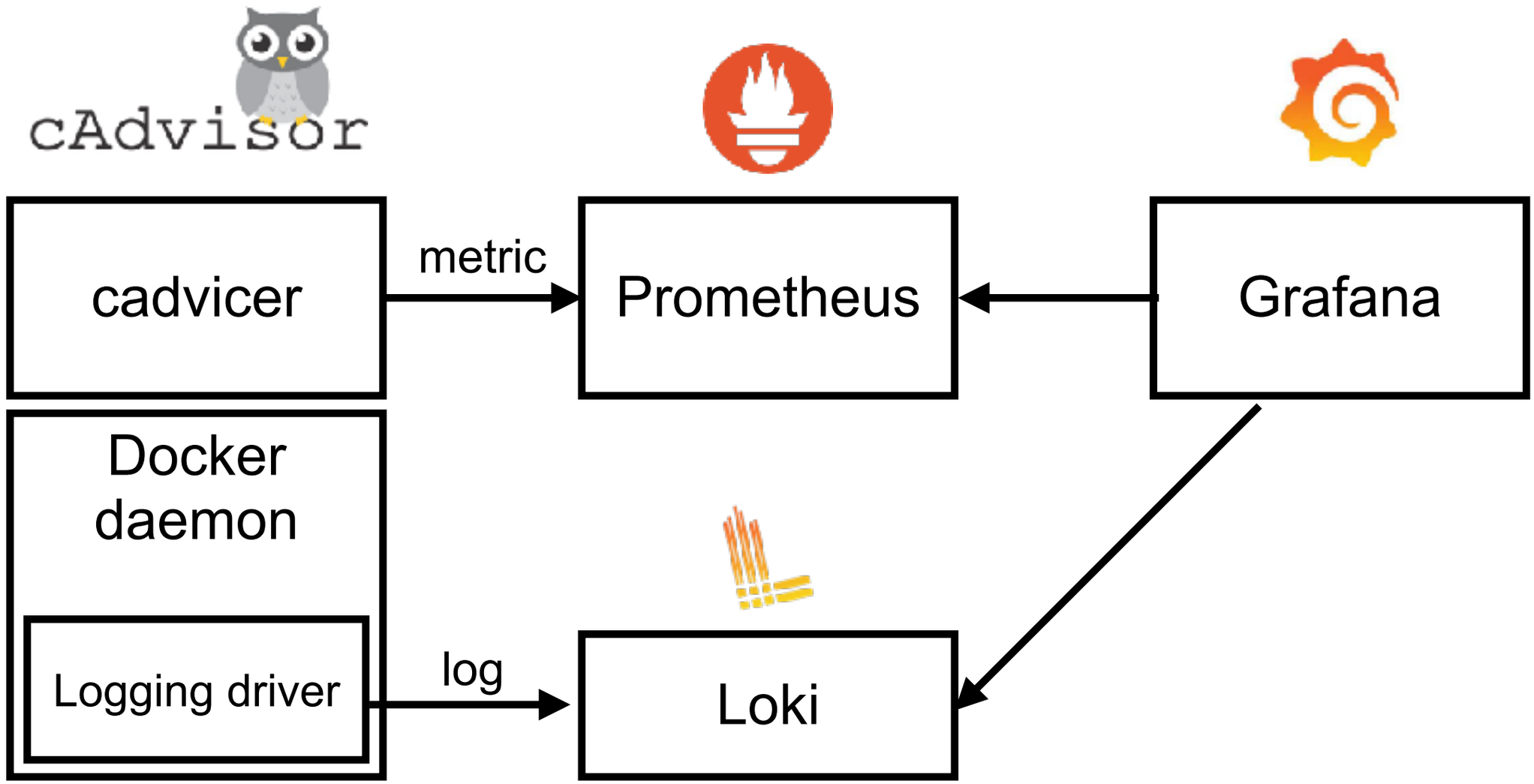
https://www.elastic.co/elastic-stack

LGTM Stack



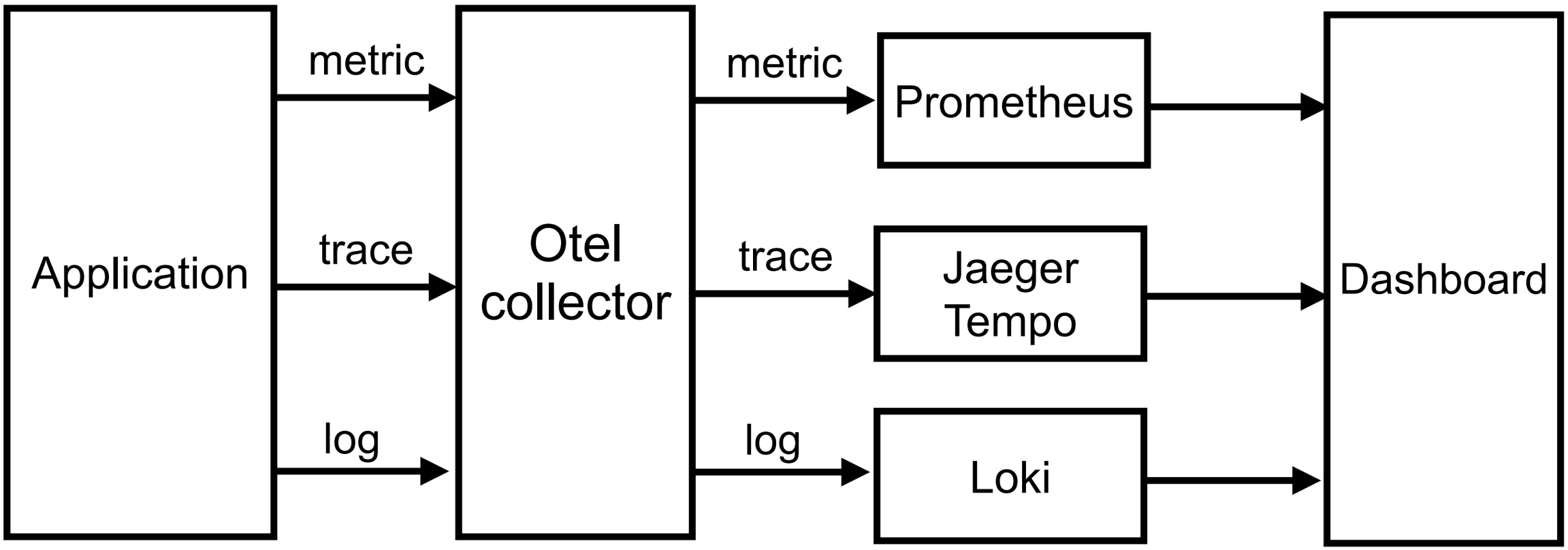
https://github.com/grafana/docker-otel-lgtm/

Docker monitoring



https://github.com/up1/workshop-docker-monitoring

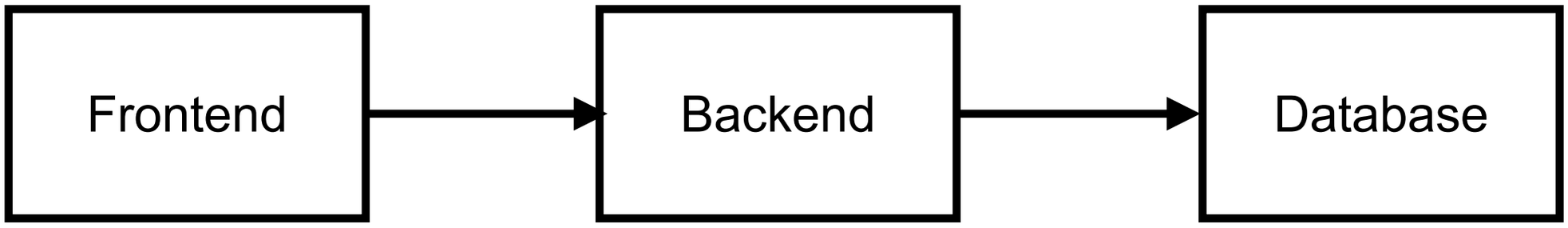
OpenTelemetry



https://github.com/up1/workshop-lgtm-go

Observability Workshop

Simple Architecture

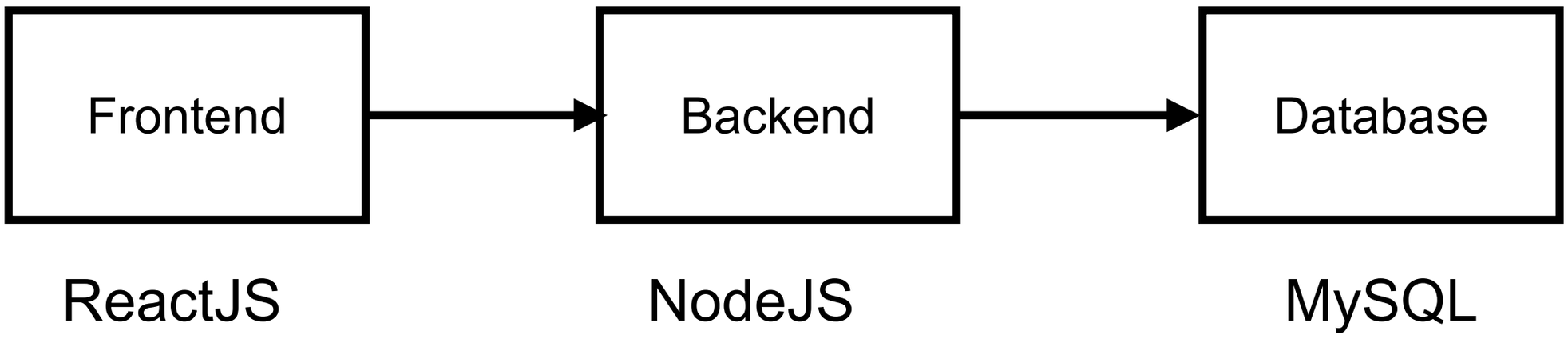


Delivery process

Working with Docker

https://github.com/up1/workshop-ci-nodejs-web-api

Simple Architecture



Delivery process

Working with Docker and Kubernetes

https://github.com/up1/workshop-ci-nodejs-web-api

Q/A